

LAPORAN PROYEK AKHIR

IMPLEMENTASI RANGKAIAN ELEKTRONIK DAN SISTEM KOMUNIKASI MULTI-PEMAIN UNTUK PERMAINAN *LASER TAG* MENGGUNAKAN ESP-NOW DAN LORA



**OLEH:
LATIFAN NURDIANSYAH
NIM: 244101077012**

**PROGRAM STUDI SARJANA TERAPAN
TEKNIK ELEKTRONIKA
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI MALANG
2026**

LAPORAN PROYEK AKHIR

IMPLEMENTASI RANGKAIAN ELEKTRONIK DAN SISTEM KOMUNIKASI MULTI-PEMAIN UNTUK PERMAINAN *LASER TAG* MENGGUNAKAN *ESP-NOW* DAN *LORA*



Oleh:

Latifan Nurdiansyah

NIM: 244101077012

**PROGRAM STUDI SARJANA TERAPAN
TEKNIK ELEKTRONIKA
JURUSAN TEKNIK ELEKTRO
POLITEKNIK NEGERI MALANG
2026**

HALAMAN PENGESAHAN
IMPLEMENTASI RANGKAIAN ELEKTRONIK DAN SISTEM
KOMUNIKASI MULTI-PEMAIN UNTUK PERMAINAN *LASER TAG*
MENGGUNAKAN *ESP-NOW* DAN *LoRa*

Oleh:

Latifan Nurdiansyah

244101077012

Proyek Akhir ini telah dipertahankan di depan dewan penguji pada tanggal April 2026 dan disahkan oleh :

Pembimbing I

Nama Pembimbing I : Indrazno Siradjuddin, S.T., M.T., Ph.D.
NIP. : 197406242000121001

Pembimbing II

Nama Pembimbing II : Gillang Al Azhar, S.S.T., M.Tr.T.
NIP. : 199506222020121003

Penguji I

Nama Penguji I :
NIP. :

Penguji II

Nama Penguji II :
NIP. :

Malang, April 2026

Mengetahui,
Ketua Jurusan Teknik Elektro

Menyetujui,
Koordinator Program Studi Sarjana
Terapan Teknik Elektronika

Prof. Ir. Mohammad Noor Hidayat, S.T., M.Sc., Ph.D.
NIP. 197409252001121003

Hari Kurnia Safitri, S.T., M.T.
NIP. 197307132002122002

PERNYATAAN KEASLIAN PROYEK AKHIR

Saya yang bertanda tangan di bawah ini:

Nama	:	Latifan Nurdiansyah
NIM	:	244101077012
Jurusan/Program Studi	:	Teknik Elektro/D4 Teknik Elektronika
Judul	:	Implementasi rangkaian elektronik dan sistem komunikasi multi-pemain untuk permainan laser tag menggunakan <i>esp-now</i> dan <i>LORA</i>

Menyatakan dengan sebenarnya bahwa:

1. Karya tulis ini benar – benar merupakan hasil karya saya sendiri, bukan merupakan alihan penelitian, tulisan atau pikiran orang lain yang saya akui sebagai hasil proyek akhir, tulisan atau pikiran saya sendiri.
2. Apabila di kemudian hari terbukti atau dapat dibuktikan bahwa karya tulis ini hasil jiplakan (plagiasi), dan saya tidak dapat memenuhi pernyataan saya ini maka saya bersedia menerima sanksi atas perbuatan saya tersebut.

Malang. April 2026

Latifan Nurdiansyah

ABSTRAK

Latifan Nurdiansyah, 2026. Implementation of Electronic Circuits and Multi-Player Communication Systems for Laser Tag Games Using ESP-NOW and LoRa

Proyek Akhir Program Studi Sarjana Terapan Teknik Elektronika, Jurusan Teknik Elektro, Politeknik Negeri Malang, 2026.

Kata Kunci:

KATA PENGANTAR

Puji syukur kehadiran Allah SWT yang telah melimpahkan rahmat serta hidayah-Nya, serta memberikan petunjuk serta kesehatan sehingga penulis dapat menyelesaikan Laporan Akhir yang berjudul “**Implementasi Rangkaian Elektronik dan Sistem Komunikasi Multi-Pemain untuk Permainan *Laser Tag* Menggunakan *ESP-NOW* dan *LoRa***”. Pada kesempatan ini penulis mengucapkan terimakasih kepada pihak-pihak yang telah membantu dan membimbing dalam penyusunan Laporan Akhir, yaitu kepada :

1. Bapak Ir. Supriatna Adhisuwignjo, S.T., M.T. selaku Direktur Politeknik Negeri Malang beserta jajarannya.
2. Bapak Prof. Ir. Mohammad Noor Hidayat, S.T., M.Sc., Ph.D. selaku Ketua Jurusan Teknik Elektro beserta jajarannya.
3. Ibu Hari Kurnia Safitri, ST., MT. selaku Ketua Program Studi Sarjana Terapan Teknik Elektronika beserta jajarannya.
4. Bapak Indrazno Siradjuddin, S.T., M.T., Ph.D. selaku Dosen Pembimbing 1, yang telah membimbing dan mendampingi sepenuh hati dari awal hingga akhir penelitian proyek akhir ini.
5. Bapak Gillang Al Azhar, S.S.T., M.Tr.T. selaku Dosen Pembimbing 2, yang telah membimbing dari awal hingga akhir penelitian proyek akhir ini.
6. Seluruh Dosen dan Staff Politeknik Negeri Malang, khususnya pada Program Studi Sarjana Terapan Teknik Elektronika.
7. Orang tua yang telah memberi dukungan do’a dan materi dan juga memberikan apapun yang aku butuhkan.
8. Teman Teman Lab Robotics Polinema yang dengan senang hati bertukar ilmu dan pikiran
9. Mahasiswa dengan NIM 2142520038 Jurusan Akutansi yang selalu memeberikan dukungan penulis dalam mengerjakan laporan ini dalama suka maupun duka sampai laporan ini selesai
10. Teman-teman mahasiswa dan pihak yang telah membantu dalam penyusunan Laporan Akhir ini.

Penulis menyadari masih banyak kekurangan dan pembuatan proyek akhir ini. Oleh karena itu, penulis mengharapkan saran dan kritik yang bersifat membangun dan semoga proyek akhir ini dapat bermanfaat.

Malang, April 2026

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN	i
PERNYATAAN KEASLIAN PROYEK AKHIR	ii
ABSTRAK	iii
KATA PENGANTAR.....	iv
DAFTAR ISI	vi
DAFTAR GAMBAR	ix
DAFTAR TABEL	xi
BAB I PENDAHULUAN	12
1.1 Latar Belakang	12
1.2 Rumusan Masalah	13
1.3 Batasan Masalah.....	13
1.4 Tujuan.....	14
1.5 Sistematika Penulisan.....	14
1.6 Luaran Laporan Proyek Akhir.....	15
BAB II TINJAUAN PUSTAKA	16
2.1 Kajian Pustaka.....	16
2.2 Teori Penunjang	19
2.2.1 Baterai Lithium Ion	19
2.2.2 Regulator Tegangan.....	20
2.2.3 Sensor Api	21
2.2.4 Sensor Suara.....	23
2.2.5 Sensor Getar	26
2.2.6 Operational Amplifier Komparator	29
2.2.7 Gerbang Logika NOR	29
2.2.8 Rangkaian Push Pull	30
2.2.9 Transmitter Inframerah	32
2.2.10 Modulasi dan Demodulasi Inframerah.....	34
2.2.11 Gelombang Inframerah	35

2.2.12	Nippon Electric Company Protocol	36
2.2.13	Jarak Transmisi Bahan Optik	36
2.2.14	Lensa Fresnel	38
2.2.15	Push Button	39
2.2.16	<i>Global Positioning System (GPS)</i>	40
2.2.17	Mikrokontroler	41
2.2.18	Protokol ESP-Now	43
2.2.19	Receiver Infrared Anti Cheating	44
2.2.20	LoraWan.....	48
2.2.21	RadioLib.....	50
2.2.22	Gateway.....	51
2.2.23	Heltec Tracker V1.1	52
2.2.24	The Things Network (TTN) Intregation	53
2.2.25	SIM900A.....	55
2.2.26	Modul Kartu SD.....	56
2.2.27	MP3 Player Module	57
2.2.28	Speaker.....	57
BAB III METODOLOGI		59
3.1	Kerangka Konsep Proyek Akhir.....	59
3.2	Perancangan Sistem	62
3.3	Perancangan Mekanik	64
3.3.1	Perancangan SAT	64
3.3.2	Perancangan Box pada Helm	66
3.3.3	Perancangan pada Rompi.....	66
3.4	Perancangan Elektronik	68
3.4.1	Perancangan Rangkaian Power Source Regulator Circuit.....	68
3.4.2	Perancangan Rangkaian Pemancar Inframerah.....	70
3.4.3	Perancangan Rangkaian Helm dengan Sensor Penerima Inframerah dan Mekanisme Anti-Cheating.....	82
3.4.4	Perancangan Rangkaian Vest (Penerima Infrared, Anti-Cheating, dan Transmisi Data)	85

3.5	Perancangan Software.....	90
BAB IV PENGUJIAN DAN ANALISA		92
4.1	Pengujian perblok.....	92
4.1.1	Pengujian Rangkaian Power Regulator.....	92
4.1.2	Pengujian Rangkaian Pemancar Inframerah	93
4.1.3	Pengujian Rangkaian Penerima Inframerah.....	98
4.1.4	Pengujian Heltec Tracker V1.2	100
4.1.5	Pengujian Keseluruhan.....	104
BAB V.....		106
5.1	Kesimpulan	106
5.2	Saran.....	107
DAFTAR PUSTAKA.....		109
LAMPIRAN.....		114

DAFTAR GAMBAR

Gambar 1 Regulator LM7805	21
Gambar 2 Rangkaian Sensor Api	22
Gambar 3 Rangkaian Sensor Suara.....	25
Gambar 4 Rangkaian sensor getar.....	27
Gambar 5 Op-Amp.....	29
Gambar 6 Logika NOR	30
Gambar 7 Rangkaian Dasar Push-Pull.....	31
Gambar 8 Infrared LED	33
Gambar 9 Panjang Gelombang	35
Gambar 10 Transmisi Bahan Optik.....	37
Gambar 11 Lensa Freshnel.....	38
Gambar 12 Global Positioning System (GPS)	41
Gambar 13 ESP32	42
Gambar 14 TSOP43..	45
Gambar 15 Sudut penerimaan sensor TSOP4838	46
Gambar 16 Grafik Panjang Gelombang.....	48
Gambar 17 Kerangka Konsep penyusunan proyek akhir.....	60
Gambar 18 Perancangan Sistem.....	63
Gambar 19 Perancangan SAT	64
Gambar 20 Focal Length Lensa Fresnel	65
Gambar 21 Perancangan Helm.....	66
Gambar 22 Perancangan Box pada Rompi	67
Gambar 23 Design Rompi.....	67
Gambar 24 Power Source Regulator.....	68
Gambar 25 Rangkaian sensor suara	70
Gambar 26 Rangkaian Sensor Getar	72
Gambar 27 Rangkaian Sensor Api	73
Gambar 28 Rangkaian pengkondisi sinyal.....	75

Gambar 29 Rangkaian Pengendali	76
Gambar 30 Rangkaian Driver MOSFET dengan IC MCP1406 untuk Pengendalian Beban.....	77
Gambar 31 Rangkaian sederhana IR LED	80
Gambar 32 Gambar Rangkaian keseluruhan Infrared Driver	81
Gambar 33 Rangkaian Penerima Infrared Anti-Cheating	82
Gambar 34 Flowchart.....	90

DAFTAR TABEL

Tabel 1 Spesifikasi ESP32	43
Tabel 2 Spesifikasi TSOP43.....	45
Tabel 3 Asumsi arus	69
Tabel 4 Pengujian LM7805	93
Tabel 5 Pengujian Sensor Suara	93
Tabel 6 Pengujian Sensor Getar	94
Tabel 7 Pengujian Sensor Api	94
Tabel 8 Pengujian IC64HC4078	96
Tabel 9 Tabel Kebanaran Logika NOR	96
Tabel 10 Pengujian ESP-NOW	97
Tabel 11 Pengujian Jarak Penerimaan.....	98
Tabel 12 Pengujian Sudut Penerimaan.....	98
Tabel 13 Pengujian Anti-Cheating	99
Tabel 14 Pengujian ESPNOW	100
Tabel 15 Pegujian GPS.....	101
Tabel 16 Pengujian GSM	102
Tabel 17 Pengujian LoRa	103
Tabel 18 Pengujian Unit 2.....	104
Tabel 19 Pengujian Unit 1	105

BAB I

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi digital telah membawa perubahan signifikan dalam dunia hiburan interaktif, termasuk pada jenis permainan simulasi fisik seperti *Laser Tag*. Permainan ini menggabungkan aktivitas fisik, strategi tim, dan penggunaan perangkat elektronik berbasis sensor untuk menciptakan pengalaman bermain yang imersif. Namun, sistem *Laser Tag* komersial yang tersedia saat ini umumnya masih bersifat tertutup dan terbatas: deteksi tembakan sering kali kurang akurat karena rentan terhadap gangguan cahaya latar, tidak tersedia informasi posisi pemain secara real-time, serta tidak mendukung komunikasi data antar pemain maupun ke pusat kendali. Kondisi ini menghambat pengembangan sistem yang lebih adaptif, terukur, dan dapat digunakan tidak hanya untuk hiburan, tetapi juga untuk pelatihan atau kompetisi terstruktur.

Beberapa penelitian sebelumnya telah mengembangkan sistem berbasis sinar inframerah (IR) untuk simulasi pertempuran, seperti *Multiple Integrated Laser Engagement System* (MILES), yang mampu mengidentifikasi pemain melalui kode digital dan memberikan umpan balik saat terkena tembakan (Siradjuddin et al., 2024). Meski efektif, sistem semacam ini biasanya membutuhkan perangkat keras mahal, konsumsi daya tinggi, dan infrastruktur yang kompleks—sehingga kurang layak untuk diterapkan dalam skenario rekreasi atau komunitas dengan anggaran terbatas.

Perkembangan teknologi *Internet of Things* (IoT) membuka peluang baru untuk merancang sistem *Laser Tag* yang lebih terjangkau, modular, dan terhubung. Dua protokol komunikasi nirkabel yang menarik untuk diintegrasikan adalah ESP-NOW dan LoRa. ESP-NOW, yang berjalan pada mikrokontroler ESP32, memungkinkan komunikasi *peer-to-peer* berkecepatan tinggi dengan latensi sangat rendah tanpa memerlukan jaringan Wi-Fi (Hailan et al., 2024). Protokol ini ideal untuk mengirimkan data tembakan antar pemain dalam jarak dekat (hingga ± 30 meter), seperti dari helm ke rompi atau antar rompi pemain yang saling berinteraksi. Namun, jangkauannya terbatas, sehingga tidak memadai untuk arena permainan skala luas..

Di sisi lain, teknologi LoRa (Long Range) menawarkan kemampuan transmisi data hingga beberapa kilometer dengan konsumsi daya sangat rendah (Konate et al., 2025; Yahya et al., 2024). Dengan LoRa, data permainan—seperti skor, posisi berdasarkan modul GPS, atau status nyawa—dapat dikirim ke *gateway* pusat dan ditampilkan melalui antarmuka web, memungkinkan pemantauan real-time oleh wasit, pelatih, atau penonton. Kombinasi kedua protokol ini menciptakan arsitektur komunikasi dua lapis: ESP-NOW untuk interaksi cepat antar pemain, dan LoRa untuk integrasi ke sistem eksternal jarak jauh. Berdasarkan latar belakang maka, pada proyek akhir ini akan dikembangkan sistem Permainan Laser Tag Menggunakan ESP-NOW dan LoRa. Sistem yang dirancang terdiri dari helm dan rompi yang dilengkapi sensor infrared, modul GPS, serta modul komunikasi berbasis ESP32. Data dari hasil deteksi maupun posisi pemain akan dikirim melalui ESP-NOW dan LoRa ke server, lalu ditampilkan dalam bentuk web interface secara real-time.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, dapat dirumuskan beberapa masalah sebagai berikut:

- a. Bagaimana mengimplementasikan rangkaian elektronik pada sistem tembak (senjata) serta rangkaian penerima (helm dan rompi), dan membangun sistem komunikasi dua arah antara server dan banyak unit pemain?
- b. Bagaimana mengintegrasikan protokol ESP-NOW dan LoRa pada mikrokontroler untuk mendukung komunikasi real-time dalam permainan Laser Tag?
- c. Bagaimana mengimplementasikan sistem komunikasi dua arah untuk mendukung pencatatan skor secara *real-time*?

1.3 Batasan Masalah

Untuk memfokuskan proyek akhir, diberikan pembatasan masalah sebagai berikut:

- a. Komunikasi nirkabel hanya menggunakan ESP-NOW (jarak dekat antar helm dan rompi) dan LoRa (jarak jauh ke server), dengan sistem terbatas pada maksimal 2 klien dan 1 server, serta hanya mengirim data status tanpa audio, video, atau enkripsi tingkat lanjut

- b. Sistem hanya menggunakan sensor inframerah dengan modulasi 38 kHz sesuai protokol NEC, termasuk rangkaian IR receiver anti-cheating yang dirancang untuk beroperasi di siang hari, dan khusus diterapkan dalam permainan laser tag.
- c. Sumber daya dibatasi pada baterai lithium-ion untuk senjata, rompi, dan helm, tanpa membahas alternatif sumber daya, pengisian daya, lokasi minim sinyal seluler, maupun aspek keamanan siber atau enkripsi data.

1.4 Tujuan

Berdasarkan rumusan masalah yang telah ditetapkan terdapat beberapa tujuan dari proyek akhir ini. Berikut merupakan tujuan dari proyek akhir ini:

- a. Mengimplementasikan rangkaian elektronik pada sistem tembak (senjata) serta rangkaian penerima (helm dan rompi) dan sistem komunikasi dua arah antara server dan banyak unit pemain
- b. Mengintegrasikan protokol ESP-NOW dan LoRa pada mikrokontroler untuk mendukung komunikasi real-time dalam permainan Laser Tag.
- c. Mengembangkan sistem komunikasi dua arah untuk mendukung pencatatan skor secara *real-time*.

1.5 Sistematika Penulisan

Pembahasan dalam proyek akhir ini disusun secara terstruktur dalam lima bab utama, yang masing-masing memiliki fokus tersendiri sesuai dengan alur proyek akhir dan pengembangan sistem. Sistematika penulisan pada laporan proyek akhir ini :

Bab I Pendahuluan

Bab ini berisi latar belakang pemilihan topik proyek akhir, rumusan masalah beserta batasan masalah yang diteliti. Selain itu, juga dijelaskan tujuan proyek akhir serta manfaat yang ingin dicapai. Bab ini juga memberikan gambaran umum mengenai struktur penulisan laporan proyek akhir.

Bab II Tinjauan Pustaka

Bab ini membahas materi teoretis yang relevan dengan proyek akhir, termasuk konsep-konsep dasar, teknik, peralatan, atau teknologi yang digunakan dalam proses proyek akhir. Selain itu, juga disertakan referensi pustaka dan hasil proyek akhir sebelumnya yang menjadi dasar pengembangan sistem.

Bab III Metodologi

Bab ini menjelaskan secara rinci pendekatan atau metode yang digunakan dalam melaksanakan proyek akhir. Meliputi desain sistem, tahapan pengembangan, alur kerja, serta langkah-langkah yang dilakukan untuk mencapai tujuan proyek akhir.

Bab IV Hasil Dan Pembahasan

Pada bab ini disajikan hasil implementasi sistem, data pengujian perblok dan keseluruhan, serta analisis yang dilakukan berdasarkan metode proyek akhir yang telah ditetapkan. Pembahasan bertujuan untuk mengevaluasi kinerja sistem dan membandingkannya dengan tujuan serta hipotesis awal.

Bab V Penutup

Bab terakhir ini memuat kesimpulan utama dari seluruh rangkaian proyek akhir, berdasarkan hasil dan pembahasan yang telah diuraikan. Selain itu, juga disertakan saran-saran untuk pengembangan sistem pada proyek akhir lanjutan.

1.6 Luaran Laporan Proyek Akhir

Proyek akhir ini memiliki 4 luaran yaitu

- a. Naskah Laporan proyek akhir
- b. Alat Sistem berupa senjata, helm dan rompi
- c. Manual Book
- d. Artikel yang disubmit di jurnal yang ber-ISSN.

BAB II

TINJAUAN PUSTAKA

2.1 Kajian Pustaka

Berikut adalah beberapa hasil penelitian terdahulu yang relevan dengan topik proyek akhir ini dan dapat digunakan sebagai landasan dan tinjauan pustaka untuk mendukung acuan proyek akhir ini sebagai berikut.

Dalam dunia pelatihan militer modern, simulasi tempur yang realistis dan aman menjadi kebutuhan utama untuk meningkatkan kesiapan tempur tanpa harus mengorbankan nyawa atau menggunakan amunisi aktif. Salah satu teknologi yang telah lama dikembangkan dan digunakan secara luas untuk tujuan ini adalah Multiple Integrated Laser Engagement System (MILES). Sistem ini memungkinkan simulasi pertukaran tembakan antar personel atau kendaraan tempur dengan menggunakan pancaran cahaya infrared (IR) sebagai pengganti peluru. Pada sistem MILES, setiap senjata dilengkapi dengan perangkat yang disebut Small Arm Transmitter (SAT), yang memancarkan sinyal infrared saat pelatih menembakkan senjata. Sinyal ini kemudian dideteksi oleh sensor IR pada lawan, yang mencatat "kena tembak" berdasarkan data identitas pemain (Player ID/PID) yang turut dikodekan dalam sinyal tersebut. Oleh karena itu, performa SAT sangat menentukan keberhasilan simulasi, termasuk jangkauan deteksi, akurasi respon waktu, dan reliabilitas transmisi data. (Siradjuddin et al., 2024).

Salah satu komponen krusial dalam SAT adalah infrared (IR) emitter driver circuit, yang mengendalikan arus listrik ke LED infrared berdaya tinggi, memastikan pancaran cahaya IR dengan intensitas optimal dan waktu respons yang cepat. Desain rangkaian ini menjadi fokus utama dalam proyek akhir (Siradjuddin et al., 2024), yang memperkenalkan rancangan baru sirkuit penggerak emitter infrared khusus untuk aplikasi SAT dalam sistem MILES. Tujuan dari desain ini adalah untuk memaksimalkan daya optik dan memastikan frekuensi pensaklaran 38 kHz yang sesuai dengan standar protokol komunikasi IR NEC, yang menjadi kunci dalam efektivitas komunikasi infrared. (Siradjuddin et al., 2024)

Menurut jurnal *Wireless Infrared Communications: A Survey*, komunikasi infrared memanfaatkan propagasi cahaya pada pita near-infrared dengan pemancar

berupa LED atau laser diode dan penerima berupa silicon p-i-n photodiode karena efisiensinya yang baik dan biaya rendah. Sistem ini menggunakan teknik Intensity Modulation with Direct Detection (IM/DD), di mana arus fotodetektor sebanding dengan intensitas optik yang diterima. Namun, penerimaan sinyal dapat terganggu oleh sumber cahaya sekitar seperti sinar matahari atau lampu, sehingga diperlukan filter optik dan pemilihan modulasi yang tepat untuk menjaga kualitas sinyal. Sementara itu, menurut jurnal *Wireless Infrared Communications*, komunikasi infrared menawarkan keunggulan dibanding radio, seperti bandwidth yang sangat besar, biaya rendah, serta keamanan tinggi karena sinyal tidak menembus dinding. Selain itu, penggunaan IM/DD dengan detektor berukuran besar mencegah terjadinya multipath fading, sehingga desain sistem lebih sederhana. Meski begitu, keterbatasan komunikasi infrared adalah jangkauannya yang terbatas, kebutuhan daya pancar relatif tinggi, serta sensitivitas terhadap noise cahaya sekitar. (Kahn & Barry, 1997; Zade et al., 2012)

Menurut jurnal *Penerapan Protokol Komunikasi ESP-NOW pada Portable Traffic Light*, implementasi komunikasi berbasis ESP-NOW dengan skema master-slave mampu bekerja stabil dengan tingkat error rata-rata hanya 3% dan jangkauan optimal hingga 25 meter meskipun terdapat halangan berupa casing modul. Sementara itu, pada jurnal *ESPNow Protocol-Based IIoT System for Remotely Monitoring and Controlling Industrial Systems*, ESP-NOW dimanfaatkan untuk menghubungkan beberapa node sensor dalam sistem Industrial Internet of Things (IIoT), yang terbukti mendukung pemantauan industri secara real-time dengan latensi rendah, konsumsi daya hemat, serta keamanan data yang terjamin melalui enkripsi. Di sisi lain, proyek akhir berjudul *Indoor Performance Evaluation of ESP-NOW* mengevaluasi performa ESP-NOW dalam kondisi indoor dan menemukan bahwa meskipun faktor lingkungan seperti pantulan dan penghalang dapat memengaruhi kinerja, protokol ini tetap menunjukkan efisiensi daya lebih dari 30% dibanding WiFi serta jangkauan yang lebih baik hingga 15%. Berdasarkan temuan-temuan tersebut, ESP-NOW sangat relevan untuk digunakan pada sistem permainan Laser Tag, di mana komunikasi antar perangkat harus berlangsung cepat, stabil, dan hemat daya dalam area permainan. (Fajar Arofah et al., 2023; Hailan et al., 2024; Urazayev et al., 2023)

Menurut proyek akhir *Design and Implementation of a Backscatter Communication System Based on LoRa Technology*, LoRa backscatter mampu meningkatkan efisiensi energi dan memperluas jangkauan komunikasi dengan memanfaatkan sinyal pantulan dari perangkat pasif, sehingga sangat potensial untuk aplikasi IoT berskala besar. Sementara itu, dalam jurnal *Analisa Performansi Komunikasi LoRa (Long Range) pada Sistem Monitoring Buoy di Laut*, komunikasi LoRa 915 MHz terbukti dapat menjaga kualitas dengan nilai RSSI rata-rata lebih baik dari -120 dB dan SNR lebih besar dari -20 dB meskipun terdapat interferensi, sehingga layak digunakan untuk sistem monitoring di lingkungan laut. Selanjutnya, hasil proyek akhir *Analisis Jarak Jangkauan LoRa (Long Range) di Lingkungan Urban*, menunjukkan bahwa jangkauan LoRa di area perkotaan dapat mencapai hingga 2 km dengan nilai RSSI relatif stabil, meskipun packet loss meningkat pada jarak yang lebih jauh. Dari ketiga kajian tersebut dapat disimpulkan bahwa LoRa tidak hanya handal untuk komunikasi jarak jauh dengan konsumsi energi rendah, tetapi juga fleksibel untuk diterapkan dalam berbagai skenario, baik di laut, lingkungan urban, maupun melalui dukungan teknologi backscatter. (Konate et al., 2025; Nihayatus Sa'adah et al., 2024; Yanziah et al., 2020)

Global Positioning System (GPS) merupakan sistem navigasi berbasis satelit yang terdiri dari 24 satelit operasional yang mengorbit Bumi dan menyediakan informasi posisi secara kontinu melalui tiga segmen utama, yaitu segmen pengguna, ruang angkasa, dan kontrol, dimana menurut jurnal *"Implementation of GPS for Location Tracking"*, data yang diterima berupa format NMEA-0183 yang berisi koordinat latitude dan longitude yang perlu diekstrak untuk memperoleh data posisi. Menurut jurnal *"Missing Pilgrims Tracking System Using GPS, GSM and Arduino Microcontroller"*, GPS merupakan sistem navigasi universal yang dapat bekerja dalam segala kondisi cuaca dan dapat dikombinasikan dengan teknologi GSM untuk menghasilkan sistem pelacakan berbasis peta digital, sementara jurnal *"A GPS Receiver for High-Altitude Satellite Navigation"* mengatakan bahwa GPS telah digunakan untuk navigasi satelit presisi di orbit rendah Bumi (LEO), meskipun penerapannya di orbit geosinkron (GEO) masih terbatas akibat lemahnya sinyal. Lebih lanjut, GPS merupakan bagian dari GNSS (Global Navigation Satellite System) bersama dengan GLONASS, BeiDou, dan

Galileo, dengan akurasi 3–10 meter di lingkungan outdoor, namun menurun menjadi 30–100 meter di indoor sehingga mendorong pengembangan Indoor Positioning System (IPS) berbasis RTK dengan akurasi hingga 2 cm atau teknologi UWB dengan akurasi 20 cm, serta aspek waktu yang penting dimana distribusi UTC melalui sinyal GPS memiliki akurasi ≤ 30 nanodetik. (Ariffin et al., 2011; Itera, 2021; Parveen & Aldhlan, 2016; Winternitz et al., 2009)

Menurut proyek akhir *Web Server Based Smart Agriculture System for Real-Time Monitoring and Control*, pemanfaatan web server dalam integrasi IoT memungkinkan penyajian data sensor secara real-time melalui dashboard interaktif yang mendukung sistem pertanian cerdas. Hal ini sejalan dengan studi *A Web Server-Based IoT System for Monitoring and Control of Agricultural Environment*, yang menekankan pentingnya akses global dan pengelolaan data terdistribusi secara aman melalui jaringan berbasis web. Lebih lanjut, proyek akhir *Real-Time Monitoring of Cable Sag and Overhead Line Parameters Based on a Distributed Sensor Network and Implementation in a Web Server and IoT*, menunjukkan bagaimana web server dipadukan dengan database MySQL dan IoT server untuk memantau kondisi kabel transmisi listrik secara aman, baik melalui intranet maupun internet. Sementara itu, jurnal *Web-Based Integration of IoT and Artificial Intelligence for Monitoring Aquariums*, menegaskan bahwa dashboard web mampu membangun visualisasi interaktif yang membantu pengawasan kondisi lingkungan, seperti suhu, pH, dan kecerahan air pada sistem akuarium pintar. Dari keempat kajian tersebut, dapat disimpulkan bahwa web server berperan penting dalam integrasi IoT, tidak hanya untuk pemantauan data sensor secara real-time, tetapi juga dalam mendukung keamanan akses, keterhubungan global, serta pengembangan dashboard interaktif yang relevan untuk berbagai bidang, mulai dari pertanian, energi, hingga sistem hiburan seperti permainan Laser Tag. (Deepika & Renuka Prasad, 2023; Guerbaoui et al., 2025; Nicola et al., 2024; Nugraha & Rosita, 2024).

2.2 Teori Penunjang

2.2.1 Baterai Lithium Ion

Baterai *Lithium-Ion* (Li-ion) merupakan salah satu jenis sel baterai sekunder (*rechargeable*) yang paling banyak digunakan dalam sistem elektronika portabel

dan perangkat berbasis mikrokontroler. Keunggulannya terletak pada rapat energi yang tinggi, konsumsi daya rendah, dan ukuran yang kompak (Lesmana et al., 2023). Dalam aplikasi sistem *embedded* seperti yang dirancang pada proyek akhir ini, baterai *Lithium-Ion* sering dikonfigurasi dalam bentuk pack seri untuk menghasilkan tegangan yang sesuai dengan kebutuhan sistem. Misalnya, konfigurasi 2S (2 seri) menghasilkan tegangan nominal sekitar 7,4 V, yang dapat digunakan untuk menyuplai daya ke mikrokontroler, motor, modul komunikasi, dan sensor melalui regulator tegangan (*step-down converter*). Dengan rumus pemakaian sebagai berikut

$$\text{Kapasitas Baterai (mAh)} = \text{Arus (mA)} \times \text{Waktu (jam)}$$

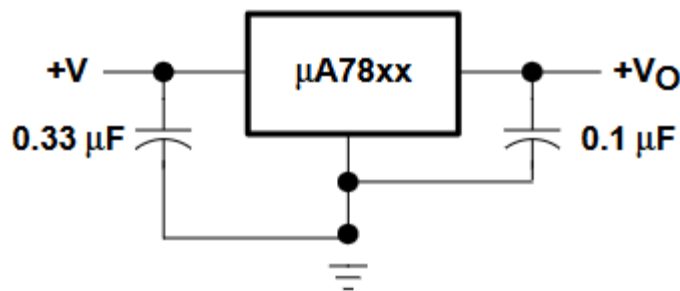
Dalam proyek permainan laser tag ini baterai lithium ion digunakan sebagai catu daya pada senjata, helm dan juga rompi. Pemilihan baterai ini didasarkan pada kemampuannya menyediakan energi secara efisien dengan bobot yang ringan, sehingga memungkinkan perangkat beroperasi secara mandiri di lapangan tanpa ketergantungan pada sumber daya eksternal. Hal ini sangat penting dalam mendukung mobilitas tinggi dan fleksibilitas gerak selama permainan berlangsung.

2.2.2 Regulator Tegangan

Regulator tegangan adalah perangkat yang digunakan untuk menjaga kestabilan tegangan pada suatu sistem kelistrikan, meskipun terjadi fluktuasi pada tegangan masukan. Fungsi utama regulator tegangan adalah untuk memastikan tegangan output tetap pada nilai yang diinginkan, melindungi perangkat elektronik atau sistem yang sensitif terhadap perubahan tegangan yang ekstrem. Regulator tegangan dapat dibedakan menjadi dua jenis utama, yaitu regulator linear dan regulator switching. Regulator linear menurunkan tegangan dengan cara mengalirkan arus melalui komponen pengatur, namun sering menghasilkan banyak panas dan kurang efisien. Sementara regulator switching menggunakan metode pemutusan dan penyambungan untuk mengatur tegangan dengan lebih efisien, menghasilkan sedikit energi terbuang.

Prinsip kerja regulator tegangan didasarkan pada kemampuan untuk mengontrol aliran listrik sesuai dengan perbedaan antara tegangan masukan dan tegangan output yang diinginkan. Regulator linear mengurangi tegangan melalui komponen elektronik seperti transistor, Rangkaian dasar regulator tegangan yang

digunakan dalam sistem ini ditunjukkan pada Gambar 1 regulator ini mampu menghasilkan tegangan keluaran stabil pada 5 V dengan toleransi antara 4,75 V hingga 5,25 V, meskipun terdapat variasi pada tegangan masukan (7–25 V) maupun perubahan arus beban (5 mA hingga 1,5 A). Regulator ini memiliki tingkat penolakan ripple yang tinggi (62–78 dB), resistansi output sangat rendah (sekitar $0,017\ \Omega$), serta noise output kecil (sekitar $40\ \mu\text{V}$), sehingga mampu memberikan suplai daya yang bersih untuk rangkaian sensitif. Selain itu, dropout voltage sekitar 2 V menunjukkan bahwa tegangan masukan minimal harus lebih besar dari 7 V untuk menghasilkan 5 V stabil pada keluaran. Dengan adanya proteksi arus pendek hingga 750 mA serta kemampuan arus puncak hingga 2,2 A, LM7805 menjadi regulator tegangan yang handal untuk berbagai aplikasi elektronika..



Gambar 1 Regulator LM7805
Sumber: Datasheet LM7805(Texas, 1976)

2.2.3 Sensor Api

Sensor api merupakan komponen kritis dalam sistem deteksi kebakaran modern, terutama dalam aplikasi yang memerlukan respons cepat terhadap keberadaan nyala api. Berbeda dengan sensor asap atau sensor suhu yang mengandalkan perubahan kondisi lingkungan sekunder, sensor api mendeteksi radiasi elektromagnetik yang dipancarkan langsung oleh nyala api—khususnya pada rentang inframerah dekat (NIR) antara 760 nm hingga 1100 nm (A132002 Datasheet, n.d.). Modul sensor api yang umum digunakan dalam sistem embedded, seperti yang berbasis fototransistor YG1006 dan komparator LM393, menawarkan solusi berbiaya rendah dengan keandalan tinggi untuk aplikasi deteksi dini kebakaran.

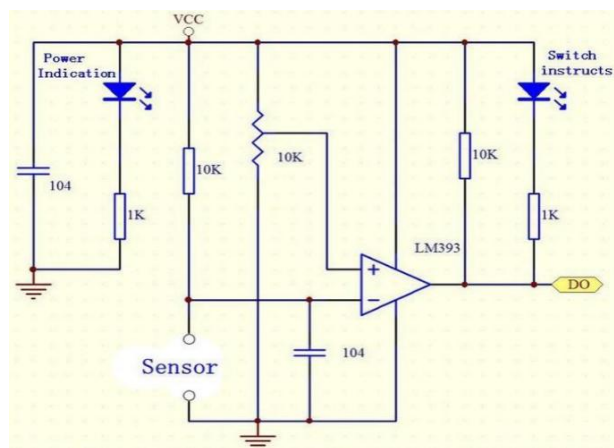
Sensor ini memanfaatkan fototransistor YG1006, yang sensitif terhadap radiasi inframerah dalam rentang 760–1100 nm—rentang spektral di mana

nyala api karbon-hidrokarbon (seperti api kayu, gas alam, atau bensin) memancarkan intensitas maksimum (Khan, 2023). Ketika radiasi inframerah mengenai basis fototransistor, pasangan elektron-lubang dihasilkan, meningkatkan arus kolektor-emitor secara proporsional terhadap intensitas cahaya yang diterima.

Modul ini menyediakan dua jenis output:

- Analog Output (AO): tegangan kontinu yang merepresentasikan intensitas radiasi inframerah,
- Digital Output (DO): sinyal biner yang dihasilkan oleh komparator LM393 berdasarkan perbandingan antara sinyal analog dan ambang referensi yang diatur melalui potensiometer.

Dalam kondisi normal (tanpa api), output digital berada pada logika HIGH; ketika intensitas radiasi melebihi ambang batas, output berubah menjadi LOW. Indikator LED hijau menyala sebagai konfirmasi deteksi.



Gambar 2 Rangkaian Sensor Api

Sumber: (Khan, 2023)

Pada rangkaian diatas merupakan sistem deteksi analog-digital berbasis komparator op-amp LM393, yang membandingkan tegangan dari sensor resistif dengan tegangan referensi tetap sebesar setengah VCC. Tegangan referensi (V_{ref}) dihasilkan oleh pembagi tegangan dua resistor 10 k Ω , sehingga dapat dirumuskan sebagai berikut:

$$V_{ref} = \frac{V_{cc}}{2} \quad (1.0)$$

Sementara itu, tegangan input non-inverting (V_+) berasal dari pembagi tegangan antara sensor dan resistor $10\text{ k}\Omega$, dirumuskan sebagai

$$V_+ = V_{cc} \times \frac{10\text{ k}\Omega}{R_{\text{sensor}} + 10\text{ k}\Omega} \quad (1.1)$$

Output komparator akan HIGH (mendekati VCC) jika $V_+ > V_-$, dan LOW (mendekati 0V) jika $V_+ < V_-$. Titik pemicu (threshold) terjadi ketika $V_+ = V_- = 2.5\text{V}$ (dengan asumsi $V_{CC} = 5\text{V}$), yang secara matematis menghasilkan nilai resistansi sensor kritis sebesar $R_{\text{sensor}} = 10\text{ k}\Omega$. Artinya, rangkaian akan “menyala” (output HIGH) ketika resistansi sensor turun di bawah $10\text{ k}\Omega$ — misalnya, saat cahaya meningkat pada LDR atau suhu naik pada NTC thermistor.

Output digital (DO) dari komparator dapat langsung dihubungkan ke mikrokontroler untuk pengolahan lebih lanjut, sementara LED indikator memberikan visualisasi status secara real-time. Kapasitor 104 ($0.1\text{ }\mu\text{F}$) berfungsi sebagai decoupling dan filter noise, menstabilkan suplai daya dan meminimalkan gangguan pada input sensor. Namun, karena tidak ada hysteresis (umpan balik positif), rangkaian ini rentan terhadap osilasi output akibat fluktuasi kecil pada sinyal sensor.

2.2.4 Sensor Suara

Sensor suara sering menggunakan teknologi piezoelektrik atau elektret untuk mendeteksi perubahan tekanan akibat gelombang suara dan menghasilkan sinyal listrik yang sesuai. Dalam prinsip kerjanya, sensor ini mendeteksi getaran suara dan mengonversinya menjadi tegangan listrik melalui efek piezoelektrik atau lapisan dielektrik permanen pada mikrofon elektret. Sinyal yang dihasilkan oleh sensor suara umumnya sangat lemah, sehingga sering memerlukan amplifikasi sebelum diproses lebih lanjut oleh mikrokontroler seperti ESP32.

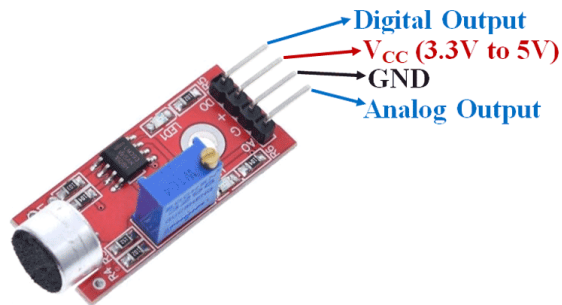
Sensor suara memiliki berbagai aplikasi, termasuk deteksi aktivitas dalam sistem keamanan, kontrol perangkat elektronik berbasis suara, perekaman audio, dan bahkan penggunaan dalam sistem senjata otomatis. Menurut Brown et al. (2019), sensor piezoelektrik sangat sensitif terhadap getaran dan cocok untuk aplikasi deteksi suara pada jarak dekat, sementara mikrofon elektret lebih sering

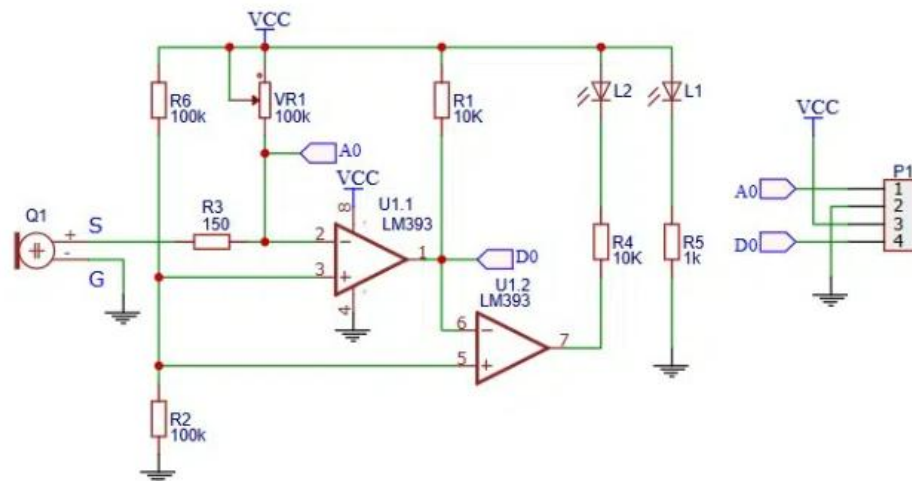
digunakan dalam aplikasi elektronika modern karena keluarannya yang stabil dan rendah impedansi.

Beberapa spesifikasi umum sensor suara yang sering digunakan:

- Rentang Frekuensi: Biasanya berkisar dari 20 Hz hingga 20 kHz (rentang frekuensi suara manusia).
- Sensitivitas: Diukur dalam dBV/Pa (*decibel volt per pascal*), yang menunjukkan respons sensor terhadap intensitas suara.
- Impedansi *Output*: Biasanya rendah (sekitar 2 k Ω hingga 600 Ω), sehingga kompatibel dengan mikrokontroler atau IC pengolah sinyal.

Integrasi sensor suara dengan mikrokontroler memungkinkan pengembangan aplikasi cerdas seperti asisten suara virtual atau sistem deteksi suara otomatis, sebagaimana dijelaskan oleh Miller & Lee (2021).





Gambar 3 Rangkaian Sensor Suara

Sumber: (Zhengyihong, 2025)

Pada rangkaian diatas merupakan sistem deteksi berbasis sensor kapasitif (Q1) yang diproses oleh dua channel komparator LM393, di mana channel pertama (U1.1) menghasilkan output analog (AO) melalui perbandingan antara tegangan input sensor dan referensi variabel yang dikendalikan potensiometer VR1. Tegangan referensi pada pin inverting U1.1 dihitung dengan rumus

$$V_{ref,U1,1} = V_{cc} \times \frac{VR1}{R6 + VR1} \quad (1.2)$$

memungkinkan kalibrasi sensitivitas secara manual dari 0 hingga 5V (dengan asumsi $V_{CC}=5V$). Sementara itu, tegangan input sensor (V_+) diasumsikan sebanding dengan sinyal DC yang dihasilkan oleh sensor kapasitif — meskipun dalam praktiknya, sensor kapasitif memerlukan modulasi AC untuk menghasilkan sinyal terukur. Output AO, meskipun dinamis, tidak bersifat linier murni karena kurangnya feedback negatif penuh, sehingga lebih tepat diinterpretasikan sebagai representasi analog dari selisih antara input dan referensi, bukan amplifier analog konvensional.

$$V_{AO} \propto V_{in} - V_{ref} \quad (1.3)$$

Channel kedua (U1.2) berfungsi sebagai komparator digital yang membandingkan output U1.1 dengan ground (0V), sehingga menghasilkan sinyal digital DO yang HIGH ketika output U1.1 positif — mencerminkan keadaan “deteksi aktif”. Output DO kemudian digunakan untuk mengendalikan LED indikator L1 (melalui R5=1 kΩ) dan L2 (melalui R4=10 kΩ), dengan arus LED dihitung menggunakan rumus

$$I_{-(LED)} = \frac{V_{cc} - V_{-(LED)}}{R_{limit}} \quad (1.4)$$

menunjukkan bahwa L1 menyala lebih terang (≈ 3 mA) dibanding L2 (≈ 0.3 mA), sesuai fungsinya sebagai indikator utama dan sekunder. Secara akademik, rangkaian ini menunjukkan integrasi antara deteksi non-kontak, pengolahan sinyal analog-digital, dan antarmuka visual — namun memiliki keterbatasan stabilitas karena tidak adanya hysteresis dan ketergantungan pada karakteristik sensor kapasitif yang non-linear. Untuk aplikasi real-world, diperlukan penambahan rangkaian kondisioning sinyal (C-to-V converter) serta filter atau hysteresis untuk meningkatkan akurasi dan reliabilitas sistem.

2.2.5 Sensor Getar

Sensor getaran merupakan komponen penting dalam sistem pemantauan kondisi mesin, deteksi keamanan, hingga sistem peringatan dini bencana seperti gempa bumi. Salah satu jenis sensor getaran yang umum digunakan dalam aplikasi berbasis mikrokontroler adalah modul berbasis SW-420, yang merupakan normally closed (NC) vibration switch dengan karakteristik mekanis sederhana namun efektif. Sensor SW-420 bekerja berdasarkan prinsip mechanical switch displacement. Di dalamnya terdapat sebuah roller-type conductive switch yang dalam keadaan stabil—tanpa getaran—berada dalam posisi tertutup (closed), sehingga menghasilkan impedansi rendah dan memungkinkan aliran arus listrik. Ketika terjadi gangguan vibrasi atau perubahan orientasi, elemen mekanis di dalam switch bergerak secara dinamis, menyebabkan pemutusan sementara aliran arus

Rangkaian ini merupakan sistem deteksi analog-digital berbasis komparator LM393, yang membandingkan tegangan dari sensor resistif (terhubung ke pin INA+) dengan tegangan referensi tetap sebesar

$$V_{ref} = \frac{V_{cc}}{2} \quad (1.5)$$

dihasilkan oleh pembagi tegangan dua resistor 10 k Ω (R2 dan ground).

$$V_+ = V_{cc} = \frac{10k\Omega}{R_{sensor} + 10k\Omega} \quad (1.6)$$

Tegangan input sensor bersifat dinamis dan berubah sesuai perubahan resistansi sensor. Titik pemicu (threshold) terjadi ketika $V_+ = V_-$, yang secara matematis menghasilkan nilai resistansi sensor kritis sebesar $R_{sensor} = 10 k\Omega$. Artinya, output komparator akan berubah state (HIGH/LOW) saat resistansi sensor melewati nilai tersebut. Dengan demikian, rangkaian ini berfungsi sebagai *analog-to-digital converter* sederhana, mengonversi perubahan fisik menjadi sinyal logika digital.

Output digital (OUTA) diimplementasikan melalui LED indikator (D2) dan resistor pembatas arus (R4), yang juga berfungsi sebagai pull-up untuk output open-collector LM393. Arus LED dihitung dengan rumus

$$I_{LED} \approx \frac{V_{cc} - V_{D2}}{R4} \quad (1.7)$$

memberikan intensitas cahaya yang aman dan cukup terlihat. Sementara itu, output analog (AO) diambil dari pembagi tegangan antara R5 dan R2, menghasilkan

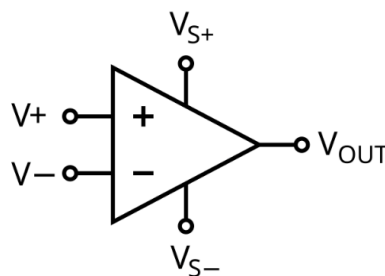
$$V_{AO} = \frac{V_{out}}{2} \quad (1.8)$$

sehingga merepresentasikan level logika dalam bentuk voltase analog (misalnya 2.5V untuk HIGH, 0V untuk LOW). Kapasitor decoupling (C1, C2 = 0.1 μ F) menstabilkan suplai daya dan meredam noise, namun karena tidak ada hysteresis, rangkaian rentan osilasi di sekitar titik trigger. Dalam konteks akademik, rangkaian ini menjadi studi kasus ideal untuk memahami prinsip komparator, desain antarmuka sensor, dan konversi sinyal analog-ke-digital — meskipun untuk aplikasi nyata, penambahan hysteresis sangat direkomendasikan demi stabilitas sistem.

2.2.6 Operational Amplifier Komparator

Operational Amplifier (Op-Amp) merupakan komponen fundamental dalam elektronika analog yang berfungsi sebagai penguat tegangan diferensial dengan penguatan sangat tinggi, impedansi masukan yang besar, dan impedansi keluaran yang rendah. Secara ideal, op-amp memiliki penguatan tegangan tak hingga, impedansi masukan tak hingga, impedansi keluaran nol, serta offset tegangan nol saat kedua masukannya berada pada potensial yang sama (Sedra & Smith, 2021).

Dalam konfigurasi komparator, Op-Amp bekerja tanpa umpan balik (*open-loop configuration*), sehingga *gain* sangat besar, hampir tak terbatas. Hal ini menyebabkan *output* langsung jenuh ($+V_{sat}$ atau $-V_{sat}$) ketika ada perbedaan kecil antara kedua *input*



Gambar 5 Op-Amp

Sumber: M. Rahman et al. (2021)

Komparator memiliki dua *input*:

- *Input Non-Inverting(+)*: Digunakan untuk menerima tegangan referensi (V_{ref}).
- *Input Inverting(-)*: Digunakan untuk menerima sinyal input (V_{in}).

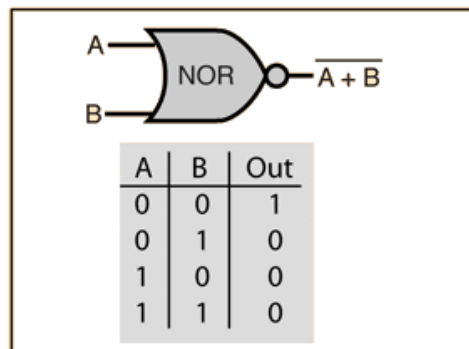
Prinsip kerja komparator sangat sederhana:

- Jika $V_{in} > V_{ref}$, *output* akan menjadi *HIGH* (mendekati $+V_{cc}$).
- Jika $V_{in} < V_{ref}$, *output* akan menjadi *LOW* (mendekati $-V_{ee}$).

2.2.7 Gerbang Logika NOR

Menurut Floyd (2019), gerbang NOR adalah salah satu dari lima jenis dasar gerbang logika digital yang menghasilkan *output HIGH* hanya ketika semua input bernilai *LOW* (Floyd, T. L., 2019). Gerbang NOR merupakan kependekan dari *NOT-OR*, yaitu inversi dari gerbang OR. Gerbang ini sangat penting dalam desain

rangkaian logika karena kemampuan kombinasinya untuk membentuk semua fungsi logika lainnya jika digunakan secara tepat. Prinsip Kerja dan Fungsi Logika gerbang NOR bekerja berdasarkan tabel kebenaran yang ditunjukkan pada Gambar 7



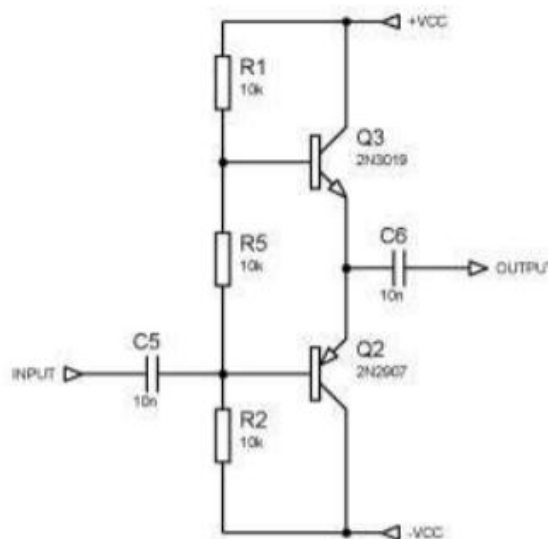
Gambar 6 Logika NOR

Sumber: A. Kumar et al. (2019)

IC NOR seperti M74HC4078 atau 74LS02 merupakan contoh penerapan NOR dalam bentuk sirkuit terpadu (IC) TTL atau CMOS yang sering digunakan dalam sistem digital dan *embedded*. IC ini memiliki beberapa saluran *input* dan *output*, serta dirancang untuk beroperasi pada tegangan tertentu (misalnya 5V untuk TTL, 2–6 V untuk CMOS).

2.2.8 Rangkaian Push Pull

Rangkaian *push-pull* merupakan konfigurasi elektronika daya yang menggunakan dua transistor aktif (biasanya satu NPN dan satu PNP atau satu NMOS dan PMOS) untuk mengendalikan arus beban ke atas dan ke bawah secara bergantian. Konfigurasi ini memungkinkan sirkuit menghasilkan *output* dengan impedansi rendah, kemampuan sinkronisasi tinggi, serta respon frekuensi yang baik. Rangkaian *push-pull* banyak digunakan dalam penguat daya (*power amplifier*), penggerak motor, dan aplikasi digital seperti driver MOSFET atau buffer logika. Berikut merupakan gambar Rangkaian Dasar Push-Pull dapat dilihat pada Gambar 7.



Gambar 7 Rangkaian Dasar Push-Pull
Sumber: (Azarov et al., 2025)

Dalam konteks sistem simulasi tembakan berbasis sensor, rangkaian *push-pull* dapat diterapkan sebagai unit penguat (*amplifier*) untuk pulsa *infrared* atau sebagai driver untuk aktuator seperti buzzer, LED, atau motor miniatur senjata pada alat pelatihan tempur.

- Prinsip Kerja Rangkaian *Push-Pull*

Rangkaian *push-pull* bekerja dengan prinsip bahwa salah satu transistor "menarik" arus (*pull*) sedangkan transistor lainnya "mendorong" arus (*push*) tergantung polaritas sinyal *input*. Dengan demikian, rangkaian mampu menghasilkan gelombang keluaran yang simetris dan hampir tidak terdistorsi.

- Konfigurasi Dasar:

- Transistor atas (*push*): Biasanya PNP atau PMOS.
- Transistor bawah (*pull*): NPN atau NMOS.
- Beban (*load*): Terhubung di tengah kedua transistor.

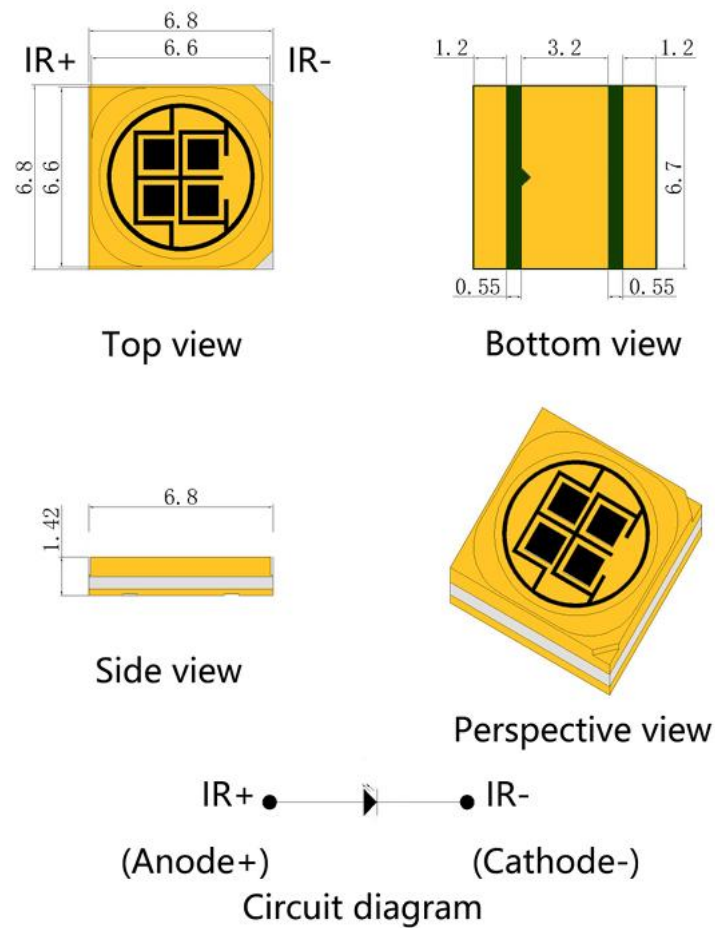
Ketika sinyal positif masuk, transistor atas (*pull*) *cut-off* dan transistor bawah (*push*) aktif, sehingga menyediakan jalur arus ke tanah. Sebaliknya, ketika sinyal negatif masuk, transistor atas aktif dan transistor bawah *cut-off*, sehingga menyediakan arus dari tegangan catu daya ke beban

2.2.9 Transmitter Inframerah

Transmitter *infrared* adalah perangkat elektronik yang menggunakan sinyal *infrared* untuk mengirimkan data atau perintah kepada perangkat penerima. Sinyal *infrared* dihasilkan oleh *Light Emitting Diode* (LED) *infrared* yang mengubah data digital menjadi pulsa cahaya. Pulsa ini diterima oleh sensor penerima seperti *Phototransistor*, yang kemudian mendekodekannya menjadi data digital kembali. Panjang gelombang *infrared* yang umum digunakan berkisar antara 700 nm hingga 1 mm, tergantung pada aplikasinya.

Transmitter *infrared* menawarkan kecepatan transfer data yang tinggi, keamanan yang baik karena sinyalnya tidak menembus dinding, serta biaya produksi yang relatif rendah. Namun, teknologinya memiliki keterbatasan, seperti kebutuhan akan jalur pandang langsung antara transmitter dan receiver serta kerentanan terhadap interferensi dari cahaya matahari (Yunardi, 2017).

Inframerah LED (IR LED) adalah jenis dioda emisi cahaya (LED) yang memancarkan radiasi elektromagnetik pada panjang gelombang *infrared* (~700 nm hingga 1 mm), yang tidak terlihat oleh mata manusia namun dapat dideteksi oleh *photodiode* atau fototransistor IR. Cahaya *infrared* yang dipancarkan oleh LED ini sering digunakan dalam sistem *remote control*, *proximity sensing*, pendeteksi gerakan, dan komunikasi nirkabel jarak dekat. Berikut merupakan gambar *Infrared* LED dapat dilihat pada Gambar 9.



Gambar 8 *Infrared LED*
 Sumber: M. Ali et al. (2021)

Prinsip kerja IR LED pada gambar 2.15 mirip dengan LED biasa, yaitu mengubah arus listrik menjadi foton cahaya. Namun, material semikonduktor yang digunakan dirancang agar memancarkan cahaya di rentang *infrared*. Pada sistem permainan laser tag, IR LED digunakan sebagai bagian dari *Small Arms Transmitter* (SAT) untuk memancarkan pulsa *infrared* setiap kali pelatuk senjata ditarik. Pulsa tersebut membawa informasi identifikasi senjata dan waktu penembakan yang akan diterima oleh sensor IR pada target.

IR LED dalam SAT harus dirancang untuk bekerja dalam kondisi lapangan yang keras, termasuk paparan debu, guncangan fisik, dan variasi suhu. Oleh karena itu, banyak modul IR LED dilengkapi dengan lensa Fresnel untuk meningkatkan jarak pancar dan akurasi deteksi. Selain itu, pulsa yang dikirimkan bersifat *encoded*

agar dapat dibedakan antara tembakan dari satu senjata dengan senjata lain, memastikan evaluasi hasil tembakan akurat dan *real-time*.

2.2.10 Modulasi dan Demodulasi Inframerah

Modulasi dan demodulasi *infrared* merupakan dua proses yang saling terkait dalam komunikasi berbasis cahaya *infrared*. Modulasi adalah proses menumpangkan sinyal informasi (misalnya data digital) pada gelombang pembawa *infrared*, sedangkan demodulasi adalah proses memisahkan kembali sinyal informasi dari gelombang pembawa yang telah dimodulasi.

1. Modulasi *Infrared*

Modulasi *infrared* adalah proses memodifikasi sinyal *infrared* untuk membawa informasi spesifik seperti perintah kontrol maupun data digital. Teknologi ini sangat penting pada aplikasi seperti sistem komunikasi nirkabel yang menggunakan diode pemancar *infrared (IR)* dan penerima *IR*, misalnya pada perangkat kontrol jarak jauh, sensor kedekatan, serta sistem pengenalan dan keamanan serupa lainnya.

Dalam modulasi *infrared*, impuls cahaya yang dipancarkan oleh *IR-LED* dimodulasi sesuai dengan pola biner yang mewakili data. Proses ini dilakukan dengan mengatur arus penggerak pada *IR-LED*, sehingga intensitas cahaya *infrared* berubah sesuai dengan data yang ingin ditransmisikan.

Metode modulasi yang paling umum digunakan adalah *Carrier Frequency Modulation (CFM)*, yaitu modulasi pada frekuensi pembawa. Pada sistem ini, sinyal data digunakan untuk mendukung atau mengubah fase dan amplitudo gelombang pembawa *infrared* pada frekuensi tertentu (biasanya 36-38kHz). Teknik ini memungkinkan penerima *infrared* dapat membedakan sinyal yang sengaja dikirim dan noise dari cahaya lingkungan sekitar.

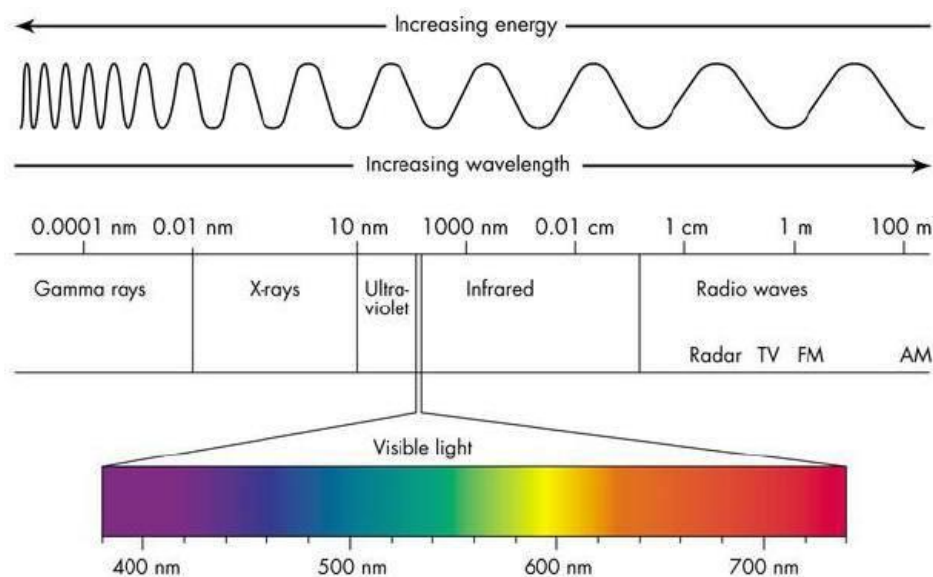
2. Demodulasi *Infrared*

Demodulasi *infrared* adalah proses pemulihan sinyal informasi dari gelombang *infrared* yang telah dimodulasi di sisi pemancar. Proses ini dilakukan oleh penerima *infrared (IR receiver)* untuk mengubah impuls optik menjadi data biner atau perintah digital, yang selanjutnya dapat diproses oleh sistem kontrol atau mikrokontroler.

Pada sistem komunikasi *infrared* seperti kontrol jarak jauh dan sensor nirkabel, sinyal data umumnya tidak ditransmisikan sebagai cahaya kontinu, melainkan dimodulasi pada frekuensi pembawa (36-38kHz). Proses demodulasi di penerima digunakan untuk memisahkan frekuensi pembawa dan mengekstrak data asli yang dibawa oleh sinyal *infrared*.

2.2.11 Gelombang Inframerah

Gelombang *infrared* merupakan bagian dari spektrum elektromagnetik yang terletak di antara cahaya tampak dan gelombang mikro, dengan panjang gelombang berkisar antara 700nm hingga 1mm. Meskipun tidak dapat dilihat oleh mata manusia, radiasi *infrared* dapat dirasakan sebagai panas dan telah dimanfaatkan dalam berbagai aplikasi teknologi, seperti pemantauan suhu, deteksi gerakan, serta sistem komunikasi nirkabel. Berikut merupakan gambar dari Panjang Gelombang dapat dilihat pada Gambar 5.



Gambar 9 Panjang Gelombang

Sumber: National Radio Astronomy Observatory (2012)

Dalam system permainan laser tag, gelombang *infrared* digunakan untuk mentransmisikan data tembakan dari senjata ke sensor penerima pada target. Pulsa *infrared* ini membawa informasi penting seperti ID senjata, waktu penembakan, dan lokasi tembakan. Sensor penerima kemudian menganalisis data tersebut untuk menentukan apakah tembakan tersebut mengenai area vital atau tidak. Proses ini

memastikan bahwa sistem dapat memberikan umpan balik instan kepada para pemain terkait status “terkena tembakan” secara akurat.

Pulsa *infrared* yang digunakan pada senjata umumnya berada pada rentang *Near Infrared (NIR)*, yaitu antara 700–1.400nm. Rentang ini dipilih karena banyak material optik memiliki transmisi tinggi di wilayah tersebut, sehingga pulsa dapat diterima dengan baik oleh sensor *infrared*. Selain itu, sinar *infrared* dalam rentang ini relatif aman untuk penggunaan luar ruang (*outdoor*) dan tidak terganggu oleh cahaya tampak, menjadikannya pilihan ideal dalam sistem pelatihan tempur modern.

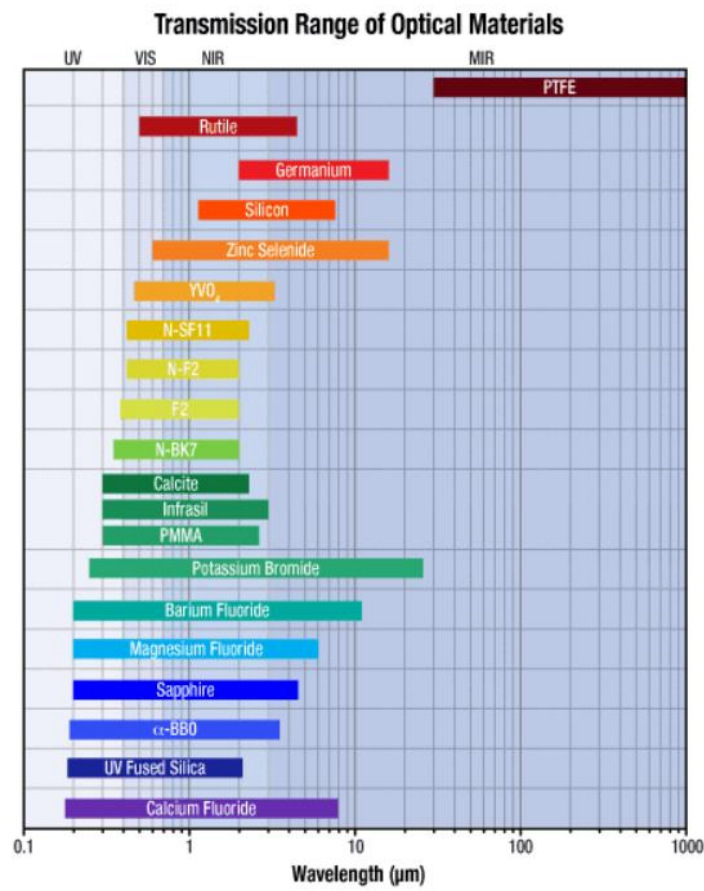
2.2.12 Nippon Electric Company Protocol

Nippon Electric Company (NEC) adalah salah satu standar komunikasi *infrared* yang sering digunakan pada perangkat kontrol jarak jauh, televisi, pendingin ruangan, pemutar DVD, serta berbagai alat elektronik lainnya, terutama yang dibuat di Jepang. Protokol ini dikembangkan oleh perusahaan Jepang bernama NEC dan telah menjadi salah satu protokol komunikasi *infrared* yang paling banyak digunakan karena sifatnya yang mudah dipahami, efisien, serta mudah diimplementasikan dalam berbagai aplikasi teknologi terhubung (IoT).

Pada protokol NEC, komunikasi terjadi melalui pengiriman sinyal cahaya *infrared* dari remote control ke penerima di perangkat yang dituju. Sinyal tersebut berupa pola waktu yang mewakili data berbentuk biner. Protokol NEC memiliki beberapa ciri khas seperti frekuensi 38 kHz, panjang data sebanyak 32 bit atau 4 byte, alamat perangkat sepanjang 8 bit, serta mampu mengulang sinyal setiap 110 ms.

2.2.13 Jarak Transmisi Bahan Optik

Jarak transmisi bahan optik merujuk pada kemampuan suatu material untuk meneruskan gelombang elektromagnetik—seperti cahaya tampak atau inframerah—tanpa banyak menyerap atau memantulkannya (Smith, W. J., 2020). Setiap bahan memiliki karakteristik transmisi berbeda tergantung pada panjang gelombang cahaya yang digunakan. Misalnya, kaca borosilikat memiliki transmisi tinggi pada cahaya tampak, sementara polikarbonat atau akrilik lebih cocok untuk transmisi inframerah dekat (NIR).



Gambar 10 Transmisi Bahan Optik

Sumber: Applied Optics (2008)

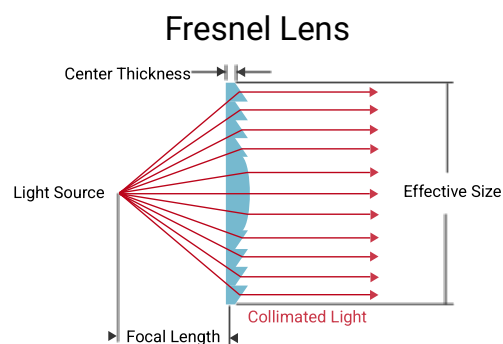
Panjang gelombang cahaya sangat mempengaruhi seberapa besar cahaya dapat ditransmisikan melalui bahan. Beberapa bahan seperti silikon (Si) dan selenium seng (ZnSe) memiliki transmisi tinggi di rentang inframerah dekat (NIR), sehingga sangat cocok untuk aplikasi seperti LED IR dan sensor IR. Di sisi lain, bahan seperti fluorit kalsium (CaF₂) dan fluorit barium (BaF₂) memiliki transmisi tinggi di rentang UV hingga NIR, menjadikannya pilihan populer dalam sistem optik tingkat tinggi.

Ketebalan dan kejernihan bahan juga memainkan peran penting dalam transmisi cahaya. Bahan yang lebih tipis dan bebas cacat atau gelembung udara akan memiliki transmisi yang lebih baik. Sudut datang cahaya juga memengaruhi efisiensi transmisi, di mana cahaya yang datang tegak lurus memberikan hasil transmisi maksimum. Oleh karena itu, dalam sistem simulasi tembakan berbasis

laser, pemilihan bahan optik harus mempertimbangkan faktor-faktor ini agar sistem tetap akurat dan efisien.

2.2.14 Lensa Freshnel

Lensa Fresnel adalah jenis lensa optik yang dirancang untuk memiliki sifat konvergen seperti lensa cembung biasa, tetapi dengan struktur bertingkat yang memungkinkan pengurangan ketebalan dan berat secara signifikan (Hecht, E., 2017). Lensa ini ditemukan oleh Augustin-Jean Fresnel dan awalnya digunakan dalam mercusuar untuk memfokuskan cahaya sejauh mungkin. Struktur uniknya terdiri dari lingkaran konsentris yang berfungsi sebagai permukaan refraksi individual, sehingga cahaya dapat difokuskan tanpa memerlukan bahan yang tebal.



Gambar 11 Lensa Freshnel
Sumber: Meetoptics (2002)

Secara prinsip, lensa Fresnel bekerja dengan cara membelokkan (refraksi) sinar cahaya yang melewatinya menuju satu titik fokus tertentu. Dibandingkan dengan lensa cembung konvensional, lensa ini lebih efisien dalam hal transmisi cahaya karena mengurangi pembelokan internal dan distorsi. Meskipun demikian, lensa Fresnel cenderung memiliki sedikit distorsi optik di tepi, namun tetap sangat berguna dalam aplikasi yang tidak memerlukan resolusi tinggi seperti proyektor, lampu sorot, atau sensor jarak jauh.

Lensa Fresnel banyak digunakan dalam aplikasi optik yang membutuhkan fokus cahaya kuat namun dengan ukuran yang ringkas. Contoh penggunaannya meliputi lampu mobil, proyektor LCD, dan sistem pendeteksi inframerah. Dalam sistem MILES dan permainan laser tag, lensa Fresnel digunakan dalam SAT untuk

memfokuskan pulsa inframerah agar mencapai jarak yang lebih jauh dengan intensitas tetap stabil, meningkatkan akurasi deteksi saat target terkena tembakan.

Untuk menentukan *focal length* (jarak fokus) lensa Fresnel, kita menggunakan prinsip dasar optik geometri. Lensa Fresnel dirancang untuk memfokuskan atau menyebarkan cahaya dengan cara yang mirip dengan lensa konvensional, tetapi dengan profil yang lebih tipis dan ringan. Berikut adalah rumus dasar yang digunakan untuk menghitung *focal length* lensa Fresnel:

$$f = \frac{R}{n - 1} \quad (1.9)$$

Dimana:

f = Focal Length

R = Radius Kelengkungan Permukaan Lensa

n = Indeks Bias Material Lensa

2.2.15 Push Button

Push button switch, atau saklar tombol tekan, merupakan perangkat dasar namun penting dalam sistem kontrol elektronik. Push button berfungsi sebagai penghubung atau pemutus aliran arus listrik, dengan sistem kerja "unlock", yang berarti saklar hanya akan mengalirkan arus saat tombol ditekan, dan kembali ke posisi semula (off) saat tombol dilepas. Sistem operasi "on-off" ini mengatur kondisi 1 (on) dan 0 (off), yang menjadi dasar pengoperasian banyak perangkat listrik yang memerlukan pengendalian aliran listrik dengan dua kondisi utama. Hal ini sangat relevan dalam dunia industri dan teknologi, di mana kehadiran push button switch sangat krusial untuk mengatur pengoperasian mesin dan peralatan lainnya.

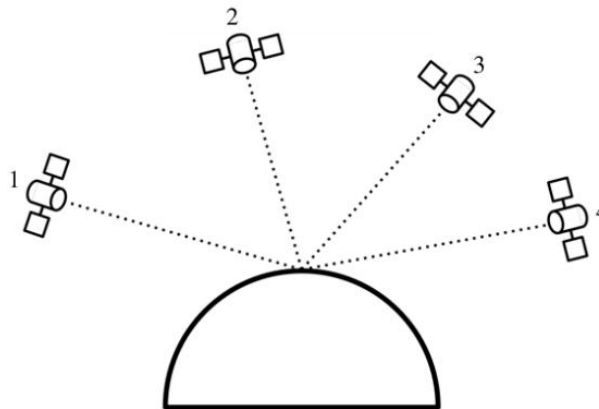
Dalam aplikasi industri dan otomasi, push button switch sering digunakan sebagai alat untuk memulai dan menghentikan proses kerja mesin. Meskipun mesin atau sistem kontrol yang lebih canggih mungkin digunakan, sistem push button tetap menjadi elemen kunci dalam kontrol manual dan otomatis. Keandalan, kesederhanaan, dan daya tahannya menjadikan push button sebagai komponen yang sangat digunakan dalam berbagai aplikasi industri, dari pengoperasian mesin hingga perangkat rumah tangga otomatis. (Munandar et al., 2023)

Dengan peranannya yang begitu fundamental dalam pengoperasian sistem-sistem elektronik dan otomatisasi, push button switch menjadi komponen yang tak terhindarkan dalam dunia teknik dan rekayasa sistem kontrol. Sifatnya yang sederhana namun efektif membuatnya sangat sesuai untuk aplikasi yang membutuhkan interaksi langsung antara operator dan perangkat.

2.2.16 Global Positioning System (GPS)

Global Positioning System (GPS) adalah teknologi navigasi berbasis satelit yang memungkinkan penentuan lokasi dan waktu secara akurat di seluruh permukaan bumi. GPS bekerja dengan menerima sinyal dari minimal empat satelit dan menghitung posisi melalui proses trilaterasi berdasarkan perbedaan waktu sinyal diterima. Teknologi ini awalnya dikembangkan untuk keperluan militer, namun kini telah digunakan secara luas di berbagai sektor seperti transportasi, pertanian, navigasi darat dan laut, hingga sistem otonom dan *Internet of Things* (IoT). Perangkat GPS modern menggunakan teknik pemrosesan sinyal canggih seperti RTK (*Real-Time Kinematic*) dan DGPS (*Differential GPS*) untuk meningkatkan akurasi hingga tingkat sentimeter. Meskipun menghadapi tantangan seperti gangguan sinyal dan kondisi atmosfer, integrasi GPS dengan sistem GNSS lain seperti Galileo, GLONASS, dan BeiDou, serta dukungan teknologi seperti 5G, semakin meningkatkan keandalan dan presisi GPS untuk berbagai aplikasi masa kini dan masa depan (Sunil Fulzele & Athawale, 2024).

Prinsip kerja GPS didasarkan pada metode trilaterasi, yaitu teknik penentuan posisi dengan menghitung jarak antara penerima GPS dan setidaknya tiga satelit GPS. Setiap satelit mengorbit bumi dan secara periodik memancarkan sinyal yang berisi informasi mengenai waktu dan posisi satelit tersebut. Perangkat penerima GPS mengukur waktu tempuh sinyal dari masing-masing satelit untuk menghitung jarak secara presisi. Dengan memanfaatkan data dari minimal empat satelit, sistem GPS mampu menentukan posisi tiga dimensi suatu objek, yakni lintang, bujur, dan ketinggian, dengan tingkat akurasi yang tinggi (Pretz et al., 2017). Berikut merupakan gambar dari GPS dapat dilihat pada Gambar 12.



Gambar 12 Global Positioning System (GPS)

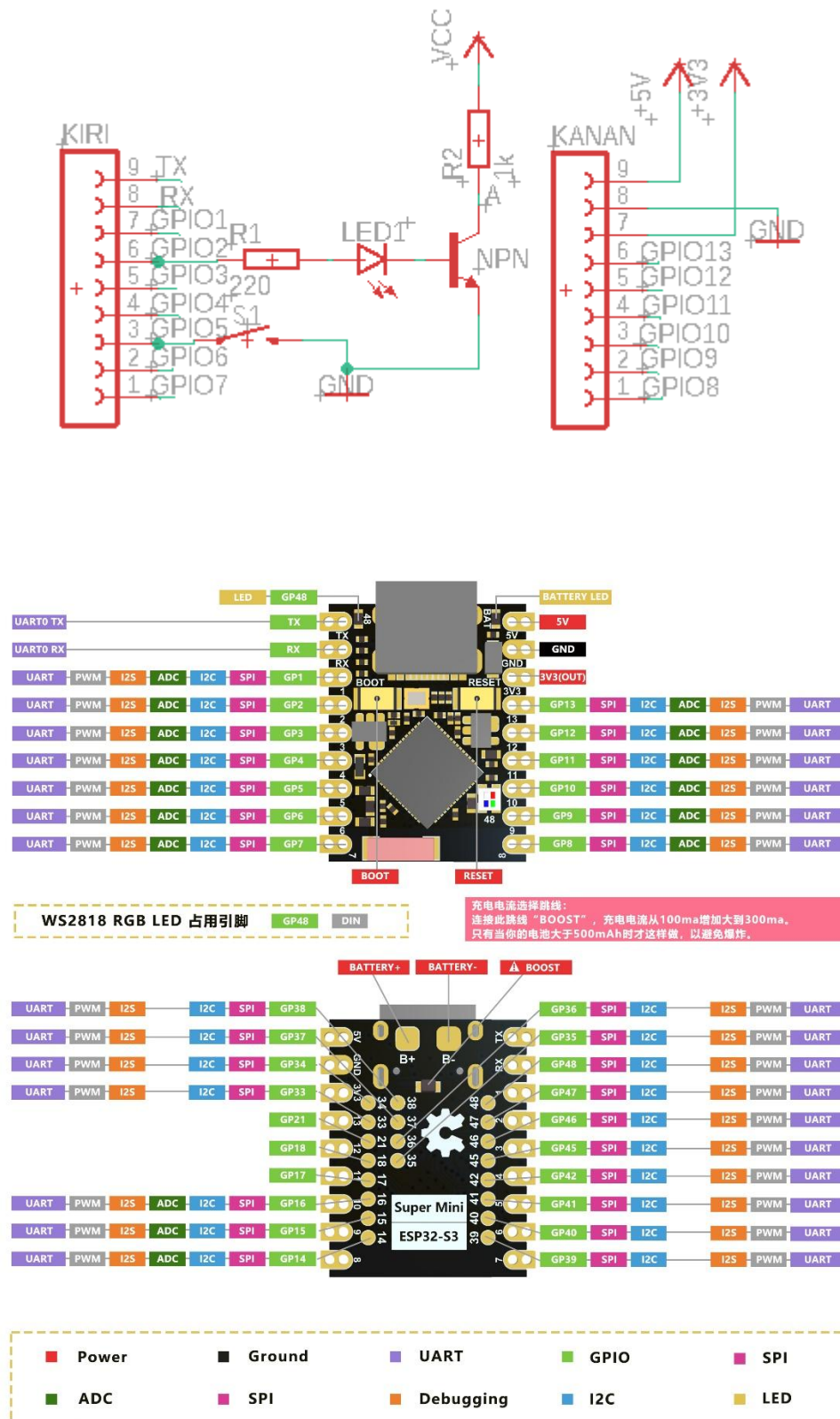
Sumber : (Prettz et al., 2017)

2.2.17 Mikrokontroler

Mikrokontroler adalah *Integrated Circuit (IC)* yang dirancang untuk mengendalikan fungsi elektronik tertentu dalam suatu sistem. Mikrokontroler memiliki elemen-elemen utama seperti prosesor (*CPU*), memori (*RAM* dan *ROM/Flash*), serta modul *Input/Output (I/O)* yang terintegrasi dalam satu chip.

Salah satu fungsi utama mikrokontroler adalah mengelola komunikasi data antar perangkat yang berbeda. Mikrokontroler umumnya dilengkapi dengan modul komunikasi seperti *UART*, *SPI*, *I2C*, atau *CAN*, yang memungkinkan transfer data dengan perangkat eksternal seperti sensor, aktuator, atau mikrokontroler lain. Dalam jaringan sensor nirkabel, mikrokontroler bertugas mengolah data dari *node sensor*, mengonversinya menjadi format yang dapat diterima, dan mengirimkan data ke node berikutnya atau *gateway*. Mikrokontroler menawarkan efisiensi biaya, keandalan tinggi, dan fleksibilitas untuk aplikasi-aplikasi khusus. Namun, keterbatasan mikrokontroler meliputi kemampuan komputasi yang relatif rendah dan kapasitas memori yang terbatas dibandingkan mikroprosesor.

ESP32 adalah salah satu mikrokontroler yang paling populer dan menjadi varian board pengembangan keluaran DOIT. Board ini menyediakan akses lengkap ke semua fitur *ESP32* dengan layout pin yang *user-friendly*. Berikut merupakan gambar dari *ESP32 DevKit V1* dapat dilihat pada Gambar 13.



Gambar 13 ESP32
(Sumber: (Pratama & Kiswantono, 2023))

Berikut merupakan tabel spesifikasi dari ESP32 dapat dilihat pada Tabel 1.

Tabel 1 Spesifikasi ESP32

Fitur	Spesifikasi
Chip	ESP32 S3 Supermini
Core	Dual-core Xtensa LX7 CPU
Clock Speed	hingga 240 MHz (600 DMIPS)
Wi-Fi	802.11 b/g/n (2.4 GHz)
Bluetooth	Bluetooth 5 (BLE)
Flash	4 MB
SRAM	512 KB internal SRAM + 8 MB PSRAM eksternal (umumnya)
GPIO	~20+ pin GPIO
ADC	2x 12-bit SAR ADC (8 channel)
Touch Sensor	hingga 14 channel capacitive touch sensing
PWM	11 channel
Interface	UART, I ² C, SPI, I ² S, RMT, TWAI, USB OTG
Power Supply	5V via USB, regulated to 3.3V

2.2.18 Protokol ESP-Now

ESP-NOW merupakan protokol komunikasi nirkabel berbasis *link layer* yang dikembangkan oleh *Espressif Systems* untuk memungkinkan pertukaran data secara langsung antar perangkat yang menggunakan chip ESP seperti ESP32 dan ESP8266. Protokol ini dirancang untuk mendukung komunikasi cepat dengan latensi rendah serta konsumsi daya yang efisien, sehingga sangat cocok digunakan dalam aplikasi *Internet of Things* (IoT) dan sistem embedded. *ESP-NOW* tidak memerlukan koneksi infrastruktur Wi-Fi seperti router atau jaringan internet, melainkan bekerja secara *peer-to-peer* (P2P) menggunakan frekuensi 2.4 GHz. Dalam pengoperasiannya, *ESP-NOW* memanfaatkan teknologi *Wi-Fi Direct* melalui *action frames* untuk mentransmisikan data tanpa perlu koneksi TCP/IP, sehingga proses pengiriman data dapat dilakukan secara cepat dan sederhana. Komunikasi dapat dilakukan dalam mode unicast maupun broadcast dengan batasan payload maksimal sebesar 250 byte per paket. Selain itu, *ESP-NOW* juga mendukung enkripsi AES 128-bit untuk menjaga keamanan data selama proses pengiriman. Kelebihan utamanya termasuk kemampuan komunikasi langsung tanpa infrastruktur tambahan, hemat daya, latensi rendah, serta mudah diimplementasikan dalam berbagai proyek IoT seperti smart home automation,

sensor jaringan nirkabel, kontrol robot, dan sistem alarm. Meskipun memiliki keterbatasan dalam jumlah node maksimum dan ukuran payload, ESP-NOW tetap menjadi solusi komunikasi nirkabel yang efektif dan efisien bagi pengembangan sistem berbasis mikrokontroler ESP.(Adolfo & Setia Budi, 2023)

ESP-NOW menggunakan *frame* data Wi-Fi jenis *Action Frames*, yang memungkinkan pengiriman data dalam bentuk payload berukuran maksimal 250 byte. Komunikasi ini dapat berlangsung antara :

- Perangkat ESP sebagai satu lawan satu (*Peer-To-Peer*)
- Satu pengirim ke banyak penerima (*One-To-Many*)
- Beberapa pengirim ke satu penerima (*Many-To-One*)

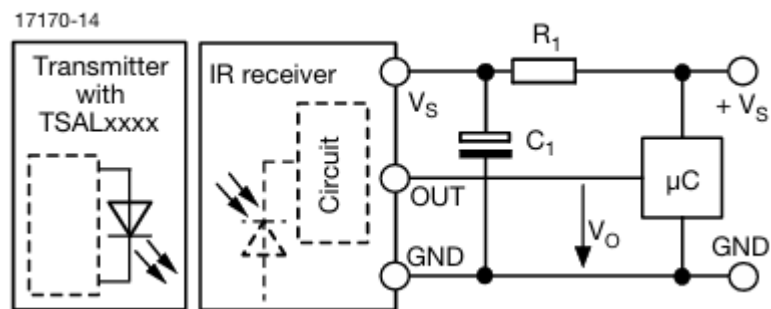
2.2.19 Receiver Infrared Anti Cheating

Penelitian ini bertujuan untuk mendokumentasikan eksperimen implementasi fitur anti-cheating pada sistem MILES (Multiple Integrated Laser Engagement System), yang dirancang guna mencegah kecurangan berupa penutupan sengaja terhadap sensor inframerah pada rompi peserta latihan. Dalam konfigurasi standar, rompi dilengkapi penerima TSOP4838 untuk mendeteksi sinyal inframerah termodulasi 38 kHz yang dipancarkan oleh SAT (Simulated Ammunition Transmitter) lawan. Namun, praktik umum menunjukkan bahwa peserta sering menutupi sensor tersebut dengan bahan buram (misalnya lakban), sehingga membuat mereka kebal terhadap deteksi tembakan.

Sebagai solusi, dirancang unit sensor tambahan yang terintegrasi dalam satu dome bersama TSOP4838, terdiri atas LED inframerah sebagai sumber cahaya aktif dan photodiode sebagai detektor refleksi lokal. Prinsip kerjanya didasarkan pada hilangnya sinyal pantulan inframerah ketika dome ditutupi kondisi ini menyebabkan output photodiode turun ke logika rendah. Delapan unit sensor (empat di bagian depan dan empat di belakang rompi) dihubungkan ke gerbang logika NOR 8-input IC 74HC4078, sehingga keberadaan setidaknya satu dome yang ditutupi menghasilkan logika tinggi pada keluaran gerbang tersebut. Sinyal ini kemudian dibaca oleh mikrokontroler ESP32 melalui salah satu pin GPIO-nya sebagai indikator kecurangan.

Eksperimen utama difokuskan pada verifikasi bahwa pancaran LED inframerah yang bersifat kontinu dan tidak termodulasi tidak mengganggu kemampuan

TSOP4838 dalam mendeteksi dan mendekode sinyal modulasi 38 kHz dari SAT lawan. Desain rangkaian memastikan isolasi spektral dan temporal antara kedua fungsi optoelektronik tersebut, sehingga integritas sistem MILES tetap terjaga sambil menambahkan lapisan keamanan tambahan terhadap manipulasi fisik sensor. Dokumentasi ini mencakup prinsip kerja, arsitektur rangkaian, prosedur pengujian, serta analisis hasil eksperimen secara sistematis untuk keperluan akademik dan implementasi praktis dalam sistem pelatihan militer berbasis MILES.



Gambar 14 TSOP43..

Sumber: (Vishay Semiconductors, 2025)

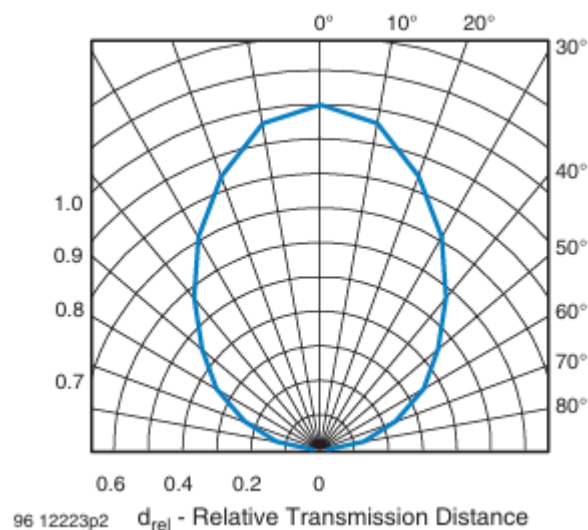
Berikut merupakan tabel spesifikasi TSOP43.. dapat dilihat pada Tabel 2.

Tabel 2 Spesifikasi TSOP43..

Product Attribute	Attribute Value
Manufacturer	Vishay
Product Category	<i>Infrared</i> Receivers
RoHS	Details
Mounting Style	Through Hole
Carrier Frequency	38 kHz
Transmission Distance	45 m
Beam Angle	90 deg

Output Current	5 mA
Operating Supply Voltage	2.5 V to 5.5 V
Operating Supply Current	700 μ A
Minimum Operating Temperature	-25 $^{\circ}$ C
Maximum Operating Temperature	+85 $^{\circ}$ C
Package/Case	Side Looker
Packaging	Tube
Brand	Vishay Semiconductors
Product Type	IR Receivers
Factory Pack Quantity	2160
Subcategory	<i>Infrared Data Communications</i>
Unit Weight	2.338 g

Sensor TSOP4838 memiliki sudut penerimaan efektif $\pm 45^{\circ}$ dari sumbu utama (axis). Dengan kata lain, jika remote control diarahkan langsung ke sensor (0°), maka sinyal diterima dengan kuat dan optimal. Jika remote digerakkan hingga membentuk sudut 45° ke kiri atau ke kanan, sensor masih bisa menerima sinyal, meskipun kekuatannya mulai berkurang. Di luar sudut tersebut ($>45^{\circ}$), kemungkinan besar sensor tidak akan mendeteksi sinyal. (Toyib et al., 2019).

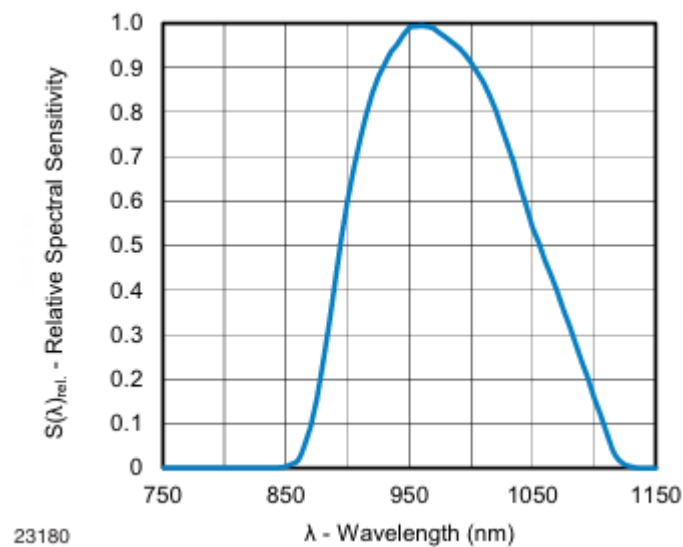


Gambar 15 Sudut penerimaan sensor TSOP4838

Gambar 34 menunjukkan grafik yang berkaitan dengan sudut penerimaan (*directivity*) sensor. Sumbu radial (sumbu horizontal) menyatakan *relative transmission distance* (d_{rel}), yaitu jarak transmisi relatif. Nilai $d_{rel} = 1.0$ pada sumbu ini menunjukkan jarak transmisi maksimum saat sinyal datang langsung ke

arah tengah sensor (0°). Semakin kecil nilai d_{rel} , semakin rendah sensitivitas sensor terhadap sinyal IR dari sudut tertentu. Sumbu sudut (sumbu lingkaran) menyatakan sudut datang sinyal IR terhadap sumbu utama sensor (arah 0°). Sudut dinyatakan dalam derajat (0° hingga $\pm 90^\circ$) di sekitar sumbu utama sensor. Garis kurva hitam pada polar plot menunjukkan hubungan antara sudut datang sinyal dan sensitivitas sensor. Pada 0° , garis mencapai nilai maksimum ($d_{rel} = 1.0$), yang menunjukkan bahwa sensitivitas sensor paling tinggi ketika sinyal datang langsung ke arah tengah. Semakin menjauh dari 0° , nilai d_{rel} menurun, menunjukkan bahwa sensitivitas sensor berkurang. Saat sinyal datang tepat di depan sensor (0°), sensitivitasnya mencapai nilai maksimum ($d_{rel} = 1.0$). Ini berarti sensor dapat menerima sinyal dengan jarak transmisi maksimal. Dari datasheet, diketahui bahwa sudut penerimaan efektif adalah $\pm 45^\circ$. Pada $\pm 45^\circ$, nilai d_{rel} turun menjadi sekitar 0.4–0.5, yang menunjukkan bahwa sensitivitas masih cukup baik tetapi sudah menurun dibandingkan dengan posisi 0° . Di luar sudut $\pm 45^\circ$, sensitivitas menurun drastis, sehingga sensor cenderung tidak dapat menerima sinyal dengan akurat, garis kurva menunjukkan bahwa d_{rel} mendekati nol. Ini berarti sensitivitas sensor sangat rendah atau hampir tidak ada.

TSOP4838 memiliki kemampuan untuk merespons panjang gelombang cahaya tertentu, yang disebut *Relative Spectral Sensitivity* (RSS). Dalam konteks TSOP4838, ini mengacu pada seberapa sensitif sensor terhadap berbagai panjang gelombang cahaya inframerah (IR), relatif terhadap panjang gelombang optimalnya LED inframerah memancarkan Cahaya dengan Panjang gelombang 930-950 nm. Agar komunikasi berjalan dengan efisien. LED pemancar harus bekerja pada panjang gelombang yang sesuai dengan puncak RSS dari TSOP4838.



Gambar 16 Grafik Panjang Gelombang

Grafik ini menunjukkan bagaimana respons sensitivitas sensor TSOP4838 berubah terhadap berbagai Panjang gelombang yang diterima. Jika Panjang gelombang mencapai 950nm maka sensitivitas akan mencapai nilai puncak yang berarti pada titik maksimal kemampuan sensitivitas sensor TSOP4838. Jika Panjang gelombang yang diterima tidak sesuai dengan puncak sensitivitas sensor, maka akan mengurangi sensitivitas sensor penerima bahkan dapat menyebabkan error atau sinyal tidak terbaca

2.2.20 LoraWan

LoRaWAN (*Long Range Wide Area Network*) merupakan protokol jaringan komunikasi nirkabel berdaya rendah yang dirancang untuk mendukung aplikasi *Internet of Things* (IoT) skala luas. Protokol ini dibangun di atas teknologi modulasi LoRa (*Long Range*), yang merupakan implementasi fisik dari *Chirp Spread Spectrum* (CSS) yang memungkinkan transmisi data jarak jauh dengan konsumsi daya yang sangat rendah (Adelantado et al., 2017).

Arsitektur LoRaWAN mengadopsi topologi *star-of-stars*, di mana *end-devices* (perangkat akhir) berkomunikasi secara langsung dengan satu atau lebih *gateways*. *Gateway* kemudian meneruskan data ke *network server* melalui jaringan backhaul (misalnya internet), yang selanjutnya mengarahkan data ke *application server* yang

sesuai. Komunikasi dalam LoRaWAN bersifat asinkron dan menggunakan mekanisme *ALOHA* untuk akses media, sehingga mengurangi kompleksitas sinkronisasi antar perangkat (Alliance, 2023)

LoRaWAN mendefinisikan tiga kelas perangkat berdasarkan pola komunikasi:

- Kelas A: Komunikasi dua arah dengan jendela penerimaan (*receive windows*) setelah transmisi dari perangkat. Paling hemat energi.
- Kelas B: Menambahkan *beacon*-based scheduling untuk penerimaan periodik, menyeimbangkan antara latensi dan efisiensi energi.
- Kelas C: Perangkat selalu dalam keadaan siap menerima, kecuali saat mengirim data—cocok untuk aplikasi berdaya tinggi.

Keamanan dalam LoRaWAN dijamin melalui dua lapis enkripsi: *Network Session Key* (NwkSKey) dan *Application Session Key* (AppSKey), serta mekanisme autentikasi berbasis *Join Accept* selama proses *over-the-air activation* (OTAA).

Di Indonesia, penggunaan spektrum frekuensi radio untuk perangkat telekomunikasi diatur oleh Kementerian Komunikasi dan Informatika (Kominfo) melalui Peraturan Menteri Kominfo Nomor 21 Tahun 2019 tentang *Penggunaan Spektrum Frekuensi Radio untuk Perangkat Telekomunikasi Short Range Device (SRD)*. Meskipun LoRaWAN secara global menggunakan beberapa pita frekuensi (misalnya EU868, US915, AS923), di Indonesia pita frekuensi yang umum digunakan dan diizinkan untuk SRD termasuk 920–923 MHz, yang secara teknis mengacu pada rencana regional AS923 (Asia 923 MHz).

Pita 920–923 MHz di Indonesia memiliki karakteristik sebagai berikut (Kominfo, 2019):

- Lebar pita: 3 MHz (920–923 MHz)
- Daya pancar maksimum: +14 dBm (25 mW) ERP (*Effective Radiated Power*)
- Duty cycle: Tidak secara eksplisit dibatasi seperti di Eropa, namun penggunaan harus mematuhi prinsip *non-interference* dan *non-exclusive*.
- Modulasi: Diizinkan untuk teknologi *spread spectrum*, termasuk LoRa.

Perlu dicatat bahwa meskipun pita ini mengacu pada profil AS923, implementasi LoRaWAN di Indonesia sering kali menyesuaikan sub-band yang diizinkan. Misalnya, hanya channel 0–7 (923.2–923.8 MHz) yang umum digunakan karena ketersediaan spektrum dan kebijakan lokal. Beberapa *network server* (seperti The Things Network/TTN) menyediakan profil khusus “AS923-1” atau “AS923-ID” untuk menyesuaikan dengan regulasi nasional.

Penggunaan frekuensi di luar ketentuan Kominfo dapat dikenai sanksi administratif atau pencabutan izin, sehingga penting bagi pengembang IoT untuk memastikan perangkat LoRaWAN mereka mematuhi batasan daya, frekuensi, dan protokol yang ditetapkan.

2.2.21 RadioLib

RadioLib merupakan sebuah open-source library yang dikembangkan untuk memfasilitasi komunikasi nirkabel pada berbagai modul radio, terutama yang digunakan dalam platform mikrokontroler seperti Arduino dan ESP32. Library ini dirancang untuk menyederhanakan proses pengembangan sistem komunikasi jarak jauh dengan menyediakan antarmuka pemrograman yang konsisten dan komprehensif untuk berbagai jenis modul radio, termasuk LoRa, FSK, OOK, dan lainnya (Gromes, 2025). Dengan dukungan terhadap berbagai chip transceiver seperti SX127x, SX126x, RF69, dan CC1101, RadioLib memungkinkan pengembang untuk mengimplementasikan protokol komunikasi nirkabel secara efisien tanpa harus menulis kode tingkat rendah secara manual.

Salah satu keunggulan utama RadioLib adalah abstraksi perangkat keras yang memungkinkan portabilitas kode antar platform. Hal ini sangat penting dalam pengembangan sistem Internet of Things (IoT), di mana fleksibilitas dan skalabilitas menjadi pertimbangan utama. Menurut dokumentasi resminya, RadioLib menyediakan fungsi-fungsi tingkat tinggi untuk konfigurasi modul, pengiriman dan penerimaan data, serta pengelolaan status perangkat, sehingga mempercepat siklus pengembangan dan pengujian (Gromes, 2025)

Dalam konteks pendidikan dan penelitian, penggunaan library seperti RadioLib memungkinkan mahasiswa dan peneliti untuk fokus pada aspek desain sistem dan protokol komunikasi, bukan pada kompleksitas implementasi perangkat keras.

Sebagai contoh, dalam eksperimen berbasis LoRa, RadioLib memungkinkan pengaturan parameter seperti spreading factor, bandwidth, dan coding rate melalui fungsi-fungsi sederhana, sehingga memudahkan eksplorasi karakteristik jaringan LoRaWAN (Gromes, 2025).

Perbandingan dengan library lain seperti LoRa.h (yang hanya mendukung modul berbasis SX127x) menunjukkan bahwa RadioLib menawarkan cakupan perangkat yang lebih luas dan konsistensi antarmuka lintas chip. Hal ini menjadikannya pilihan yang lebih fleksibel untuk proyek-proyek yang memerlukan interoperabilitas atau migrasi antar platform perangkat keras (Gromes, 2025).

2.2.22 Gateway

Gateway merupakan komponen kritis dalam arsitektur jaringan modern, berfungsi sebagai titik perantara yang menghubungkan dua atau lebih jaringan yang berbeda, baik dalam hal protokol, arsitektur, maupun teknologi fisik. Secara umum, gateway bertindak sebagai perangkat jaringan yang mampu menerjemahkan komunikasi antar sistem heterogen, sehingga memungkinkan interoperabilitas antara perangkat yang menggunakan standar berbeda (Tanenbaum & Wetherall, 2021).

Dalam konteks jaringan berbasis Internet of Things (IoT), gateway sering kali menjadi penghubung antara perangkat edge (seperti sensor atau aktuator) dengan infrastruktur cloud atau jaringan inti. Perangkat ini tidak hanya meneruskan data, tetapi juga dapat melakukan pra-pemrosesan, agregasi, enkripsi, dan manajemen protokol. Misalnya, dalam arsitektur LoRaWAN salah satu teknologi LPWAN (Low Power Wide Area Network) gateway berfungsi menerima sinyal radio dari node end-device menggunakan modulasi LoRa, lalu meneruskannya ke server jaringan melalui koneksi backhaul berbasis IP seperti Ethernet, Wi-Fi, atau seluler. (Adelantado et al., 2017)

Fungsi utama gateway mencakup konversi protokol (protocol conversion), routing, dan keamanan jaringan. Berbeda dengan router atau switch yang beroperasi pada lapisan jaringan atau lapisan data link dalam model OSI, gateway umumnya beroperasi pada lapisan aplikasi (lapisan 7), sehingga mampu memahami dan memproses konten pesan secara semantik (Forouzan, 2013). Hal ini menjadikan

gateway lebih kompleks dibanding perangkat jaringan lainnya, namun juga lebih fleksibel dalam mengintegrasikan sistem yang beraga.

Dalam implementasi dasar (basic gateway), fungsionalitas biasanya dibatasi pada penerusan data tanpa pemrosesan lanjutan. Contohnya adalah gateway LoRaWAN sederhana yang hanya menerima frame LoRa dan meneruskannya ke The Things Network (TTN) atau server LoRaWAN lain melalui protokol UDP. Meskipun demikian, bahkan gateway dasar tetap memerlukan konfigurasi jaringan yang tepat, sinkronisasi waktu, serta manajemen keamanan minimal untuk mencegah serangan seperti spoofing atau denial-of-service (Augustin et al., 2016).

2.2.23 Heltec Tracker V1.1

Heltec Tracker V1.1 adalah perangkat pelacak berbasis LoRa (Long Range) yang dikembangkan oleh Heltec Automation. Perangkat ini dirancang untuk aplikasi IoT berdaya rendah dan jangkauan jauh, seperti pelacakan aset, kendaraan, atau logistik. Heltec Tracker V1.1 mengintegrasikan modul LoRa SX1276, mikrokontroler ESP32, serta sensor pendukung seperti GNSS (Global Navigation Satellite System) untuk menentukan posisi geografis secara real-time. (Automation, 2021)

Perangkat ini mendukung protokol komunikasi LoRaWAN, sehingga kompatibel dengan jaringan publik maupun privat seperti The Things Network (TTN). Dengan konsumsi daya rendah dan kemampuan transmisi data hingga beberapa kilometer (tergantung kondisi lingkungan), Heltec Tracker V1.1 menjadi pilihan populer dalam implementasi sistem IoT berbasis lokasi (Zhang et al., 2022). Fitur utama Heltec Tracker V1.1 meliputi:

- Mikrokontroler ESP32 dengan dukungan Wi-Fi dan Bluetooth (meski dalam mode LoRaWAN, fitur ini sering dinonaktifkan untuk menghemat daya),
- Modul LoRa SX1276 yang mendukung frekuensi sub-GHz (misalnya 868 MHz di Eropa atau 915 MHz di Amerika),
- Modul GNSS (biasanya u-blox NEO-6M atau sejenis) untuk akuisisi koordinat GPS,
- Baterai isi ulang dengan manajemen daya cerdas,
- Antarmuka USB untuk pemrograman dan debugging (Chengdu, 2023)

Dalam konteks akademik dan penelitian, Heltec Tracker V1.1 telah digunakan dalam berbagai studi, termasuk sistem pelacakan logistik berbasis LoRaWAN (Kurniawan & Setiawan, 2023) dan pemantauan lingkungan bergerak (mobile environmental monitoring). (Shi et al., 2022) Keunggulan utamanya terletak pada integrasi perangkat keras yang lengkap dalam satu board, mempermudah pengembangan prototipe tanpa perlu merancang PCB tambahan.

Namun, tantangan utama dalam penggunaannya meliputi optimasi konsumsi daya (terutama saat GNSS aktif terus-menerus) dan konfigurasi OTAA/ABP pada jaringan LoRaWAN, yang sering menjadi hambatan bagi pemula (seperti mahasiswa yang baru mempelajari TTN dan pemrograman LoRaWAN).

2.2.24 The Things Network (TTN) Intregation

Sistem komunikasi berbasis LoRaWAN (Long Range Wide Area Network) telah menjadi pilihan utama dalam pengembangan solusi Internet of Things (IoT) berdaya rendah dan jangkauan jauh. Dalam implementasi ini, perangkat *end-node* berbasis Heltec Tracker V1.1 yang menggunakan modul radio SX1262 dikonfigurasi untuk berkomunikasi dengan infrastruktur jaringan LoRaWAN melalui The Things Network (TTN), sebuah platform jaringan LoRaWAN berbasis cloud yang bersifat terbuka dan komunitas-driven (The Things Industries, 2025).

Komunikasi antara perangkat dan jaringan diimplementasikan menggunakan pustaka RadioLib, yaitu pustaka universal untuk komunikasi nirkabel pada perangkat *embedded* seperti Arduino. RadioLib menyediakan dukungan penuh terhadap protokol LoRaWAN, termasuk mekanisme aktivasi Over-The-Air Activation (OTAA), manajemen sesi, enkripsi end-to-end, serta penanganan *uplink* dan *downlink* sesuai spesifikasi LoRaWAN 1.0.3 (Gromes, 2025a). OTAA dipilih karena menawarkan keamanan yang lebih tinggi dibandingkan metode ABP (*Activation by Personalization*), dengan proses autentikasi dinamis yang melibatkan pertukaran kunci jaringan (*NwkKey*) dan kunci aplikasi (*AppKey*) selama fase *join*.

Konfigurasi perangkat mencakup definisi pin SPI dan kontrol khusus untuk modul SX1262, seperti *chip select* (CS), *reset* (RST), *busy* (BUSY), dan *digital*

input/output (DIO1). Parameter wilayah frekuensi disetel ke AS923, yang merupakan alokasi spektrum LoRaWAN yang berlaku di wilayah Asia Tenggara, termasuk Indonesia. Hal ini penting untuk memastikan kepatuhan terhadap regulasi spektrum nasional dan kompatibilitas dengan *gateway* yang tersedia di wilayah tersebut. Pada sisi keamanan, perangkat dikonfigurasi dengan tiga parameter kriptografi utama:

- DevEUI (*Device Extended Unique Identifier*): identitas unik 64-bit perangkat,
- AppKey: kunci 128-bit untuk enkripsi lapisan aplikasi,
- NwkKey: kunci 128-bit untuk enkripsi lapisan jaringan.

Pada The Things Stack versi 3 (TTN v3), nilai *NwkKey* secara fungsional identik dengan *AppKey*, sehingga kedua parameter tersebut menggunakan nilai yang sama yang dihasilkan dari antarmuka TTN Console (The Things Industries, 2025). Setelah proses *join* berhasil—ditandai dengan kode status `RADIOLIB_LORAWAN_NEW_SESSION` (nilai -1118) perangkat memasuki sesi aktif dan siap mengirim data.

Dalam siklus operasi, perangkat mengirim *payload* berukuran 3 byte setiap 10 detik. Payload terdiri dari:

- 1 byte untuk nilai sensor simulasi pertama (*value1*),
- 2 byte (MSB dan LSB) untuk nilai sensor simulasi kedua (*value2*).

Data dikirim melalui metode `sendReceive()`, yang secara otomatis membuka jendela penerimaan (*RX1* dan *RX2*) untuk memungkinkan penerimaan *downlink* dari server jaringan. Setiap transmisi diverifikasi melalui kode status `RadioLib`; misalnya, `RADIOLIB_ERR_NO_JOIN_ACCEPT` (-1116) menunjukkan kegagalan menerima respons *JoinAccept* dari jaringan, yang umumnya disebabkan oleh ketidaksesuaian kunci atau jangkauan sinyal yang tidak memadai (Gromes et al., 2025)

Verifikasi keberhasilan transmisi dilakukan melalui antarmuka TTN Console. Data yang diterima ditampilkan dalam format JSON dengan *frame payload* berupa string Base64. Sebagai contoh, payload "BgCV" didekode menjadi urutan heksadesimal 06 07 15, yang merepresentasikan nilai numerik 6 dan 1813—

sesuai dengan logika pengkodean di sisi perangkat. Hal ini menunjukkan bahwa enkapsulasi data, enkripsi, transmisi, dan dekripsi di sisi server berjalan sesuai spesifikasi LoRaWAN.

Pustaka RadioLib juga menyediakan mekanisme diagnosis kesalahan yang komprehensif melalui serangkaian kode status terstandarisasi. Misalnya, `RADIOLIB_ERR_CHIP_NOT_FOUND` (-2) mengindikasikan kesalahan pada koneksi perangkat keras, sedangkan `RADIOLIB_ERR_MIC_MISMATCH` (-1112) menandakan ketidaksesuaian *Message Integrity Code* (MIC), yang dapat disebabkan oleh ketidaksesuaian kunci atau *frame counter* yang tidak sinkron (Gromes et al., 2025). Fungsi bantu seperti `stateDecode()` digunakan untuk memetakan kode numerik menjadi pesan deskriptif, mempermudah proses *debugging* selama pengembangan.

Secara keseluruhan, implementasi ini menunjukkan integrasi yang robust antara perangkat *end-node* berbasis SX1262, pustaka RadioLib, dan infrastruktur The Things Network. Arsitektur ini tidak hanya memenuhi prinsip keamanan dan efisiensi daya LoRaWAN, tetapi juga memberikan fondasi yang dapat dikembangkan lebih lanjut untuk aplikasi IoT riil, seperti pemantauan lingkungan, pelacakan aset, atau sistem pertanian presisi.

2.2.25 SIM900A

SIM900A adalah modul GSM/GPRS miniatur yang dikembangkan oleh SIMCom Wireless Solutions. Modul ini mendukung komunikasi suara, SMS, dan data GPRS pada frekuensi quad-band (850/900/1800/1900 MHz), sehingga memungkinkan konektivitas global di jaringan 2G. Modul ini sering digunakan dalam sistem IoT berbasis seluler karena konsumsi daya rendah, ukuran kecil, dan antarmuka serial TTL yang kompatibel dengan mikrokontroler seperti ESP32, Arduino, dan STM32. (Maurya et al., 2012)

Karakteristik Teknis (berdasarkan Datasheet SIM900L)

- Tegangan Operasi: 3.4 – 4.4 V
- Arus Puncak: ~2 A saat transmisi RF (namun rata-rata < 300 mA)
- Antarmuka: UART (TTL level, 3.3V logic)

- Protokol AT: Mendukung perintah AT standar untuk panggilan, SMS, dan GPRS
- Kecepatan Data GPRS: hingga 85.6 kbps (downlink)

Modul ini memerlukan antena eksternal (biasanya PCB trace antenna atau whip antenna) dan power supply stabil karena lonjakan arus saat registrasi jaringan dapat menyebabkan reset jika tidak didukung kapasitor buffer atau regulator berkualitas.

2.2.26 Modul Kartu SD

Modul kartu SD memungkinkan mikrokontroler untuk menyimpan dan membaca data dalam format file sistem (biasanya FAT16/FAT32). Modul ini berkomunikasi melalui protokol SPI (Serial Peripheral Interface), yang ringan dan didukung luas oleh platform mikrokontroler modern. (Satheesh et al., 2016)

Karakteristik Teknis (berdasarkan spesifikasi SD Card & Modul Adapter)

- Tegangan Operasi: 3.3 V (kartu SD asli); modul adapter biasanya memiliki regulator onboard untuk input 5V
- Protokol Komunikasi: SPI mode (default untuk mikrokontroler)
- Kecepatan Transfer: Tergantung kelas kartu (Class 4, 6, 10, UHS-I)
- Kapasitas: Mendukung hingga 32 GB (SDHC) secara native; SDXC (>32 GB) memerlukan driver tambahan

Modul ini terdiri dari:

- Slot fisik untuk kartu microSD
- Regulator tegangan (misal: AMS1117-3.3)
- Level shifter (opsional, untuk kompatibilitas 5V logic)

Pertimbangan Desain

- Gunakan pull-up resistor pada CS (Chip Select) untuk stabilitas
- Pastikan clock SPI ≤ 20 MHz (direkomendasikan ≤ 10 MHz untuk keandalan)
- Format kartu menggunakan FAT32 agar kompatibel dengan pustaka seperti SD.h (Arduino) atau sdmmc (ESP-IDF)

2.2.27 MP3 Player Module

MP3 Player Module adalah perangkat elektronik yang dirancang untuk memainkan file audio berformat MP3 (MPEG-1 Audio Layer III) atau format lain seperti WAV dan ADPCM dari media penyimpanan eksternal seperti microSD card atau USB flash drive. Modul ini dilengkapi dengan prosesor audio internal, DAC (Digital to Analog Converter), antarmuka komunikasi serial (UART), serta output suara langsung ke speaker atau headphone. Dengan ukuran fisik yang kecil dan konsumsi daya rendah, MP3 Player Module banyak digunakan dalam proyek DIY, alat edukasi, sistem informasi berbasis suara, robot interaktif, dan perangkat IoT.

MP3 Player Module bekerja melalui beberapa tahapan proses, yaitu:

1. Pembacaan File Audio :

Modul membaca file audio dari kartu microSD menggunakan protokol SPI atau SDIO. File harus disimpan dalam struktur tertentu agar dapat dikenali oleh firmware modul.

2. Dekode Format Audio :

Prosesor audio mendekode file audio sesuai formatnya (MP3/WAV/ADPCM) menjadi sinyal PCM (Pulse Code Modulation).

3. Konversi Digital ke Analog :

Sinyal digital hasil dekoding dikonversi menjadi sinyal analog menggunakan DAC (Digital to Analog Converter).

4. Output Suara :

Sinyal analog dialirkan ke speaker melalui penguat onboard atau ke headphone secara langsung.

5. Kontrol Serial :

Modul menerima perintah kontrol dari mikrokontroler via UART untuk menjalankan fungsi seperti play, pause, next track, volume

2.2.28 Speaker

Modul Amplifier PAM8403 merupakan penguat audio digital kelas D yang dikembangkan oleh Texas Instruments, dirancang untuk memberikan kualitas suara yang jernih dengan efisiensi daya yang tinggi. Modul ini menggunakan teknologi *Pulse Width Modulation (PWM)* untuk memperkuat sinyal audio tanpa kehilangan

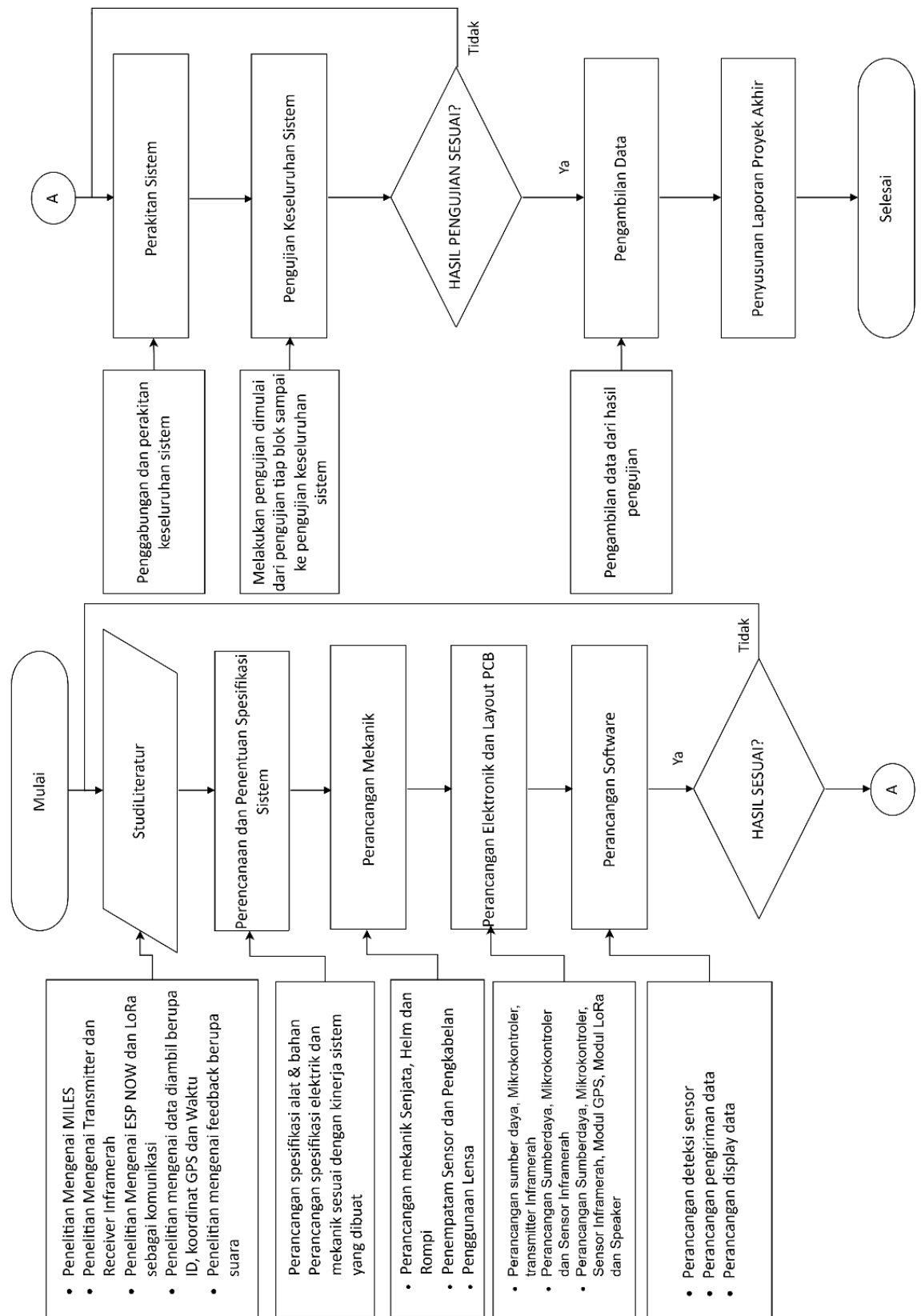
banyak energi dalam bentuk panas, sehingga sangat cocok digunakan dalam perangkat audio portabel seperti speaker mini, headphone, sistem audio mobil, hingga proyek DIY elektronika. PAM8403 memiliki kemampuan untuk menghasilkan daya output hingga 3 watt per channel (dengan beban 4Ω) pada tegangan suplai sekitar 5 volt, menjadikannya ideal untuk aplikasi berdaya rendah namun tetap membutuhkan kualitas suara yang baik. Modul ini mendukung dua saluran audio (stereo) dan dilengkapi dengan fitur perlindungan terhadap overheating, overcurrent, serta under-voltage, sehingga meningkatkan keandalan dan umur pakai perangkat. Selain itu, desainnya yang kompak dan konsumsi daya yang rendah membuat PAM8403 banyak digunakan dalam integrasi sistem berbasis mikrokontroler seperti Arduino, ESP32, Raspberry Pi, maupun perangkat audio miniaturisasi lainnya. Penggunaannya yang luas di berbagai bidang menunjukkan bahwa modul amplifier PAM8403 merupakan solusi efektif untuk kebutuhan penguatan suara dalam sistem audio digital modern.

BAB III

METODOLOGI

3.1 Kerangka Konsep Proyek Akhir

Proyek akhir ini dilakukan melalui beberapa tahap. Tahap pertama adalah studi literatur, dimana peneliti mengumpulkan dan menganalisis teori-teori yang berkaitan dengan topik proyek akhir dari buku, jurnal, artikel, dan proyek akhir sebelumnya. Selanjutnya, konsep alat dirancang dan spesifikasinya ditentukan. Tahap ketiga adalah perancangan rangkaian elektronik. Kemudian, desain mekanik sebagai casing alat dibuat. Setelah itu, rangkaian elektronik dirakit ke dalam mekanik dan dilakukan pengujian fungsionalitas. Tahap keenam, program untuk keseluruhan sistem dikembangkan dan diuji coba. Selanjutnya, pengambilan data dilakukan dan dilakukan analisa untuk menghasilkan kesimpulan dari alat yang telah dibuat. Terakhir, laporan akhir disusun berdasarkan hasil dari proyek akhir ini.



Gambar 17 Kerangka Konsep penyusunan proyek akhir

Berikut merupakan flowchart dari kerangka konsep perancangan alat dapat dilihat pada Gambar 12.

Adapun Deskripsi Alur Perancangan Alat sebagai berikut

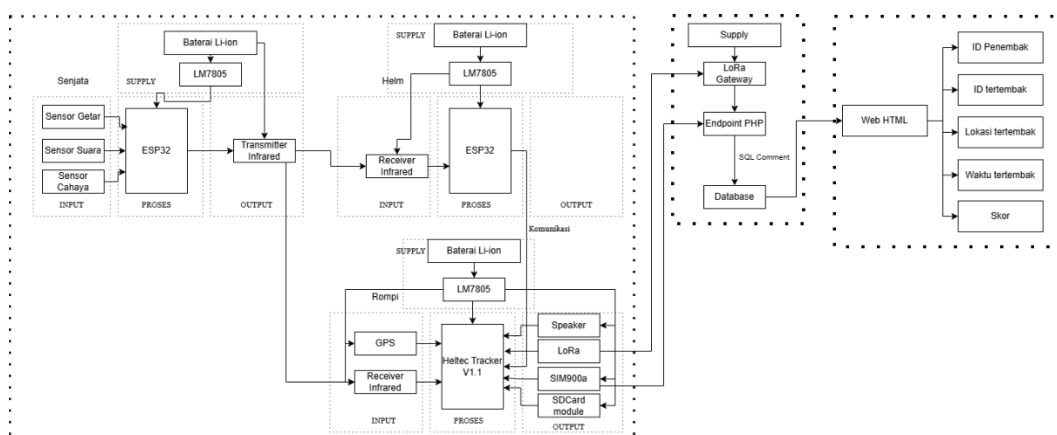
- a. Mulai, mempersiapkan segala kebutuhan proyek akhir
- b. Studi Literatur, Studi proyek akhir mengenai MILES sebagai sistem permainan berbasis sinyal *infrared*, studi proyek akhir mengenai transmitter dan receiver *infrared*, Studi Proyek akhir mengenai komunikasi nirkabel menggunakan ESP-NOW dan LoRa, studi proyek akhir mengenai jenis data yang dikirim, seperti ID, koordinat GPS, dan waktu serta studi proyek akhir mengenai feedback suara yang diberikan kepada pengguna.
- c. Perencanaan dan Penentuan Spesifikasi Sistem, merumuskan spesifikasi alat secara rinci meliputi aspek elektrik dan mekanik, disesuaikan dengan kinerja sistem yang diinginkan.
- d. Perancangan Mekanik, mendesain fisik senjata, helm, dan rompi. Termasuk menentukan penempatan sensor, jalur pengkabelan, dan penggunaan lensa untuk optimasi penerimaan sinyal.
- e. Perancangan Elektronik dan Layout PCB, membuat desain skema rangkaian dan tata letak PCB yang mencakup sumber daya, mikrokontroler, dan transmitter *infrared* pada bagian senjata, Sumber daya, mikrokontroler, dan sensor *infrared* pada bagian helm, sumber daya, mikrokontroler, sensor *infrared*, modul GPS, modul LoRa, dan speaker pada bagian rompi
- f. Perancangan Software, mengembangkan logika sistem meliputi perancangan implementasi logika sistem pada mikrokontroler untuk deteksi sensor dan mengendalikan output, pengiriman data serta menampilkan data pada web interface.
- g. Pengujian Awal Komponen, Setelah perancangan selesai, dilakukan pengujian awal pada setiap komponen utama secara terpisah. Hal ini bertujuan untuk memastikan setiap subsistem, seperti sensor, aktuator, dan driver, berfungsi dengan baik sebelum diintegrasikan.
- h. Perakitan Sistem, jika pengujian awal berhasil, semua komponen yang telah dirancang dan diuji dirakit menjadi sebuah prototipe fisik..

- i. Pengujian Sistem Keseluruhan, apakah alat berjalan baik tanpa kendala? jika ada kendala dilakukan perbaikan, kalibrasi dan pengecekan hingga bekerja normal. Setelah pengujian alat, akan mendapatkan hasil yaitu alat berjalan baik atau tidak..
- j. Pengambilan Data, dari hasil pengujian yang dilakukan diambil data mengenai performa system secara keseluruhan.
- k. Penyusunan laporan akhir, melakukan pengujian dan mencatat hasil serta analisa alat yang sudah selesai dalam bentuk laporan.
- l. Selesai.

Secara konsep, kerangka yang ditampilkan pada Gambar 12 menggambarkan tahapan pengembangan sistem secara terstruktur dari awal hingga akhir. Jika semua tahapan berhasil, maka Implementasi Rangkaian Elektronik Dan Sistem Komunikasi Multi-Pemain Untuk Permainan *Laser Tag* Menggunakan ESP-NOW dan LoRa dinyatakan siap digunakan. Konsep ini bersifat iteratif, artinya setiap hasil pengujian yang tidak sesuai akan dikembalikan ke tahap sebelumnya untuk diperbaiki.

3.2 Perancangan Sistem

Penentuan spesifikasi sistem dilakukan sebagai dasar dalam proses pengimplementasian permainan laser tag. Spesifikasi ini disusun berdasarkan kebutuhan fungsional sistem, komponen yang digunakan, serta tujuan utama dari proyek akhir, yaitu mendeteksi tembakan *infrared* dan mencatat skor secara real-time melalui sensor IR yang kemudian datanya dikirim ke server menggunakan komunikasi ESP-NOW dan LoRa untuk ditampilkan pada Web Interface atau GUI monitoring.



Gambar 18 Perancangan Sistem

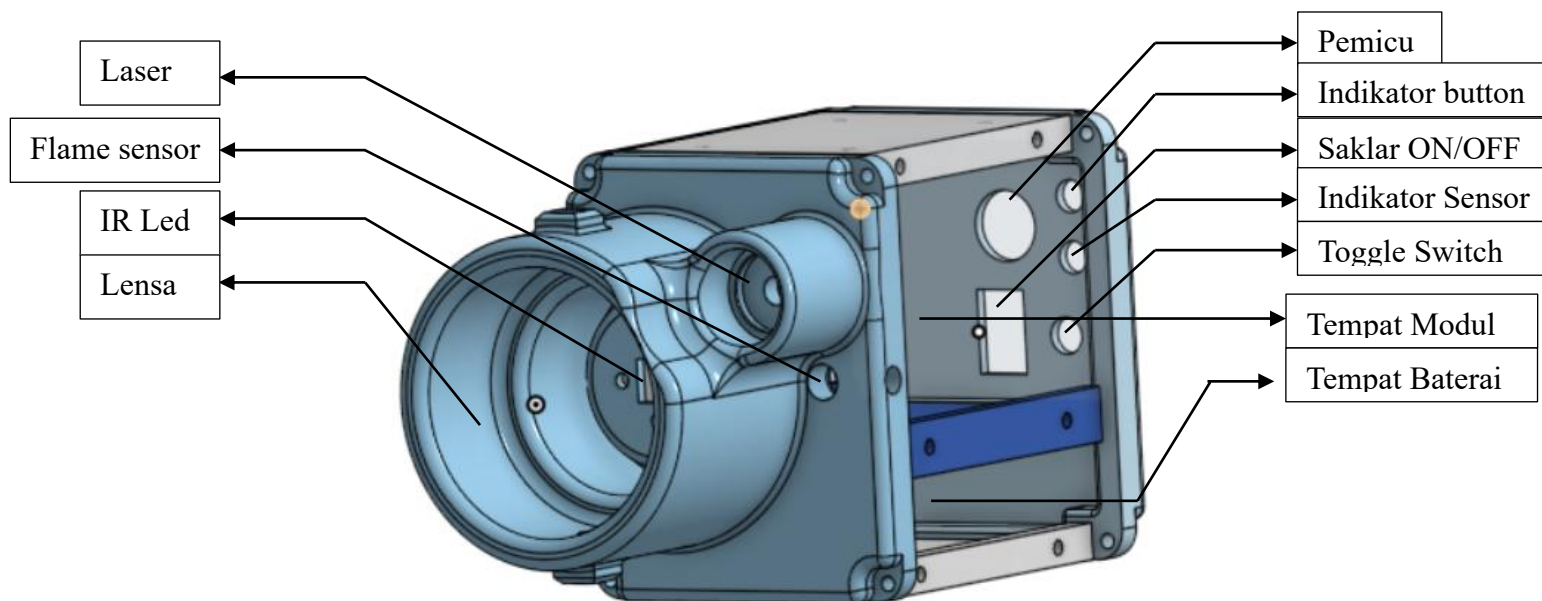
Prinsip kerja sistem laser tag berdasarkan gambar 18 Sistem laser tag IoT diawali ketika seluruh unit perangkat keras yaitu SAT, Helm, dan Rompi dinyalakan dan siap beroperasi. Ketika pemain menekan push button utama pada unit SAT, mikrokontroler ESP32-S3 SuperMini akan memeriksa apakah ketiga sensor aktivasi yaitu sensor suara, sensor getar, dan sensor api terpenuhi secara bersamaan sebagai syarat wajib sebelum tembakan dilepaskan. Apabila seluruh sensor aktif, SAT memancarkan sinyal infrared berprotokol NEC yang membawa data ID penembak melalui LED yang difokuskan lensa Fresnel ke arah lawan. Sinyal tersebut dapat diterima oleh Helm maupun Rompi. Jika Helm yang terkena, ESP32-S3 SuperMini pada Helm melakukan validasi protokol NEC sekaligus pemeriksaan anti-cheating melalui photodiode, dan apabila valid, data HIT diteruskan ke Rompi menggunakan komunikasi nirkabel ESP-NOW. Jika Rompi yang menerima sinyal secara langsung, data langsung diolah oleh mikrokontroler Heltec Wireless Tracker V1.2 yang menggabungkan informasi ID penembak, lokasi sensor, waktu tembakan, dan koordinat GPS dari modul GPS internal.

Setelah Heltec selesai memproses data, sistem secara paralel langsung menyimpan seluruh data ke kartu SD sebagai backup lokal permanen, kemudian mencoba mengirimkan data melalui jalur komunikasi berdasarkan prioritas. Jalur utama adalah GSM menggunakan modul SIM900A yang mengirimkan data ke platform ThingSpeak, kemudian ThingSpeak meneruskannya ke The Things Network (TTN). Apabila jaringan GSM tidak tersedia, sistem beralih ke jalur cadangan menggunakan LoRaWAN yang mengirimkan data langsung ke TTN.

Setelah data berhasil diterima TTN dari jalur manapun, TTN meneruskan payload melalui webhook ke platform Vercel yang berperan sebagai backend sekaligus frontend hosting. Vercel memproses data tersebut dan menyimpannya ke database PostgreSQL melalui layanan NeonDB, lalu mendorong pembaruan secara real-time ke antarmuka dashboard menggunakan teknologi Server-Sent Events (SSE) sehingga status pemain, skor, riwayat tembakan, dan posisi pemain dapat dipantau langsung tanpa perlu refresh halaman hingga pertandingan dinyatakan selesai

3.3 Perancangan Mekanik

3.3.1 Perancangan SAT



Gambar 19 Perancangan SAT

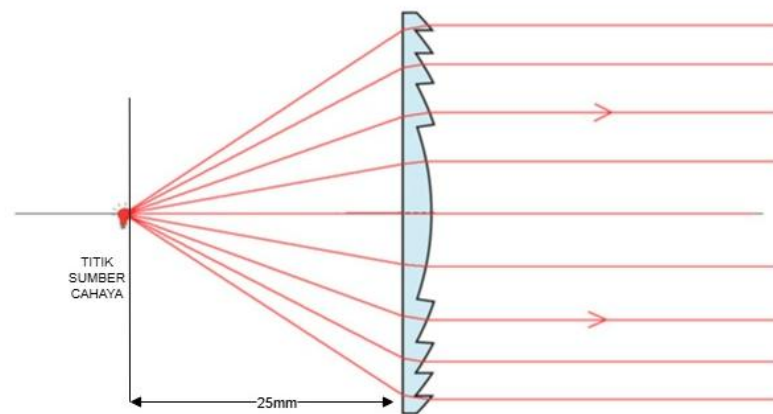
Perancangan Lensa

Untuk membuat cahaya yang diteruskan oleh lensa Fresnel menjadi paralel, kita perlu memastikan bahwa sumber cahaya (*focal length* atau jarak fokus) ditempatkan pada titik fokus lensa Fresnel. Lensa Fresnel dirancang untuk memfokuskan atau mengkolinearakan (membuat sejajar) cahaya dengan cara yang mirip dengan lensa konvensional.

Dengan parameter yang diketahui berupa material lensa yakni *polycarbonate* , diameter lensa sebesar 20mm dan radius lengkung permukaan lensa 14.5mm, maka *focal length* / titik focus cahaya dapat dirumuskan dengan persamaan 1.9. Dengan demikian titik fokus lensa sebagai berikut:

$$f = \frac{14.5mm}{1.58 - 1} = 25mm$$

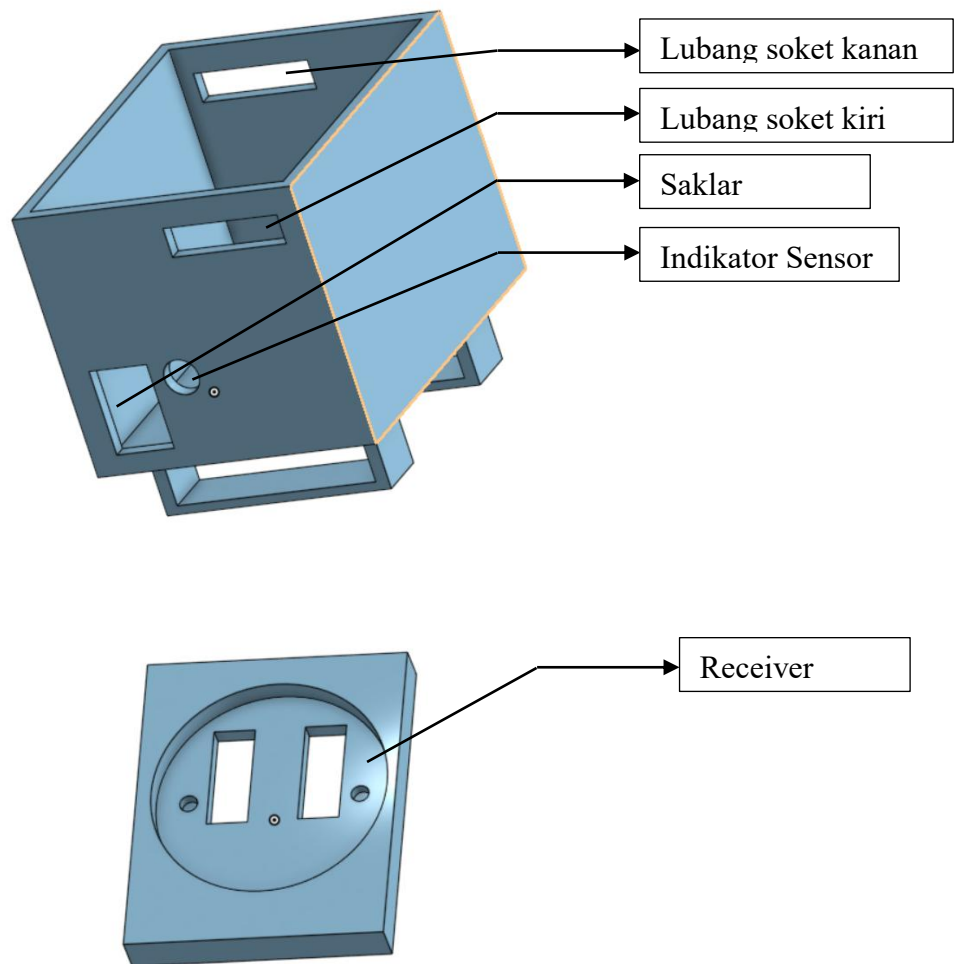
Nilai 1.58 merupakan indeks bias material lensa yakni *polycarbonate*. Didapatkan *focal length* sebesar 25mm. Dapat dilihat pada Gambar 3.30



Gambar 20 Focal Length Lensa Fresnel

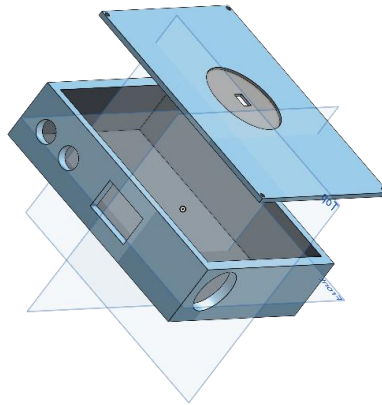
Dimana dengan jarak *focal length* sesuai dengan perhitungan, maka akan menghasilkan cahaya yang paralel dengan lebar lensa.

3.3.2 Perancangan Box pada Helm



Gambar 21 Perancangan Helm

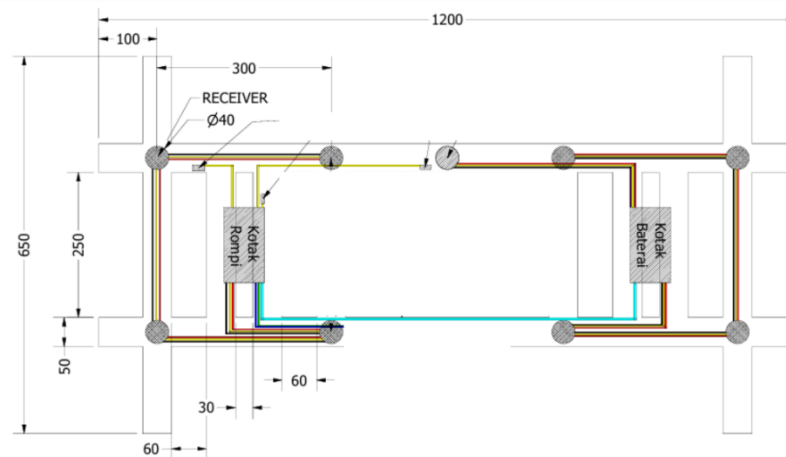
3.3.3 Perancangan pada Rompi Box



Gambar 22 Perancangan Box pada Rompi

Desain alat mencakup pengembangan desain fisik dari alat ini, mulai dari konsep struktur hingga detail implementasi. Sistem ini didesain agar semua komponen dalam masuk dan sesuai dengan seluruh kebutuhan sistem yang dibuat untuk rompi dan helm. Dimensi dibuat secompact mungkin agar memaksimalkan dimensi dan ruangan yang dibutuhkan dan tidak mengganggu pergerakan pada saat permainan.

Design Rompi



Gambar 23 Design Rompi

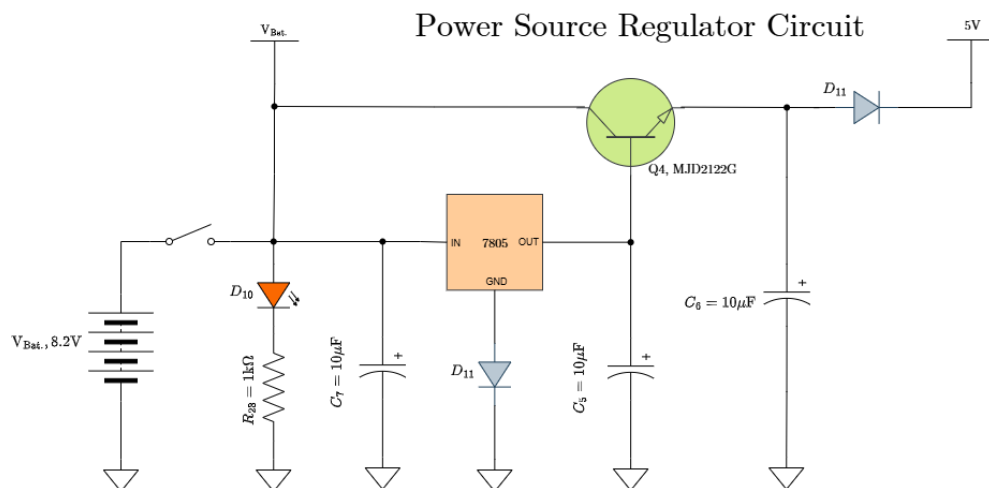
Pada merupakan desain mekanik pada sistem rompi dimana menggunakan bahan kain yang dijahit dengan model dan ukuran kurang lebih seperti pada gambar. Terdapat 8 buah jumlah lubang yang terdapat pada rompi tersebut digunakan sebagai tempat atau wadah dari sensor penerima inframerah TSOP4838. Pada lubang tersebut sensor ditutup menggunakan lensa cembung yang berfungsi untuk memfokuskan Cahaya atau sinyal yang diterima. Keberadaan lensa tersebut

cukup penting mengingat kemampuan sensor menerima hanya berkisar kurang lebih 45°.

3.4 Perancangan Elektronik

3.4.1 Perancangan Rangkaian Power Source Regulator Circuit

Perancangan rangkaian regulator sumber daya yang menggunakan baterai lithium-ion Black Cell tipe 14500 dalam konfigurasi dua sel seri (2S). Susunan seri ini menghasilkan tegangan nominal 7,4 V ($2 \times 3,7\text{V}$ per sel), dengan rentang operasional antara 6 V hingga 8,2 V. Baterai berkapasitas 5.000 mAh ini mampu menyediakan arus hingga 15 A, memadai untuk kebutuhan sistem. Tegangan keluaran baterai diatur menjadi 5 V stabil menggunakan IC regulator 7805, yang dikombinasikan dengan transistor MJD2122G sebagai penguat arus untuk meningkatkan kemampuan arus keluaran regulator. Rangkaian ini juga dilengkapi dioda D10–D11 dan kapasitor C1–C4 untuk menyaring noise serta memperkuat stabilitas respons transient terhadap perubahan beban.



Gambar 24 Power Source Regulator

Sistem power regulator ini menyuplai tegangan 5V untuk seluruh komponen elektronik SAT dan HARNESS, termasuk ESP32 S3 Supermini, berbagai sensor, IC, LED infrared, serta transistor/MOSFET. Berdasarkan perhitungan asumsi arus maksimum totalnya adalah 290mA.sebagai berikut:

Tabel 3 Asumsi arus

KOMPONEN	ASUMSI ARUS
ESP32S3 Supermini	80–150mA
Sensor Suara	20–40mA
Sensor Cahaya	15–30mA
Sensor Getaran	15–30mA
IC 74HC4078	5–10 mA
Laser	10–20 mA
BC847	1-5mA
MCP1406	1-5mA

Berdasarkan pengukuran tegangan keluaran $V_{\text{out}} \approx 5,2\text{V}$, nilai tersebut masih dalam toleransi regulator 7805, yaitu:

$$V_{\text{out}} = 5\text{V} \times (1 \pm 0,05) = 4,75\text{V} \text{ hingga } 5,25\text{V} \quad (2.0)$$

Jika transistor Q4 tidak aktif (beban ringan), keluaran berasal langsung dari pin output 7805, sehingga:

$$V_{\text{out}} \approx V_{7805}$$

Dalam konfigurasi emitter follower (jika aktif):

$$V_{\text{out}} = V_{7805} - V_{BE} \approx 5\text{V} - 0,7\text{V} = 4,3\text{V} \quad (2.1)$$

Namun karena $V_{\text{out}} = 5,2\text{V}$, maka Q4 kemungkinan tidak beroperasi sebagai emitter follower pada kondisi pengukuran. Arus keluaran dihitung berdasarkan hukum Ohm:

$$I_{\text{out}} = \frac{V_{\text{out}}}{R_{\text{beban}}} \quad (2.2)$$

Arus basis yang diperlukan untuk mengaktifkan Q4 (jika digunakan sebagai penguat arus):

$$I_B = \frac{I_{\text{out}}}{h_{FE}} = \quad (2.3)$$

Dengan h_{FE} sebagai penguatan arus DC transistor. Daya disipasi pada Q4:

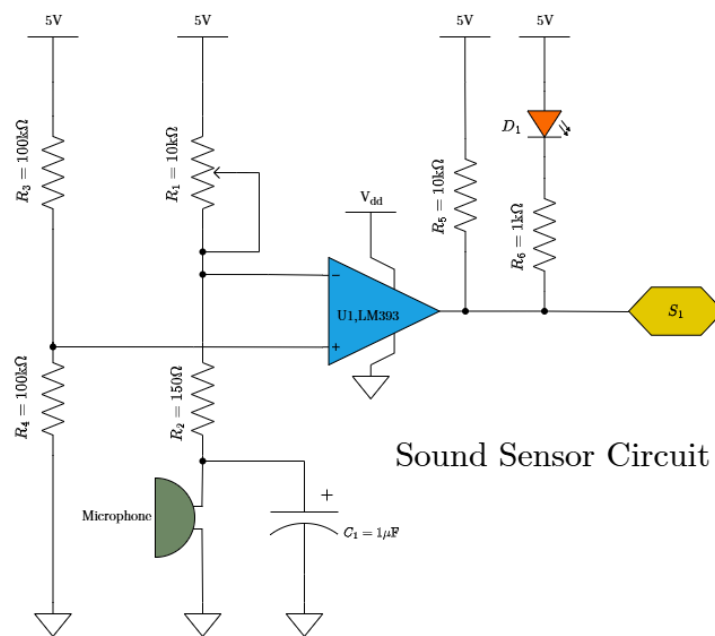
$$P_Q = (V_{in} - V_{out}) \cdot I_{out} \quad (2.4)$$

Selama arus beban kecil dan Q4 tidak jenuh, keluaran tetap mengikuti tegangan regulator 7805 tanpa penurunan V_{BE} .

3.4.2 Perancangan Rangkaian Pemancar Inframerah

a. Rangkaian Sensor Suara

Gambar 25 menampilkan rangkaian deteksi suara yang menggunakan IC LM393 sebagai komparator analog. IC LM393 merupakan komparator dua saluran yang membandingkan dua tegangan masukan dan menghasilkan keluaran logika digital berdasarkan selisih antara keduanya. Rangkaian ini dirancang untuk mengubah sinyal suara analog dari mikrofon menjadi sinyal digital berupa logika HIGH atau LOW.



Gambar 25 Rangkaian sensor suara

Tegangan masukan non-inverting (+) dari LM393 terhubung ke pembagi tegangan dari R3 dan R4, yaitu

- R3 = 100 kΩ (dari 5V ke titik A)
- R4 = 100 kΩ (dari titik A ke GND)

sehingga tegangan dapat dihitung dengan persamaan 1.2:

$$V_{ref,U1,1} = V_{cc} \times \frac{R4}{R3 + R4}$$

$$V_{+} = 5V \times \frac{100k\Omega}{100k\Omega + 100k\Omega} = 5V \times \frac{1}{2} = 2.5V$$

Kemudian arus yang digunakan pada rangkaian tersebut dihitung sebagai berikut:

$$I_{ref} = \frac{V_{CC}}{R_3 + R_4} = \frac{5V}{200k\Omega} = 0.000025A = 25\mu A \quad (2.5)$$

Tegangan masukan di invertir(-) dari LM39. Titik ini terhubung ke mic + kapasitor C_1 + resistor R_2 , dan juga ke pembagi tegangan R_1 dan R_2 . yaitu

- $R_1 = 10k\Omega$
- $R_2 = 150\Omega$

sehingga tegangan dapat dihitung dengan persamaan 1.2

$$V_{ref,U1,1} = V_{cc} \times \frac{R2}{R1 + R2}$$

$$V_{-} = 5V \times \frac{150\Omega}{10k\Omega + 150\Omega} = 5V \times \frac{150\Omega}{10150\Omega} = 0.074V = 74mV$$

Kemudian arus yang digunakan pada rangkaian tersebut dihitung seperti persamaan 2.5 sebagai berikut:

$$I_{ref} = \frac{5V}{R_1 + R_2} = \frac{5V}{10150\Omega} = 0.0004926A = 0.49 mA$$

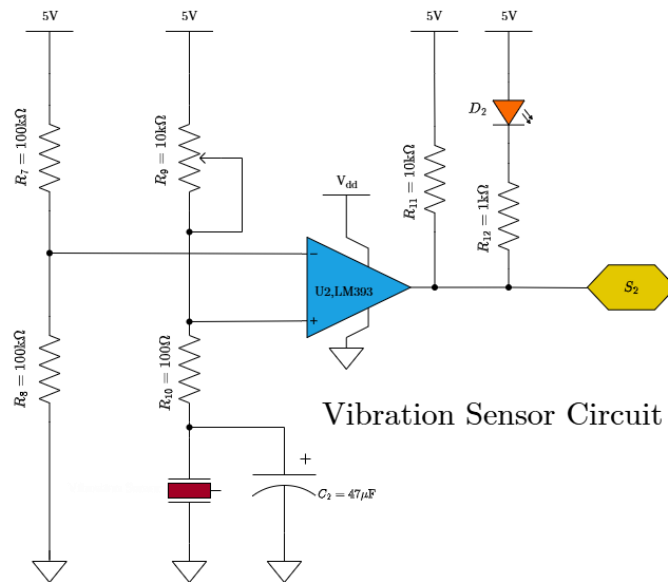
Karena tanpa suara, $pin + = 2.5V$ dan $pin - = 0.074V$ maka $pin +$ lebih besar, jadi outputnya HIGH (5V). akan tetapi, ketika ada suara, mic mengirimkan sinyal yang bisa membuat $pin -$ naik sedikit. Kalau naik melebihi 2.5V, maka output akan turun ke LOW (0V).

Tegangan di LED merah saat nyala = 2 V. Maka, tegangan jatuh $5V - 2V = 3V$. Sehingga arus LED dapat dihitung menggunakan persamaan 1.4:

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R_5 + R_6} = \frac{5V - 2V}{10k\Omega + 1k\Omega} = 0.0002727 A = 0.27 mA$$

b. Rangkaian Sensor Getar

Gambar 24 memperlihatkan rangkaian sensor getaran yang menggunakan sensor vibrasi analog dan IC komparator LM393. Rangkaian ini berfungsi mengubah sinyal fisik berupa getaran menjadi sinyal digital yaitu logika HIGH atau LOW sehingga mudah diolah oleh sistem mikrokontroler.



Gambar 26 Rangkaian Sensor Getar

Berikut adalah perhitungan dan pertimbangan desainnya:

Tegangan masukan non-inverting (+) dari LM393 terhubung ke pembagi tegangan dari R7 dan R8, yaitu

- R7 = 100 kΩ (dari 5V ke titik A)
- R8 = 100 kΩ (dari titik A ke GND)

sehingga tegangan dapat dihitung dengan persamaan 1.6:

$$V_{ref,U1,1} = V_{cc} \times \frac{R8}{R7 + R8}$$

$$V_{+} = 5V \times \frac{100k\Omega}{100k\Omega + 100k\Omega} = 5V \times \frac{1}{2} = 2.5V$$

Kemudian arus yang digunakan pada rangkaian tersebut dihitung seperti persamaan 2.5 sebagai berikut:

$$I_{ref} = \frac{5V}{R7 + R8} = \frac{5V}{200k\Omega} = 0.000025A = 25\mu A$$

Tegangan masukan di inverting (-) dari LM393. Titik ini terhubung ke mic + kapasitor C1 + resistor R2, dan juga ke pembagi tegangan R9 dan R10. yaitu

- R9 = 10kΩ
- R10 = 100kΩ

sehingga tegangan dapat dihitung dengan persamaan 1.6

$$V_{ref,U1,1} = V_{cc} \times \frac{R10}{R9 + R10}$$

$$V_{-} = 5V \times \frac{100\Omega}{10k\Omega + 100\Omega} = 5V \times \frac{100\Omega}{10100\Omega} = 0.0495V = 49.5mV$$

Kemudian arus yang digunakan pada rangkaian tersebut dihitung seperti persamaan 2.5 sebagai berikut:

$$I_{ref} = \frac{5V}{R_1 + R_2} = \frac{5V}{10100\Omega} = 0.000495A = 0.495mA$$

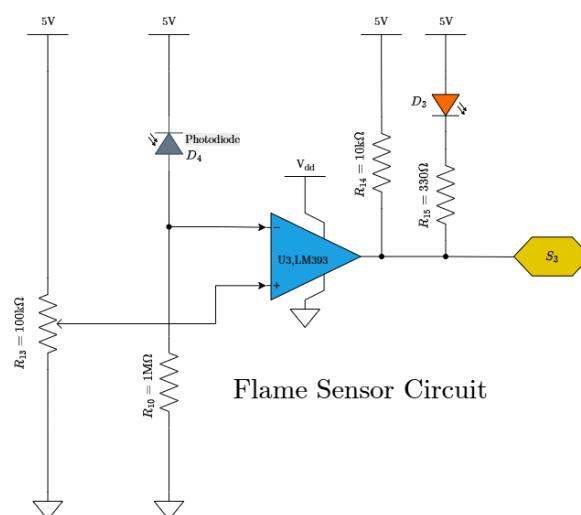
Karena tanpa suara, pin + = 2.5V dan pin - = 0.05V maka pin + lebih besar, jadi outputnya HIGH (5V). akan tetapi, ketika ada suara, mic mengirimkan sinyal yang bisa membuat pin - naik sedikit. Kalau naik melebihi 2.5V, maka output akan turun ke LOW (0V).

Tegangan di LED merah saat nyala = 2 V. Maka, tegangan jatuh $5V - 2V = 3V$. Sehingga arus LED dapat dihitung menggunakan persamaan 1.7:

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R_5 + R_6} = \frac{5V - 2V}{10k\Omega + 1k\Omega} = 0.0002727 A = 0.27 mA$$

c. Rangkaian Sensor Api

Gambar 25 menunjukkan rangkaian sensor api (flame sensor) berbasis IC LM393 yang berfungsi sebagai komparator. Rangkaian ini dirancang untuk mendeteksi keberadaan api atau sumber cahaya berintensitas tinggi dengan memanfaatkan respons optik dari photodiode terhadap cahaya inframerah yang dipancarkan oleh api.



Gambar 27 Rangkaian Sensor Api

Berikut adalah perhitungan dan pertimbangan desainnya

Resistansi photodiode berubah tergantung intensitas cahaya. Pada jarak pengujian tertentu, resistansinya terukur sekitar 900 k Ω . Resistor R_{10} dipasang bersama photodiode membentuk pembagi tegangan yang disuplai 5V. Tujuannya adalah agar tegangan di titik antara keduanya (yang diumpankan ke komparator) berada sedikit di atas ambang batas, yaitu sekitar 2,6 V. Dengan nilai $R_{\text{photodiode}} = 900 \text{ k}\Omega$, nilai ideal R_1 dapat dihitung menggunakan rumus pembagi tegangan:

$$R_1 = R_{\text{photodiode}} \cdot \frac{V_{\text{out}}}{V_{\text{in}} - V_{\text{out}}} = 900 \text{ k}\Omega \cdot \frac{2,6}{5 - 2,6} = 975 \text{ k}\Omega \quad (2.6)$$

Karena resistor dengan nilai tepat 975 k Ω sulit ditemukan di pasaran, dipilih nilai standar terdekat yang umum tersedia, yaitu 1 M Ω .

Untuk mengatur sensitivitas sensor, digunakan potensiometer $R_{13} = 100 \text{ k}\Omega$. Potensiometer ini mengatur tegangan referensi di input positif (+) komparator. Dengan memutar potensiometer, bisa menaikkan atau menurunkan ambang deteksi artinya, sensor bisa diatur agar hanya merespons api yang kuat, atau bahkan responsif terhadap cahaya IR yang lemah. Tegangan di LED merah saat nyala = 2 V. Maka, tegangan jatuh $5V - 2V = 3V$. Sehingga arus LED dapat dihitung menggunakan persamaan 1.7:

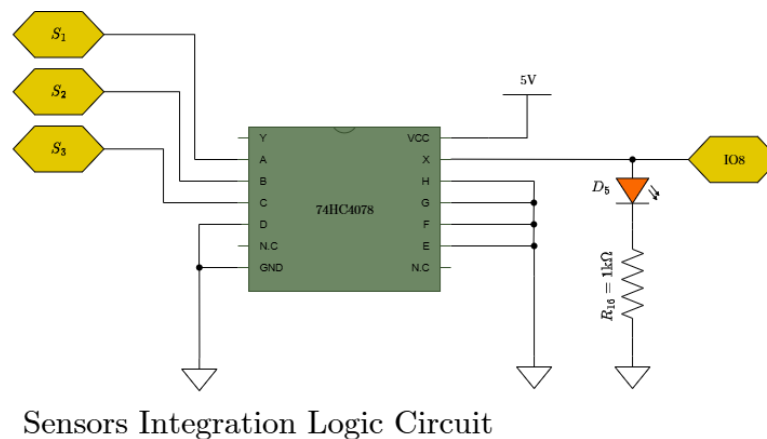
$$I_{LED} = \frac{V_{CC} - V_{LED}}{R_{14} + R_{15}} = \frac{5V - 2V}{10\text{k}\Omega + 330\Omega} = 0.000290A = 0.29mA$$

d. Rangkaian Pengkondisi sinyal

Gambar 3.4.5 merupakan rangkaian pengkondisi sinyal memanfaatkan IC 74HC4078B1R sebagai penggabung sinyal dari tiga sensor utama: getar (S_1), api/flame (S_2), dan suara (S_3), yang masing-masing dihubungkan ke input A, B, dan C dari gerbang NOR 8-input. Input sisanya (D sampai H) dihubungkan ke GND (logika 0) karena tidak digunakan, sehingga hanya ketiga sensor tersebut yang memengaruhi output. Tujuannya adalah menggabungkan sinyal dari ketiga sensor menjadi satu sinyal output.

Output diambil dari pin X (keluaran NOR). Gerbang NOR menghasilkan logika “1” jika semua input bernilai “0”. Artinya, jika ada ketiga sensor yang mendeteksi gangguan (getar, api, atau suara), input NOR menjadi “0”, dan output X langsung berubah menjadi “1”. Perubahan ini digunakan untuk menyalakan LED

indikator yang kemudian dihubungkan langsung ke pin 8 mikrokontroler untuk memberi sinyal digital yang jelas ke mikrokontroler untuk diproses lebih lanjut.



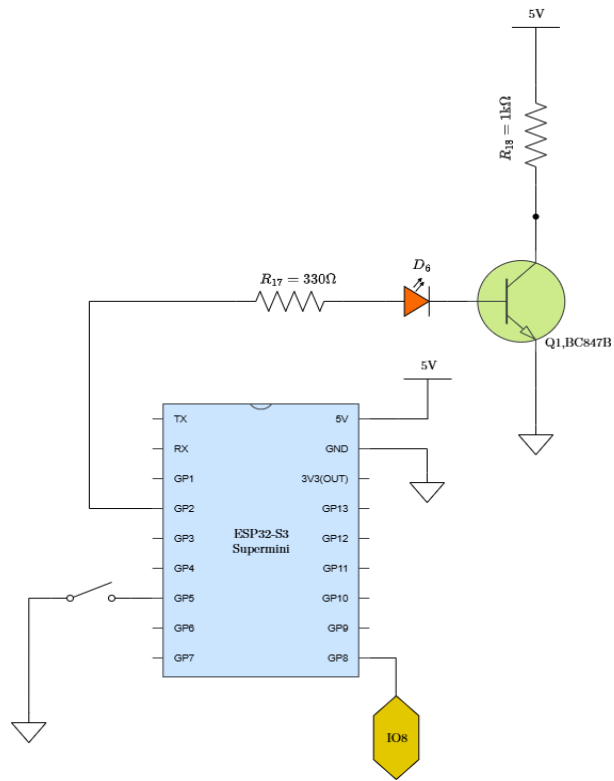
Gambar 28 Rangkaian pengkondisi sinyal

e. Rangkaian Driver SAT

Pada Gambar 30 merupakan rangkaian *driver* untuk sistem SAT, dimana rangkaian tersebut bertujuan untuk memaksimalkan keluaran daya namun tetap menjaga kompatibilitas dengan sistem *embedded* berdaya rendah seperti ESP32. Selain itu rangkaian pada Gambar 28 bertujuan untuk mempercepat *switching*, mempercepat waktu naik dan turun sehingga membantu pembentukan gelombang pulsa agar sempurna. Terdapat beberapa rangkaian penyusun pada *driver infrared*, diantaranya:

1. Rangkaian pengendali

Rangkaian yang ditampilkan pada Gambar 27 merupakan Rangkaian pengendali ini menggunakan mikrokontroler ESP32-S3 Supermini sebagai otak sistem. Ketika tombol IO5 ditekan, sinyal logika dari rangkaian sensor (melalui IC 74HC4078) akan mengaktifkan pin IO8 sebagai input ke ESP32-S3. Mikrokontroler kemudian merespons dengan menyalakan output di pin GPIO 2 (IO2), yang memicu transistor BC847B untuk menghidupkan LED indikator D6. Dengan demikian, LED D6 berfungsi sebagai indikator visual bahwa sistem telah terpicu dan siap mengaktifkan beban utama (IRLED) melalui rangkaian driver berikutnya.



Gambar 29 Rangkaian Pengendali

Menurut datasheet BC847B $V_{CE} = 0.2V$, maka $I_{C(sat)}$ dapat dihitung:

$$I_{C_{sat}} = \frac{V_{CC} - V_{CE}}{R_{18}} = \frac{5V - 0.2V}{1k\Omega} = \frac{4.8V}{1k\Omega} = 4.8mA \quad (2.7)$$

Setelah tahu $I_C = 4.8mA$, maka $I_{B(min)}$ dapat dihitung:

$$I_{B_{min}} = \frac{I_C}{hFE_{min}} = \frac{4.8mA}{100} = 0.048mA \quad (2.8)$$

Ketika GPIO2 aktif tegangan base (VB) transistor $\approx 3.3V$, $V_{LED} \approx 2V$, $V_{BE} \approx 0.7V$ maka tegangan yang jatuh di R_{17} dapat dihitung:

$$\begin{aligned} V_{R_{17}} &= V_{GPIO} - V_{LED} - V_{BE} \\ V_{R_{17}} &= 3.3V - 2V - 0.7V = 0.6V \end{aligned} \quad (2.9)$$

Arus yang mengalir di jalur basis atau I_B (Aktual) dapat dihitung dengan rumus

$$I_B = \frac{V_{R_{17}}}{R_{17}} = \frac{0.6V}{330\Omega} = 0.00182A = 1.82mA \quad (3.0)$$

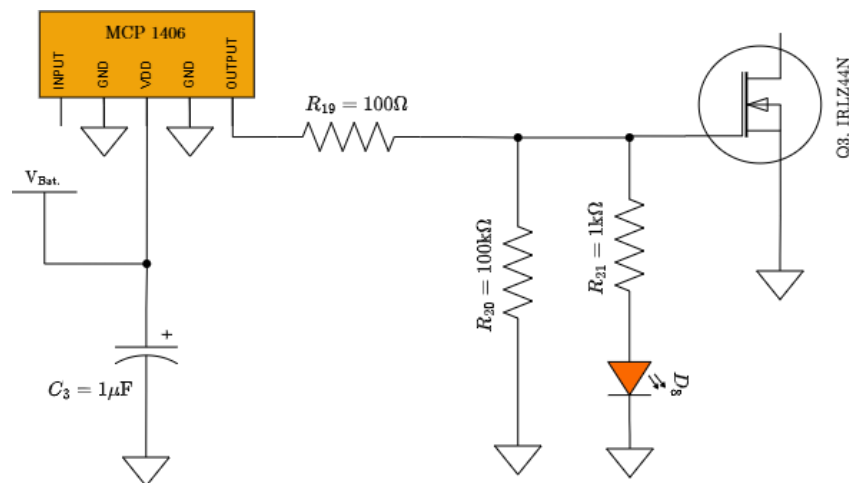
Maka, gain arus efektif dalam saturasi (sering disebut forced β atau β_{forced}) dapat dihitung sebagai:

$$\beta_{forced} = \frac{IC}{IB} = \frac{4.8mA}{1.82mA} = 2.64 \quad (3.1)$$

Nilai ini jauh di bawah hFE minimum transistor pada daerah aktif (100–200), mengonfirmasi bahwa transistor beroperasi dalam daerah saturasi, sesuai dengan prinsip desain rangkaian switching.

2. Rangkaian Driver MOSFET dengan IC MCP1406 untuk Pengendalian Beban

Untuk menghidupkan atau mematikan perangkat berdaya tinggi yaitu inframerah diperlukan sinyal digital bertegangan rendah dari rangkaian logika dan switch daya berbasis MOSFET. Pada sistem ini, sinyal digital yang dihasilkan dari keluaran kolektor transistor Q1 dikonversi menjadi sinyal penggerak yang mampu mengaktifkan MOSFET daya (IRFZ44N) secara efisien dan responsif. Konversi ini dilakukan dengan memanfaatkan IC MCP1406 sebagai gate driver MOSFET, yang berfungsi meningkatkan arus dan mempercepat transisi penyalan/pemadaman MOSFET. Dengan karakteristik output push-pull dan kemampuan mengalirkan arus hingga $\pm 2A$, MCP1406 memastikan switching MOSFET berlangsung cepat, stabil, dan minim disipasi daya.



Gambar 30 Rangkaian Driver MOSFET dengan IC MCP1406 untuk Pengendalian Beban

Input MCP1406 diambil dari keluaran kolektor transistor Q1 (rangkaiannya sebelumnya), yang diberi pull-up resistor $R_{18} = 1k\Omega$ ke 5V. Saat Q1 OFF keluaran kolektor $\approx 5V$, input MCP1406 = HIGH maka output MCP1406 = VDD ($\approx 8V$) sehingga MOSFET ON. Saat Q1 ON keluaran kolektor $\approx 0V$, input MCP1406 =

LOW maka output MCP1406 = 0V sehingga MOSFET OFF. Untuk arus input dari MCP1406 dapat dihitung:

$$I_{input} = \frac{VCC}{R_{18}} = \frac{5V}{1k\Omega} = 5mA \quad (3.2)$$

Output MCP1406 terhubung langsung ke gate MOSFET Q3 (IRFZ44N) melalui resistor R19 = 100Ω. Resistor ini berfungsi sebagai pembatas arus gate mencegah lonjakan arus saat switching, dan membantu mengurangi noise/ringing.

LED D8 terhubung antara VDD (8V) dan output MCP1406 melalui R21 = 1kΩ, Maka arus pada led dapat dihitung seperti persamaan 1.7:

$$I_{LED} = \frac{VDD - V_{LED}}{R_{21}} = \frac{8V - 2V}{1k\Omega} = 6mA$$

Arus pubcak gate mosfet dapat dihitung :

$$I_{gate_max} = \frac{VDD}{R_{19}} = \frac{8V}{1k\Omega} = 8mA \quad (3.3)$$

Tengagan pada gate mosfet, Saat MCP1406 output HIGH, $V_{GS} = 8V$ sehingga MOSFET menyala penuh. Saat MCP1406 output LOW, $V_{GS} = 0V$ maka MOSFET mati.

3. Mosfet

MOSFET (Metal-Oxide-Semiconductor Field-Effect Transistor) merupakan perangkat semikonduktor yang berfungsi sebagai sakelar atau penguat. Pada tipe-N (NMOS), arus drain (I_D) mengalir ketika tegangan gate-source (V_{GS}) melebihi tegangan ambang (V_T). Operasinya terbagi menjadi dua wilayah:

- Wilayah linear (ohmik): $V_{DS} \leq V_{GS} - V_T$,

$$I_D = k[2(V_{GS} - V_T)V_{DS} - V_{DS}^2] \quad (3.4)$$

- Wilayah saturasi: $V_{DS} \geq V_{GS} - V_T$,

$$I_D = k(V_{GS} - V_T)^2 \quad (3.5)$$

dengan k sebagai parameter konduksi (A/V²).

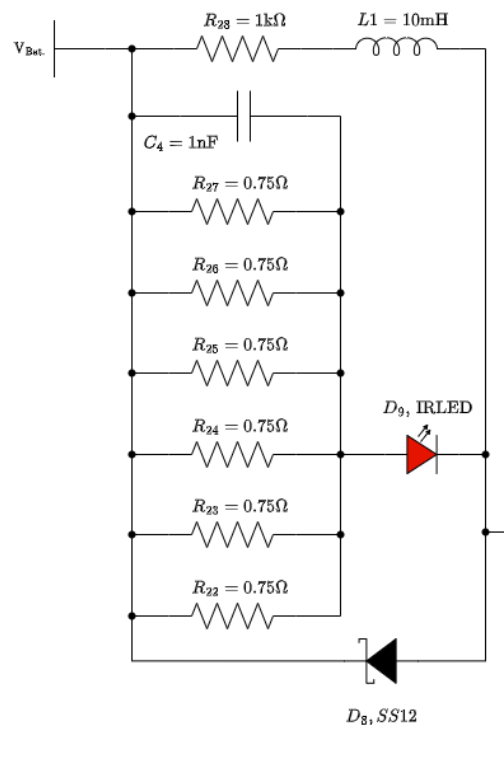
Dalam aplikasi enggerak inframerah pada sistem ini MOSFET dioperasikan sebagai sakelar yang hanya berada dalam dua keadaan: mati total ($V_{GS} < V_T$) atau nyala penuh ($V_{GS} \gg V_T$). Pada kondisi nyala, V_{DS} sangat kecil sehingga perangkat berada di ujung bawah wilayah linear, di mana ia berperilaku seperti resistor dengan nilai rendah $R_{DS(on)}$. Rugi daya saat konduksi diberikan oleh:

$$P = I_D^2 \cdot R_{DS(on)} \quad (3.6)$$

Untuk MOSFET daya seperti IRLZ44N, $V_T \approx 1-2$ V. Dengan tegangan gate penggerak $V_{GS} = 5-8$ V $\gg V_T$, MOSFET memasuki kondisi fully on dengan $R_{DS(on)}$ minimal (misal: 22 m Ω), sehingga efisiensi termal terjaga dan mampu mengalirkan arus tinggi tanpa pemanasan berlebih.

4. IR LED

Rangkaian ini menggerakkan LED inframerah berdaya tinggi (JH-7070IR18S40-T8A-LS940) dengan arus operasi 3.5 A, menggunakan MOSFET Q3 (IRLZ44N) sebagai sakelar daya, dan rangkaian LC serta resistor paralel untuk mengatur arus dan mengurangi noise/ripple.



Gambar 31 Rangkaian sederhana IR LED

Berdasarkan datasheet JH-7070IR18S40-T8A-LS940:

- Arus operasi (I_{op}) = 3.5 A (dengan kondisi $T=50^{\circ}\text{C}$)
- Tegangan operasi (V_{op}) = 2.0 V (typical)
- Resistansi seri (R_s) = 0.19 Ω
- Daya optik (P_o) = 2.9 W (typical)
- Efisiensi konversi (PCE) = 37%

Saat mosfet ON maka, $V_{LED} = 2.0 \text{ V}$ (typical), $I_{LED} = 3.5 \text{ A}$ (operating current).

Maka rugi dayanya:

$$P_{LED} = V_{LED} \times I_{LED} = 2.0\text{V} \times 3.5\text{A} = 7.0\text{W} \quad (3.7)$$

Menurut datasheet, daya optik hanya 2.9W artinya:

$$P_{thermal} = P_{LED} - P_{optical} = 7.0\text{W} - 2.9\text{W} = 4.1\text{W} \quad (3.8)$$

Tegangan di L1 berfungsi sebagai filter arus. Saat steady-state (DC), induktor berperilaku seperti kabel tidak ada drop tegangan. Jadi, tegangan di L1 $\approx 0\text{V}$.

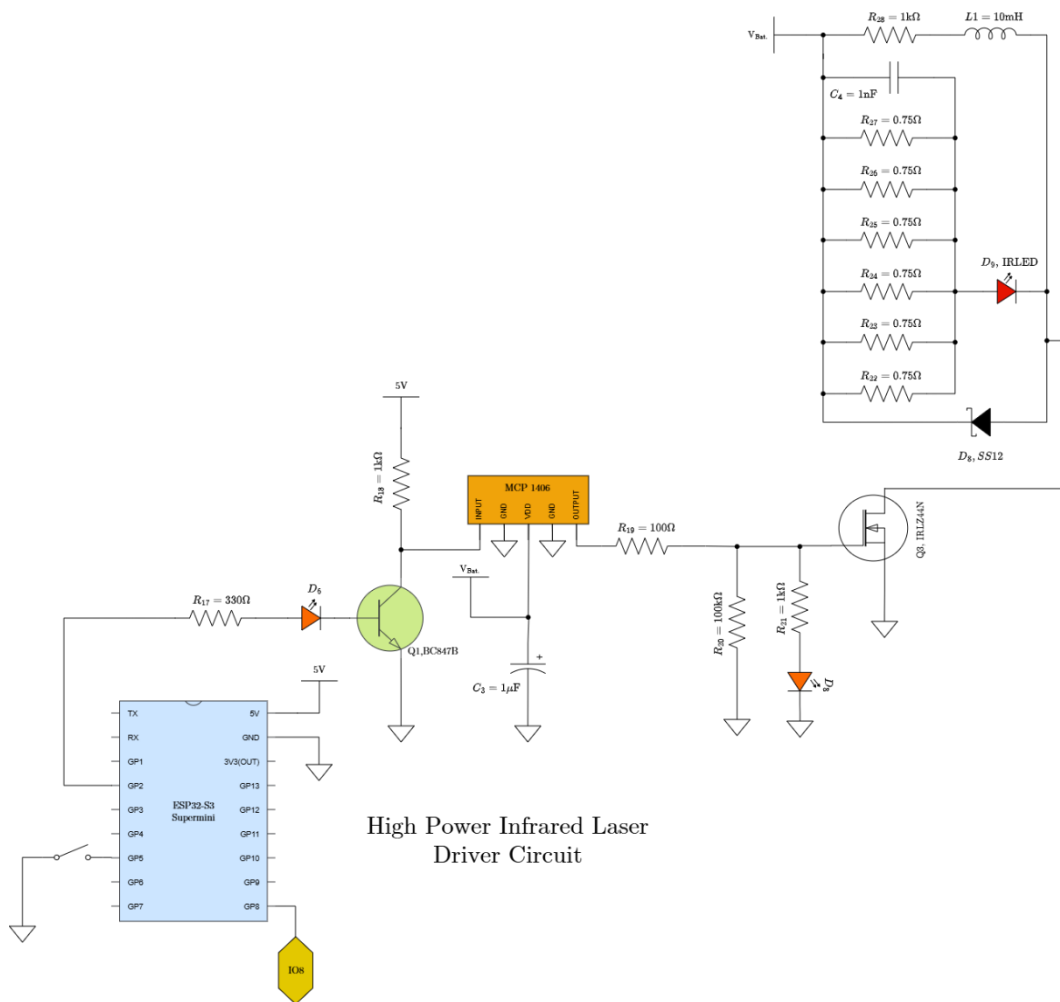
Tegangan sumber yang dibutuhkan

$$\begin{aligned} V_{total} &= VR28 + VL1 + VLED + VD8 \\ &= 0.35V + 0V + 2.0V + 0.3V = 2.65V \end{aligned} \quad (3.9)$$

MOSFET IRLZ44N memiliki $R_{DS(on)} \approx 22 \text{ m}\Omega$ (dengan $V_{GS} = 5 \text{ V}$). Maka rugi dayanya:

$$\begin{aligned} P_{MOSFET} &= I^2 \times R_{DS \text{ on}} \\ &= 3.5A^2 \times 0.022\Omega = 12.25 \times 0.022 = 0.2695W \end{aligned} \quad (4.0)$$

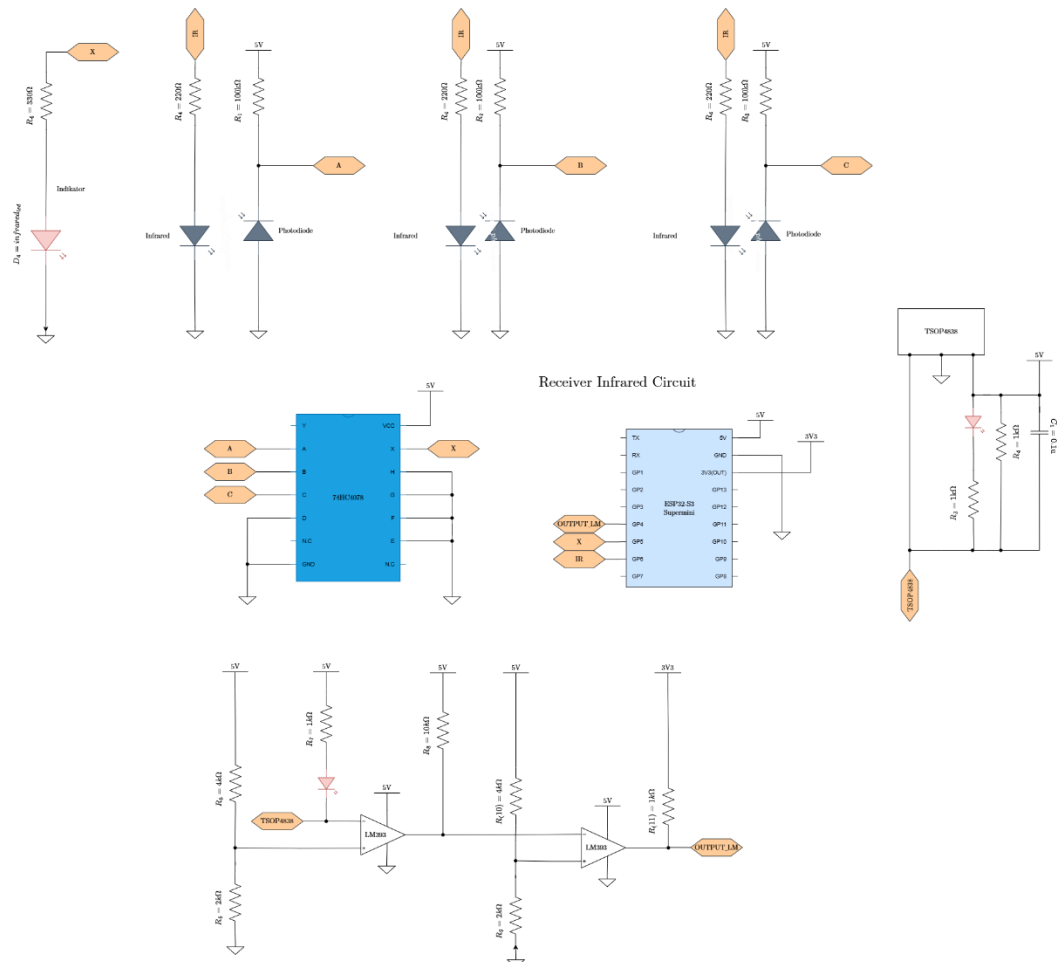
$C_4 = 1\text{nF}$ sebagai kapasitor bypass, mengurangi noise RF tidak mempengaruhi arus DC. D_8 (SS12) sebagai dioda freewheeling, melindungi MOSFET dari spike tegangan saat dimatikan arusnya hanya saat switching.



Gambar 32 Gambar Rangkaian keseluruhan Infrared Driver

3.4.3 Perancangan Rangkaian Helm dengan Sensor Penerima Inframerah dan Mekanisme Anti-Cheating

Rangkaian ini dirancang sebagai sistem deteksi *cheating* berbasis inframerah (IR), di mana keberadaan objek yang menutupi sinyal IR akan menghasilkan perubahan logika yang dapat diproses oleh mikrokontroler (ESP32-S3 Supermini) melalui rangkaian logika (74HC4078) dan komparator (LM393).



Gambar 33 Rangkaian Penerima Infrared Anti-Cheating

a. TSOP4838 dan LM393

Rangkaian penerima inframerah anti-cheating yang diimplementasikan menggunakan modul TSOP4838 dan komparator ganda LM393 dirancang untuk mendeteksi sinyal inframerah 38 kHz dengan akurasi tinggi dan respons logika yang terstruktur. Analisis berikut bertujuan untuk mengkarakterisasi perilaku

tegangan dan arus pada setiap tahap rangkaian, serta mengevaluasi integritas logika output dalam konteks aplikasi deteksi kecurangan.

Saat TSOP4838 aktif (output LOW) LED menyala. $V_{CC} = 5V$, $V_{LED} = 2V$ maka arus LED dihitung dengan hukum Ohm seperti persamaan 1.7:

$$I_{LED} = \frac{V_{CC} - V_{LED}}{R_3 + R_4} = \frac{5V - 2V}{1k\Omega + 1k\Omega} = 1.5mA$$

Komparator Pertama (U1 - LM393A). Input Non-Inverting (+): Terhubung ke pembagi tegangan dari $R_6 = 4k\Omega$ dan $R_5 = 2k\Omega$ menghasilkan referensi. Input Inverting (-): Terhubung ke output TSOP4838 melalui $R_7 = 1k\Omega$ dan LED (dengan $R_8 = 4k\Omega$ sebagai pull-up) Output U1: Terhubung ke input inverting (-) dari komparator kedua (U2) melalui $R_{10} = 10k\Omega$. Maka tegangan referensi dapat dihitung seperti persamaan 1.6:

$$V_{ref1} = V_{CC} \times \frac{R_6}{R_5 + R_6} = 5V \times \frac{2k\Omega}{4k\Omega + 2k\Omega} = 1.67V$$

Komparator Kedua (U2 - LM393A) Input Non-Inverting (+): Terhubung ke pembagi tegangan dari $R_{10} = 4k\Omega$ dan $R_9 = 2k\Omega \rightarrow$ referensi 3.3V. Input Inverting (-): Terhubung ke output U1 melalui $R_{10} = 10k\Omega$. Output U2: Terhubung ke OUTPUT LM melalui $R_{11} = 1k\Omega$ (pull-up ke sumber 3.3V). Maka tegangan referensi dapat dihitung seperti persamaan 1.6:

$$V_{ref2} = V_{CC} \times \frac{R_6}{R_5 + R_6} = 5V \times \frac{2k\Omega}{4k\Omega + 2k\Omega} = 1.67V$$

b. Rangkaian Anti-Cheating

Cara kerjanya konfigurasi photodiode: mode reverse bias. Photodiode disambungkan antara ground dan output, dengan resistor pull-up ke 5V. Saat terkena cahaya IR arus photocurrent mengalir tegangan di node output turun sehingga logika LOW. Saat tidak terkena cahaya (IR ditutup) arus bocor sangat kecil tegangan di node output $\approx 5V$ sehingga logika HIGH.

Kondisi 1: IR MENYALA (Photodiode TERKENA CAHAYA) Arus photocurrent I_{ph} atau $I_{(ra)}$ = $60\mu A$ (berdasarkan datasheet) dan resistor pull-up $R_1 = 100k\Omega$. Maka dapat dihitung sebagai berikut:

$$\begin{aligned} V_{ABC} &= VCC - I_{(photodiode)} \times R1 \\ &= 5V - (60\mu A \times 10^{-6} A) \times (100 \times 10^3 \Omega) = 5V - 6V = -1V \end{aligned} \quad (4.1)$$

Namun, untuk $V_R = 5V$, nilai $I_{ra} = 60\mu A$ adalah nilai maksimum artinya, jika sumber cahaya cukup kuat ($E_c = 1 \text{ mW/cm}^2$), photodiode akan menghasilkan arus $\sim 60\mu A$. Jadi, tegangan output tidak akan turun di bawah $0V$ maka akan jatuh ke $\sim 0V$ (saturasi). $V_{ABC} = 0V$ dengan logika LOW

Kondisi 2: IR DITUTUP (Photodiode TIDAK TERKENA CAHAYA). Arus dark current: $I_{dark} = 1 \text{ nA}$ (typical) dan resistor pull-up $R_1 = 100k\Omega$. Maka dapat dihitung sebagai berikut.

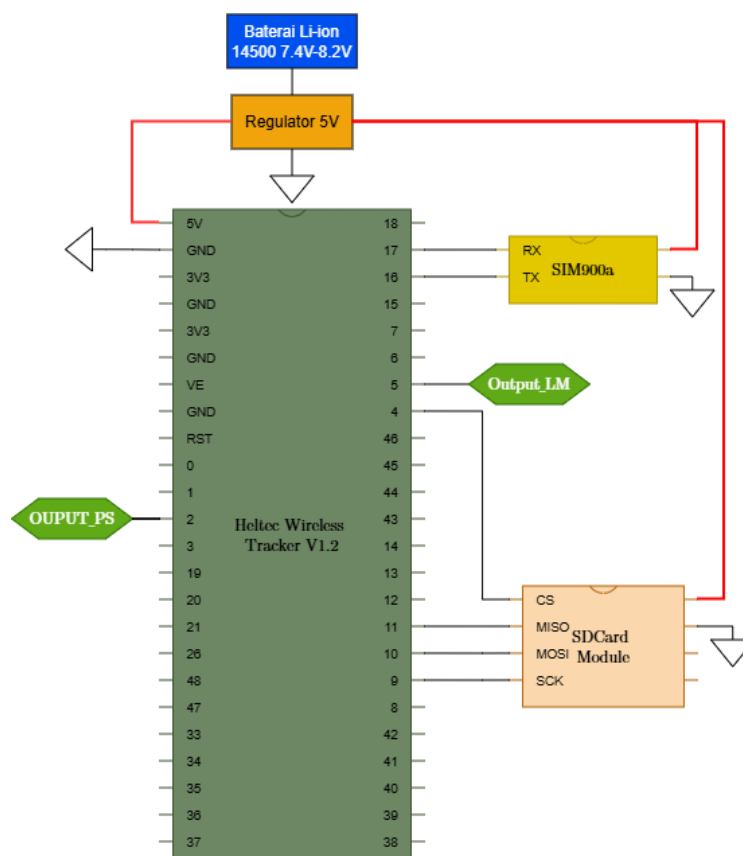
$$\begin{aligned} V_{ABC} &= VCC - I_{dark} \times R1 \\ &= 5V - (1 \times 10^{-9} A) \times 1 \times 10^5 \Omega = 4.999V \end{aligned} \quad (4.2)$$

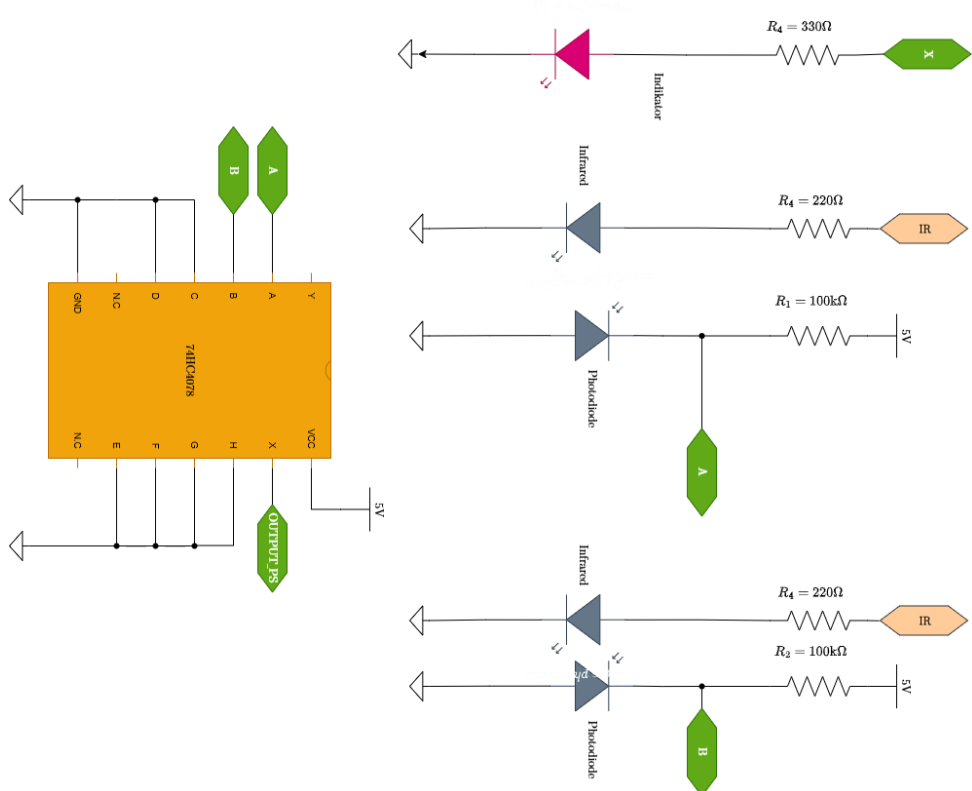
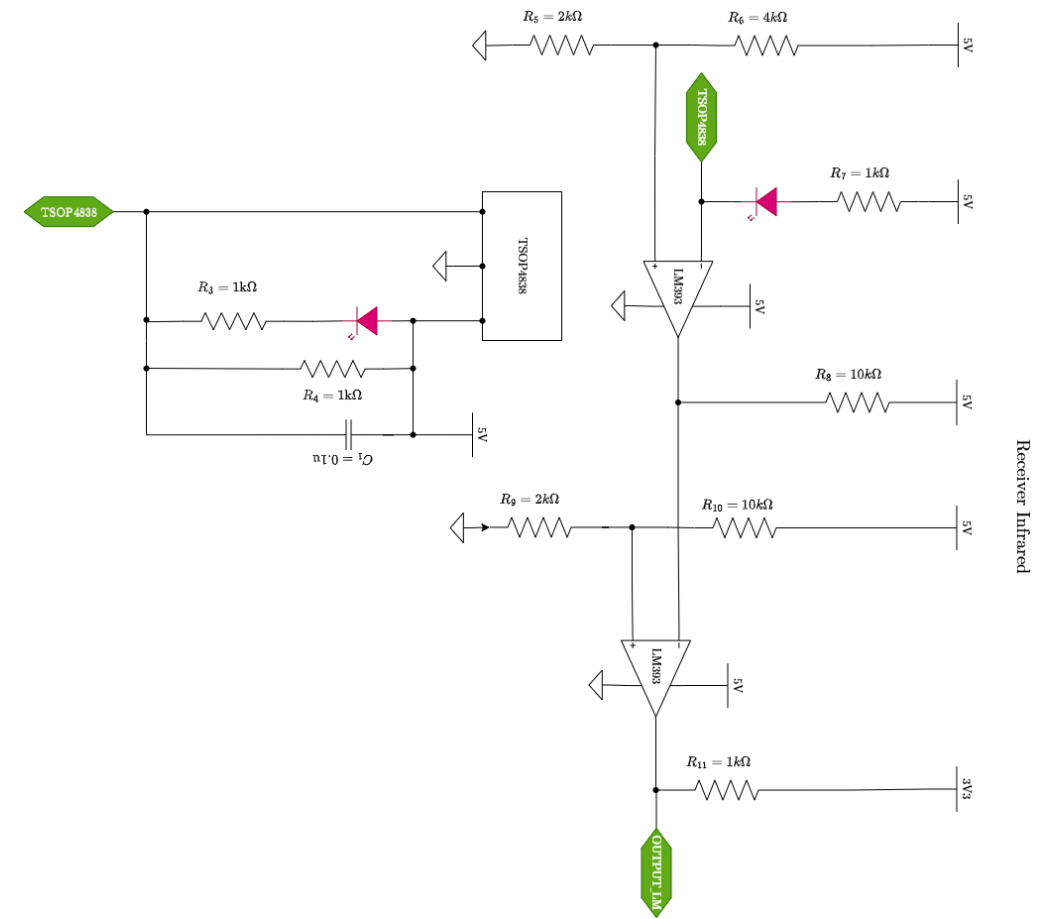
Maka $V_{ABC} = 4.999V$ dengan logika HIGH

Sistem anti-cheating menggunakan gerbang logika NOR 8 masukan dengan persamaan $Y = A + B + C + D + E + F + G + H$. Pada implementasinya di helm, hanya empat masukan yang digunakan untuk photodiode (A, B, C, dan D), sedangkan empat masukan lainnya dihubungkan ke ground (logika LOW). Dengan demikian, persamaan logika efektif menjadi $Y = A + B + C + D$. Output bernilai HIGH hanya ketika seluruh photodiode menerima cahaya ($A=B=C=D=LOW$). Apabila salah satu photodiode tidak menerima cahaya sehingga menghasilkan logika HIGH, maka output NOR berubah menjadi LOW yang menandakan terdeteksinya tindakan cheating.

3.4.4 Perancangan Rangkaian Vest (Penerima Infrared, Anti-Cheating, dan Transmisi Data)

Rangkaian vest (rompi) pada sistem permainan laser tag ini dirancang sebagai modul penerima inframerah sekaligus pusat kendali pemrosesan dan pengiriman data. Sistem ini mendeteksi status tembakan (HIT) dan potensi kecurangan (CHEAT), serta mengakuisisi data GPS berupa longitude, latitude, altitude, dan jumlah satelit. Seluruh data diproses oleh mikrokontroler Heltec Wireless Tracker V1.2, kemudian dikirimkan ke server melalui komunikasi LoRa maupun GPRS menggunakan modul SIM900A, atau disimpan pada modul SD Card sebagai cadangan data. Sumber daya yang digunakan sama seperti ala lainnya . Pada rangkaian *vest* ini menggunakan sumber daya yang sama seperti helm dan juga senjata.





1. Rangkaian SIM900A

Modul SIM900A pada sistem ini berfungsi sebagai alternatif komunikasi data jarak jauh ketika jaringan LoRaWAN tidak tersedia atau sebagai redundansi untuk meningkatkan keandalan transmisi data. Modul ini mengirimkan data status permainan (skor, lokasi GPS, waktu tembakan) ke server cloud melalui jaringan GPRS menggunakan protokol HTTP POST ke platform ThingSpeak, kemudian data diteruskan melalui Webhook ke endpoint Vercel untuk diproses dan ditampilkan pada antarmuka web secara *real-time*. Modul SIM900A terintegrasi dengan Heltec Wireless Tracker V1.2 yang berbasis mikrokontroler ESP32-S3 melalui antarmuka UART (Universal Asynchronous Receiver-Transmitter).

Perhitungan Daya Input Modul SIM900A, dengan $V_{CC} = 4,0V$ dan $I_{CC} = 300mA$ (operasi normal):

$$\begin{aligned} P_{in} &= V_{CC} \times I_{CC} \\ P_{in} &= 4,0V \times 0,3A = 1,2W \\ P_{peak} &= 4,0V \times 2A = 8,0W \end{aligned} \tag{4.3}$$

Perhitungan Kapasitor Buffer untuk Burst Current, dengan $I_{burst} = 2A$, $t_{burst} = 577\mu s$ (1 TDMA slot GSM), dan $\Delta V = 0,3V$ (toleransi tegangan):

$$\begin{aligned} P_{in} &= V_{CC} \times I_{CC} \\ P_{in} &= 3,3V \times 0,1A = 0,33W \end{aligned} \tag{4.4}$$

2. Rangkaian SD Card Module

Modul SD Card pada sistem ini berfungsi sebagai media penyimpanan data lokal (*local storage*) untuk mencatat riwayat tembakan, posisi GPS, dan status permainan ketika komunikasi LoRaWAN atau GPRS tidak tersedia. Modul ini terintegrasi dengan Heltec Wireless Tracker V1.2 yang berbasis mikrokontroler ESP32-S3 melalui antarmuka SPI (Serial Peripheral Interface). Modul SD Card menggunakan regulator tegangan onboard AMS1117-3.3 untuk menurunkan tegangan input 5 V menjadi 3,3 V yang dibutuhkan oleh kartu microSD, sehingga kompatibel dengan level logika ESP32-S3

Perhitungan Daya Input Modul SD Card, dengan $V_{CC} = 3,3V$ dan $I_{CC} = 100mA$

$$P_{in} = V_{CC} \times I_{CC}$$

$$P_{in} = 3,3V \times 0,1A = 0,33W$$
4.5

Perhitungan Kapasitas Penyimpanan Data: Kartu microSD dengan kapasitas 16 GB dapat menyimpan data dalam format teks dengan estimasi sebagai berikut:

$$Data_{per_event} = 100 \text{ byte/event}$$

$$Total_{events} = \frac{16 \times 10^9 \text{ byte}}{100 \text{ byte/event}} = 160.000.000 \text{ event}$$
4.6

3. Rangkaian MP3 Player

Pada sistem ini, modul DF Player Mini diimplementasikan sebagai mekanisme *feedback* audio untuk menginformasikan status pemain, yaitu saat terkena tembakan (*hit*) atau terdeteksi melakukan kecurangan (*cheating*). Modul ini beroperasi pada tegangan 5 V DC dengan konsumsi arus rata-rata 150 mA saat aktif memproses audio, dan dapat mencapai puncak 300 mA pada volume maksimal. Suplai daya diperoleh dari regulator LM7805 pada rompi yang menstabilkan tegangan baterai 7,4 V (2S Li-Ion) menjadi 5 V, memastikan catu daya yang bersih untuk mencegah *noise* pada output audio.

Komunikasi data antara Heltec Wireless Tracker V1.2 (ESP32-S3) dan DF Player Mini menggunakan antarmuka UART (Universal Asynchronous Receiver-Transmitter) dengan konfigurasi *baud rate* 9600 bps. Pin TX pada Heltec (GPIO 43) dihubungkan ke pin RX pada modul DF Player, sedangkan pin RX pada Heltec (GPIO 44) dihubungkan ke pin TX modul melalui pembagi tegangan (1 kΩ dan 2 kΩ) untuk menurunkan level logika 5 V DF Player menjadi 3,3 V agar kompatibel dengan ESP32-S3, sehingga mencegah kerusakan pada jalur penerima mikrokontroler. Output audio dihubungkan ke speaker 8 Ω, 5 Watt melalui penguat kelas-D PAM8403 yang memiliki efisiensi daya hingga 90%, meminimalkan disipasi panas selama operasi lapangan. File audio berformat MP3 disimpan pada kartu microSD dengan kapasitas 16 GB, dimana setiap file diberi penomoran indeks (0001–0099) yang dipanggil secara langsung oleh mikrokontroler berdasarkan event yang terdeteksi.

Perhitungan daya input DF Player Mini, dengan $V_{CC} = 5,0V$ dan $I_{CC} = 300mA$ (kondisi maksimum):

$$\begin{aligned} P_{in} &= V_{CC} \times I_{CC} \\ P_{in} &= 5V \times 300mA \end{aligned} \quad 4.7$$

Perhitungan daya output amplifier PAM8403, dengan $V_{rms} \approx 3,5V$ (estimasi dari 5V supply) dan $R_{load} = 8\Omega$:

$$\begin{aligned} P_{out} &= \frac{V_{rms}^2}{R_{load}} \\ P_{out} &= \frac{3,5V^2}{8\Omega} = \frac{12,25}{8} = 1,53W \end{aligned} \quad 4.8$$

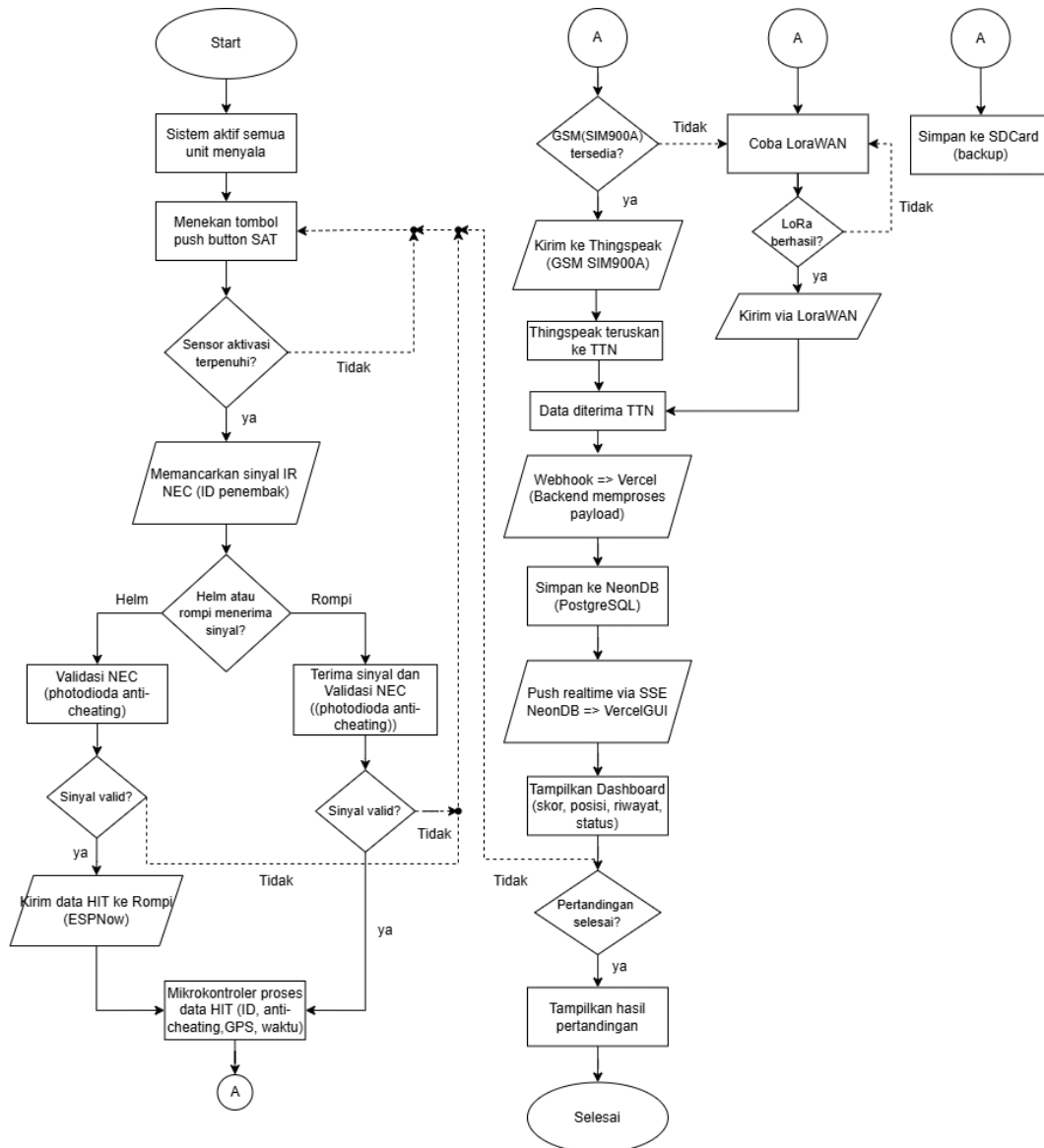
Perhitungan efisiensi amplifier Kelas-D:

$$\begin{aligned} \eta &= \frac{P_{out}}{P_{in}} \times 100 \\ \eta &= \frac{1,53W}{1,5W + 0,3W} \times 100\% \approx 85\% \end{aligned} \quad 4.9$$

Berdasarkan hasil perhitungan kebutuhan daya pada sistem rompi (vest), penggunaan baterai Li-Ion 7,4 V tipe 14500 (konfigurasi 2S) mampu menyuplai rangkaian yang bekerja pada tegangan 5 V DC melalui regulator LM7805. Sistem memiliki arus standby sebesar 215,08 mA, arus aktif 1352,08 mA, serta arus puncak (burst) mencapai 3052,08 mA, yang umumnya terjadi saat modul komunikasi seluler melakukan proses transmisi data. Daya yang dikonsumsi sistem sebesar 0,83 W pada kondisi standby dan 5,92 W pada kondisi aktif. Dengan kapasitas baterai yang digunakan, sistem diperkirakan mampu beroperasi selama sekitar 15 jam pada kondisi penggunaan normal dan sekitar 9 jam pada kondisi penggunaan intensif.

3.5 Perancangan Software

Berikut merupakan flowchart sistem dari alat yang akan dirancang dapat dilihat pada Gambar 34 Flowchart.



Gambar 34 Flowchart

Deskripsi Flowchart:

Sistem laser tag IoT diawali ketika seluruh unit perangkat keras yaitu SAT, Helm, dan Rompi dinyalakan dan siap beroperasi. Ketika pemain menekan push button utama pada unit SAT, mikrokontroler ESP32-S3 SuperMini akan memeriksa apakah ketiga sensor aktivasi yaitu sensor suara, sensor getar, dan sensor api

terpenuhi secara bersamaan sebagai syarat wajib sebelum tembakan dilepaskan. Apabila seluruh sensor aktif, SAT memancarkan sinyal infrared berprotokol NEC yang membawa data ID penembak melalui LED yang difokuskan lensa Fresnel ke arah lawan. Sinyal tersebut dapat diterima oleh Helm maupun Rompi. Jika Helm yang terkena, ESP32-S3 SuperMini pada Helm melakukan validasi protokol NEC sekaligus pemeriksaan anti-cheating melalui photodiode, dan apabila valid, data HIT diteruskan ke Rompi menggunakan komunikasi nirkabel ESP-NOW. Jika Rompi yang menerima sinyal secara langsung, data langsung diolah oleh mikrokontroler Heltec Wireless Tracker V1.2 yang menggabungkan informasi ID penembak, lokasi sensor, waktu tembakan, dan koordinat GPS dari modul GPS internal.

Setelah Heltec selesai memproses data, sistem secara paralel langsung menyimpan seluruh data ke kartu SD sebagai backup lokal permanen, kemudian mencoba mengirimkan data melalui jalur komunikasi berdasarkan prioritas. Jalur utama adalah GSM menggunakan modul SIM900A yang mengirimkan data ke platform ThingSpeak, kemudian ThingSpeak meneruskannya ke The Things Network (TTN). Apabila jaringan GSM tidak tersedia, sistem beralih ke jalur cadangan menggunakan LoRaWAN yang mengirimkan data langsung ke TTN. Setelah data berhasil diterima TTN dari jalur manapun, TTN meneruskan payload melalui webhook ke platform Vercel yang berperan sebagai backend sekaligus frontend hosting. Vercel memproses data tersebut dan menyimpannya ke database PostgreSQL melalui layanan NeonDB, lalu mendorong pembaruan secara real-time ke antarmuka dashboard menggunakan teknologi Server-Sent Events (SSE) sehingga status pemain, skor, riwayat tembakan, dan posisi pemain dapat dipantau langsung tanpa perlu refresh halaman hingga pertandingan dinyatakan selesai.

BAB IV PENGUJIAN DAN ANALISA

Pada bab ini disajikan hasil pengujian sistem yang telah dilakukan baik secara per blok komponen maupun secara keseluruhan, beserta analisis terhadap data yang diperoleh dari hasil pengujian. Proses pengujian dimulai dari masing-masing blok rangkaian elektronik dan modul komunikasi, kemudian dilanjutkan dengan pengujian integrasi sistem secara menyeluruh. Hasil pengujian tersebut selanjutnya akan dianalisis untuk mengetahui apakah sistem telah bekerja sesuai dengan rancangan yang direncanakan, termasuk kemampuan deteksi serangan, akurasi pengiriman data, serta keandalan komunikasi dalam berbagai kondisi medan latihan yang beragam.

4.1 Pengujian perblok

Pengujian sistem perblok bertujuan untuk memverifikasi fungsi masing-masing komponen dalam Sistem Senjata sebelum melakukan pengujian keseluruhan. Dengan menguji setiap blok secara terpisah, kita dapat memastikan bahwa tidak ada masalah pada komponen individu yang dapat menyebabkan gangguan pada sistem keseluruhan.

4.1.1 Pengujian Rangkaian Power Regulator

1. LM7805

Pengujian dilakukan dengan memberikan *input* tegangan yang bervariasi (dari 7.0V hingga 8.2V). Setiap level tegangan masukan diukur menggunakan multimeter digital, dan tegangan keluaran regulator dicatat. Pengujian dilakukan masing masing sebanyak lima kali percobaan dengan kondisi tanpa beban. *Error* (%) dihitung menggunakan rumus sebagai berikut:

$$\text{Error \%} = \left(\frac{\text{Output} - \text{TargetOutput}}{\text{TargetOutput}} \right) \times 100 \quad (5.0)$$

Hasil pengujian ditunjukkan pada Tabel .7 dengan hasil pengujian sebagai berikut

Tabel 4 Pengujian LM7805

REGULATOR LM7805				
	<i>INPUT</i>	<i>OUTPUT</i>	TARGET <i>OUTPUT</i>	ERROR
1	7.98V	5.08V	5V	0.16%
2	8.02V	5.08V	5V	0.16%
3	8.10V	5.08V	5V	0.16%
4	8.11V	5.08V	5V	0.16%
5	8.12V	5.08V	5V	0.16%
6	8.14V	5.08V	5V	0.16%

4.1.2 Pengujian Rangkaian Pemancar Inframerah

1. Sensor suara

Pengujian dilakukan dengan memberikan suara berintensitas 30–90 dB (diukur menggunakan sound level meter), lalu mencatat tegangan keluaran sensor dan kondisi outputnya. Tegangan referensi dipantau untuk stabilitas sistem, dengan threshold ditetapkan pada setengah VCC sebagai acuan pembanding logika tinggi dan rendah. Hasil pengujian ditampilkan pada Tabel 5.

Tabel 5 Pengujian Sensor Suara

SENSOR SUARA				
No	Desible (dB)	Tegangan Refrensi	<i>Threshold</i>	Kondisi Ouput
1.	35	2.83V	2.5V	1
2.	59	2.77V	2.5V	1
3.	82	1.82V	2.5V	0
4.	86	1.67V	2.5V	0

Berdasarkan Tabel 5, sistem sensor suara beroperasi dengan logika invers terhadap threshold: ketika tegangan referensi melebihi nilai threshold (2,5 V), kondisi output bernilai logika 1 yang menandakan keadaan “tidak aktif”; sebaliknya, ketika tegangan referensi berada di bawah threshold, output menghasilkan logika 0 yang menyatakan keadaan “aktif”.

2. Sensor getar

Pengujian dilakukan dengan memberikan *input* getaran berupa guncangan yang bervariasi (diam, diayunkan, ketukan, dijatuhkan). Respon sensor dicatat dalam bentuk tegangan keluaran serta kondisi *output*. Tegangan referensi juga diamati untuk memastikan stabilitas sistem selama pengujian. *Threshold* diberi masukan setengah dari nilai tegangan VCC sebagai titik referensi pembandingan antara logika tinggi dan logika rendah. Tabel 6. berikut menunjukkan hasil pengujian rangkaian sensor suara.

Tabel 6 Pengujian Sensor Getar

SENSOR GETAR				
NO	Input	Tegangan Refrensi	<i>Threshold</i>	Kondisi Output
1	Diam	2.88V	2.5V	1
2	Diayunkan	2.71V	2.5V	1
3	Diketuk	1.91V	2.5V	0
4	Dijatuhkan	120mV	2.5V	0

Berdasarkan Tabel 6, sensor getar dihubungkan ke suatu komparator dengan tegangan referensi (threshold) tetap sebesar 2,5 V. Output komparator dinyatakan sebagai logika 1 (misalnya HIGH) jika tegangan input lebih kecil dari threshold, dan logika 0 (LOW) jika tegangan input lebih besar dari threshold.

3. Sensor Api

Pengujian dilakukan dengan memberikan cahaya korek api pada jarak 0–30 cm (diukur menggunakan alat ukur jarak), lalu mencatat tegangan keluaran photodiode dan kondisi outputnya. Tegangan referensi dipantau untuk memastikan stabilitas sistem, dengan threshold ditetapkan pada setengah VCC sebagai acuan pembandingan antara logika tinggi dan rendah. Tabel 7 berikut menunjukkan hasil pengujian rangkaian sensor suara.

Tabel 7 Pengujian Sensor Api

SENSOR API				
No	Jarak (cm)	Tegangan Referensi	<i>Threshold</i>	Kondisi Ouput
1	265	2.88V	2.5V	1

2	246	530mV	2.5V	0
3	227	200mV	2.5V	0
4	208	163mV	2.5V	0
5	189	161mV	2.5V	0
6	170	161mV	2.5V	0
7	151	160mV	2.5V	0
8	132	160mV	2.5V	0
9	113	160mV	2.5V	0
10	94	160mV	2.5V	0
11	75	160mV	2.5V	0
12	56	160mV	2.5V	0
13	37	160mV	2.5V	0
14	18	160mV	2.5V	0
15	0	160mV	2.5V	0

Tabel menunjukkan respons sensor api terhadap variasi jarak dari sumber api dengan nilai threshold komparator sebesar 2,5 V. Pada jarak 265 cm, tegangan keluaran sensor sebesar 2,88 V berada di atas nilai threshold sehingga output bernilai 0 yang menandakan api tidak terdeteksi. Sementara itu, pada jarak 246 cm hingga 0 cm, tegangan keluaran sensor berada pada rentang 160 mV–530 mV, lebih kecil dari nilai threshold, sehingga output bernilai 1 yang menunjukkan api terdeteksi.

4. IC 74HC4078

Pengujian dilakukan dengan memberikan kombinasi tegangan *input* yang bervariasi pada pin *input* A, B, dan C. Pin *input* D, E, F, G dan H selalu diberi logika 0, sehingga hanya ada 3 *input* yang memiliki 8 kombinasi *input*. Setiap kombinasi *input* dicatat, dan tegangan *output* pada pin X dan Y diamati menggunakan multimeter digital. Hasil pengukuran kemudian dianalisis untuk memastikan akurasi IC dalam menghasilkan *output* yang sesuai dengan tabel kebenaran NOR. Tabel 8 menunjukkan hasil pengujian IC 74HC4078.

Tabel 8 Pengujian IC64HC4078

IC 74HC4078					
NO	TEGANGAN <i>INPUT</i> (LOGIKA)			TEGANGAN <i>OUTPUT</i> (LOGIKA)	
	A	B	C	X	Y
1	2.80V(1)	2.80V(1)	2.80V(1)	0V(0)	2.93V(1)
2	2.77V(1)	2.79V(1)	110mV(0)	90mV(0)	2.82V(1)
3	2.89V(1)	170mV(0)	2.88V(1)	0mV(0)	2.85V(1)
4	2.81V(1)	1.08V(0)	110mV(0)	0mV(0)	2.78V(1)
5	1.51V(0)	2.80V(1)	2.80V(1)	90mV(0)	2.93V(1)
6	1.75V(0)	2.80V(1)	160mV(0)	90mV(0)	2.93V(1)
7	1.79V(0)	1.91V(0)	2.80V(1)	90mV(0)	2.93V(1)
8	1.70V(0)	1.29V(0)	120mV(0)	2.60V(1)	100mV(0)

Setelah didapatkan data hasil pengujian IC 74HC4078, selanjutnya data hasil pengujian tersebut dibandingkan dengan tabel kebenaran gerbang NOR 3 *input*, sebagai berikut:

Tabel 9 Tabel Kebanaran Logika NOR

<i>INPUT</i>			OUT
A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	0
1	0	1	0
1	1	0	0
1	1	1	0

Berdasarkan hasil pengujian terhadap tiga input (A, B, dan C) yang berasal dari ketiga sensor berbasis komparator LM393. Output X menghasilkan logika tinggi (3,37 V) hanya ketika seluruh input berada pada kondisi logika rendah (di bawah 0,8 V; dalam pengujian berkisar 110–160 mV), sedangkan pada semua kombinasi lain, yaitu ketika minimal satu input berada pada logika tinggi (di atas 2,0 V; dalam

pengujian sekitar 3,40 V), output X berada pada logika rendah (mendekati 0 V). Hal ini mengonfirmasi bahwa X merepresentasikan fungsi NOR(A,B,C), sementara output Y yang bersifat komplementer secara implisit berfungsi sebagai OR(A,B,C).

5. Pengujian protocol komunikasi ESP-NOW

Pengujian protokol ESP-NOW dilakukan untuk mengevaluasi komunikasi nirkabel satu arah antara unit Helm dan Rompi berbasis ESP32, tanpa bergantung pada infrastruktur Wi-Fi atau perangkat perantara. Pada pengujian ini, ESP32-S3 Supermini yang berperan sebagai Helm berfungsi sebagai pengirim (sender), sedangkan Heltec Tracker V1.2 sebagai Rompi berfungsi sebagai penerima (receiver). Kedua perangkat ditempatkan pada jarak sekitar 4 meter, masing-masing dimasukkan ke dalam kardus dan dipisahkan oleh dinding sebagai penghalang. Helm mengirimkan data secara broadcast berupa pesan teks real-time “Hello, World! #n” (dengan n sebagai nomor urut pesan). Tujuan pengujian adalah memverifikasi keandalan pengiriman data dalam kondisi lingkungan yang menantang. Hasil pengujian ini disajikan pada Tabel 10.

Tabel 10 Pengujian ESP-NOW

No	Halo Status	Data Send	Harness Status	Data Receive	RSSI
1	Sent	Hello, World! #4	Receive	Hello, World! #4	-95Bm
2	Sent	Hello, World! #5	Receive	Hello, World! #5	-94dBm
3	Sent	Hello, World! #6	Receive	Hello, World! #6	-91dBm
4	Sent	Hello, World! #7	Receive	Hello, World! #7	-91dBm
5	Sent	Hello, World! #8	Receive	Hello, World! #8	-91dBm
6	Sent	Hello, World! #9	Receive	Hello, World! #9	-92dBm
7	Sent	Hello, World! #10	Receive	Hello, World! #10	-92dBm
8	Sent	Hello, World! #11	Receive	Hello, World! #11	-92dBm
9	Sent	Hello, World! #12	Receive	Hello, World! #12	-93dBm

10	Sent	Hello, World! #13	Receive	Hello, World! #13	-93dBm
----	------	----------------------	---------	----------------------	--------

Berdasarkan hasil pengujian komunikasi dua arah menggunakan protokol ESP-NOW menunjukkan bahwa semua pesan "Hello, World! #n" berhasil diterima oleh rompi tanpa error, meskipun perangkat ditempatkan di dalam kardus dan terhalang dinding. Nilai RSSI berkisar antara -91 dBm hingga -95 dBm, menandakan sinyal lemah namun masih cukup andal untuk komunikasi jarak dekat.

4.1.3 Pengujian Rangkaian Penerima Inframerah

Tujuan dari pengujian rangkaian penerima inframerah adalah untuk memastikan data yang diterima oleh rangkaian penerima inframerah sama dengan data yang dikirim oleh rangkaian pemancar inframerah.

1. Parameter Jarak

Tabel 11 Pengujian Jarak Penerimaan

No	Jarak	Indikator LED	Output Sensor	Status Data
1	50 meter	Aktif		Diterima
2	150 meter	Aktif		Diterima
	170 meter	Aktif		Diterima
3	200 meter	Aktif		Diterima
4	250 meter	Aktif		Tak Diterima
5	300 meter	Aktif		Tak Diterima

Berdasarkan data pada tabel dapat dianalisa bahwa sensor TSOP4838 dapat menerima sinyal inframerah dari jarak 50 meter sampai 500 meter. Hal ini juga di pengaruhi oleh kemampuan pemancar yang sangat kuat yang membuat pemancar bisa memancar dalam jarak yang cukup jauh. Pengujian dilakukan pada malam hari agar tidak ada gangguan dari cahaya lain seperti cahaya matahari bertujuan untuk memaksimalkan kinerja pemancar maupun penerima sinyal atau cahaya inframerah

2. Parameter Sudut

Tabel 12 Pengujian Sudut Penerimaan

No	Parameter Sudut	Tanpa Lensa	Menggunakan Lensa
1	0°	Respon	Respon
2	15°	Respon	Respon

3	30°	Respon	Respon
4	45°	Respon	Respon
5	60°	Tidak Respon	Respon
6	75°	Tidak Respon	Respon
7	90°	Tidak Respon	Respon
8	105°	Tidak Respon	Respon
9	135°	Tidak Respon	Respon
10	180°	Tidak Respon	Respon

Berdasarkan hasil pengujian sudut inframerah, sistem hanya mampu mendeteksi sinyal secara konsisten hingga sudut 45° tanpa lensa dan hingga 180° saat menggunakan lensa; namun, pada sudut 180°, meskipun respons tercatat, jangkauan deteksi maksimalnya terbatas hanya sekitar 50-60 meter, menunjukkan bahwa meskipun lensa memperluas cakupan sudut deteksi, efektivitas jaraknya tetap terbatas dan tidak optimal dibandingkan pada sudut lebih kecil.

3. Receiver Anti-Cheating

Tabel 13 Pengujian Anti-Cheating

No	Kondisi Anti-Cheating				Output NOR
	A	B	C	D	X
1	0	0	0	0	1
2	0	0	0	1	0
3	0	0	1	0	0
4	0	0	1	1	0
5	0	1	0	0	0
6	0	1	0	1	0
7	0	1	1	0	0
8	0	1	1	1	0
9	1	0	0	0	0
10	1	0	0	1	0
11	1	0	1	0	0
12	1	0	1	1	0
13	1	1	0	0	0
14	1	1	0	1	0
15	1	1	1	0	0

16	1	1	1	1	0
----	---	---	---	---	---

Berdasarkan **Tabel 13** tersebut ketika salah satu sensor itu ditutup makan sistem akan mendeteksi cheat sebaliknya Ketika tidak ada yang ditutupi sisitem tidak akan mendeteksi cheat

4.1.4 Pengujian Heltec Tracker V1.2

1. Pengujian protocol komunikasi ESP-NOW

Pengujian protokol ESP-NOW dilakukan untuk mengevaluasi komunikasi nirkabel satu arah antara unit Helm dan Rompi berbasis ESP32, tanpa bergantung pada infrastruktur Wi-Fi atau perangkat perantara. Pada pengujian ini, ESP32-S3 Supermini yang berperan sebagai Helm berfungsi sebagai pengirim (sender), sedangkan Heltec Tracker V1.2 sebagai Rompi berfungsi sebagai penerima (receiver). Kedua perangkat ditempatkan pada jarak sekitar 4 meter, masing-masing dimasukkan ke dalam kardus dan dipisahkan oleh dinding sebagai penghalang. Helm mengirimkan data secara broadcast berupa pesan teks real-time “Hello, World! #n” (dengan n sebagai nomor urut pesan). Tujuan pengujian adalah memverifikasi keandalan pengiriman data dalam kondisi lingkungan yang menantang. Hasil pengujian ini disajikan pada Tabel 14.

Tabel 14 Pengujian ESPNOW

No	Halo Status	Data Send	Harness Status	Data Receive	RSSI
1	Sent	Hello, World! #4	Receive	Hello, World! #4	-95Bm
2	Sent	Hello, World! #5	Receive	Hello, World! #5	-94dBm
3	Sent	Hello, World! #6	Receive	Hello, World! #6	-91dBm
4	Sent	Hello, World! #7	Receive	Hello, World! #7	-91dBm
5	Sent	Hello, World! #8	Receive	Hello, World! #8	-91dBm
6	Sent	Hello, World! #9	Receive	Hello, World! #9	-92dBm
7	Sent	Hello, World! #10	Receive	Hello, World! #10	-92dBm
8	Sent	Hello, World! #11	Receive	Hello, World! #11	-92dBm

9	Sent	Hello, World! #12	Receive	Hello, World! #12	-93dBm
10	Sent	Hello, World! #13	Receive	Hello, World! #13	-93dBm

Berdasarkan hasil pengujian komunikasi dua arah menggunakan protokol ESP-NOW menunjukkan bahwa semua pesan "Hello, World! #n" berhasil diterima oleh rompi tanpa error, meskipun perangkat ditempatkan di dalam kardus dan terhalang dinding. Nilai RSSI berkisar antara -91 dBm hingga -95 dBm, menandakan sinyal lemah namun masih cukup andal untuk komunikasi jarak dekat.

2. Pengujian GPS

Tujuan dari pengujian modul GPS adalah untuk memastikan tingkat keakuratan modul serta memastikan bahwa data yang diterima dari satelit dapat diproses dengan benar oleh mikrokontroler Heltec Wireless Tracker. Selanjutnya, modul GPS dipindahkan sejauh 5 meter setiap 1 menit sambil diamati nilai koordinat geografis berupa *Latitude*, *Longitude*, dan *Satellite* melalui serial monitor dan TFT pada Heltec Wireless Tracker. Perubahan posisi dimaksudkan untuk mengevaluasi respon dan akurasi pembacaan lokasi oleh sistem. Hasil pengujian modul GPS disajikan pada **Error! Reference source not found.** berikut

Tabel 15 Pegujian GPS

No	Posisi	<i>Latitude</i>	<i>Longitude</i>	<i>Satelite</i>
1	Posisi 1	-7.942939	112.613513	12
2	Posisi 2	-7.942937	112.613512	12
3	Posisi 3	-7.942940	112.613514	12
4	Posisi 4	-7.942738	112.613485	22
5	Posisi 5	-7.942737	112.613483	21
6	Posisi 6	-7.942755	112.613568	23

7	Posisi 7	-7.942758	112.613727	18
8	Posisi 8	-7.942760	112.613731	18
9	Posisi 9	-7.942808	112.613787	10
10	Posisi 10	-7.942817	112.613767	0

3. Pengujian modul GSM

Pengujian komunikasi data menggunakan jaringan GPRS dilakukan untuk mengevaluasi kemampuan Heltec Wireless Tracker V1.2 dalam mengirimkan data ke server melalui modul GSM SIM900A dengan APN *indosatgprs*. Pada pengujian ini, ESP32 mengirimkan data sensor dan status permainan menggunakan metode HTTP POST ke endpoint ThingSpeak (<http://api.thingspeak.com/update>) dengan autentikasi *Write API Key*, kemudian data tersebut diteruskan secara otomatis ke endpoint Vercel (</api/track>) melalui mekanisme ThingHTTP dan React dalam format JSON. Hasil pengujian menunjukkan bahwa koneksi GPRS, pengiriman data ke ThingSpeak, serta penerusan data ke Vercel sebagaimana ditunjukkan pada hasil pengujian pada tabel berikut.

Tabel 16 Pengujian GSM

No	Bit Kirim	Status	Bit Terima	Status Data	Waktu Respon
1	lat:-7.942981 long:112.613945 alt:521.3 sat:12 rssi:-76 snr:9.5	Send	lat:-7.942981 long:112.613945 alt:521.3 sat:12 rssi:-76 snr:9.5	Received	20 detik
2	lat:-7.943014 long:112.613968 alt:509.2 sat:9 rssi:-76 snr:9.5	Send	lat:-7.943014 long:112.613968 alt:509.2 sat:9 rssi:-76 snr:9.5	Received	18 detik
3	lat:-7.943003 long:112.613846 alt:517.6 sat:9 rssi:-72 snr:9.5	Send	lat:-7.943003 long:112.613846 alt:517.6 sat:9 rssi:-72 snr:9.5	Received	24 detik
4	lat:-7.943038 long:112.614037 alt:496.6 sat:10 rssi:-72 snr:9.5	Send	lat:-7.943038 long:112.614037 alt:496.6 sat:10 rssi:-72 snr:9.5	Received	15 detik

5	lat:-7.943038 long:112.614037 alt:496.6 sat:10 rssi:-72 snr:9.5	Send	lat:-7.943038 long:112.614037 alt:496.6 sat:10 rssi:-72 snr:9.5	Received	13 detik
---	--	------	--	----------	----------

4. Pengujian LoRa

Pengujian komunikasi LoRaWAN dilakukan untuk mengevaluasi kemampuan ESP32 pada Heltec Wireless Tracker dalam mengirimkan data status permainan melalui jaringan LoRa hingga tersimpan di server cloud. Perangkat melakukan *join network* ke The Things Network menggunakan metode OTAA, kemudian mengirimkan data sebagai payload biner melalui pesan uplink. Data tersebut didekodekan oleh TTN dan diteruskan melalui *webhook* ke endpoint Vercel (/api/ttn) menggunakan metode HTTP POST dalam format JSON. Hasil pengujian selanjutnya disajikan dalam tabel untuk menunjukkan performa dan keandalan komunikasi LoRaWAN yang digunakan.

Tabel 17 Pengujian LoRa

No	Status	Data send	Status Data	Data receiver	Waktu Respon
1	Send	Lat: -7.942899, Long: 112.613647, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 10.0 dB	Received	Lat: -7.942899, Long: 112.613647, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 10.0 dB	3 detik
2	Send	Lat: -7.942917, Long: 112.613663, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB	Received	Lat: -7.942917, Long: 112.613663, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB	3 detik
3	Send	Lat: -7.942923, Long: 112.613670, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB	Received	Lat: -7.942923, Long: 112.613670, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB	5 detik
4	Send	Lat: -7.942907, Long: 112.613678, Alt:	Received	Lat: -7.942907, Long: 112.613678, Alt:	3 detik

		1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB		1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB	
5	Send	Lat: -7.942935, Long: 112.613686, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB	Received	Lat: -7.942935, Long: 112.613686, Alt: 1 m, Sat: 0, RSSI: -70 dBm, SNR: 9.0 dB	5 detik

4.1.5 Pengujian Keseluruhan

Pengujian sistem secara keseluruhan pada perangkat Laser Tag dilakukan untuk memverifikasi bahwa seluruh subsistem meliputi senjata (SAT), helm, dan rompi yang dapat bekerja secara terintegrasi, sinkron, dan konsisten dalam skenario penggunaan nyata. Pengujian ini tidak hanya berfokus pada fungsi masing-masing perangkat, tetapi juga pada keandalan komunikasi antar modul, kestabilan sistem berbasis mikrokontroler, serta kemampuan sistem dalam mengirim, menerima, dan memproses data secara akurat dan real-time.

Dalam pelaksanaannya, pengujian dilakukan dengan mensimulasikan kondisi permainan menggunakan dua set unit Laser Tag yang dioperasikan oleh dua pemain. Seluruh perangkat dinyalakan untuk memulai proses inisialisasi, yang ditandai dengan indikator LED pada rompi. Setelah semua modul berhasil terhubung dengan server dan sistem dinyatakan siap, simulasi permainan dimulai untuk mengamati respons sistem terhadap interaksi tembak-menembak, termasuk deteksi sensor, pengiriman data, serta sinkronisasi informasi antar perangkat.

Pengujian pertama unit dengan ID “1” melakukan lima kali tembakan kearah Helm atau Rompi dari unit dengan ID “2” pada jarak 20 m. Data yang diterima oleh server dari hasil pengujian ini disajikan pada table 18.

Tabel 18 Pengujian Unit 2

ID	Status	Penembak	Helm / Rompi	Waktu	Lokasi Unit
2	Hit	Unit 1	Helm	14:02:15	-7.942939, 112.613513

2	Hit	Unit 1	Rompi	14:02:47	-7.942940, 112.613514
2	Hit	Unit 1	Helm	14:03:22	-7.942941, 112.613515
2	Miss	Unit 1	-	14:03:59	7.942939, 112.613513
2	Hit	Unit 1	Helm	14:04:34	-7.942938, 112.613512

Pengujian kedua unit dengan ID “2” melakukan lima kali tembakan kearah Helm dan Rompi dari unit dengan ID “1” pada jarak 20 m. Data yang diterima oleh server dari hasil pengujian ini disajikan pada tabel 19. berikut ini

Tabel 19 Pengujian Unit 1

ID	Status	Penembak	Helm / Rompi	Waktu	Lokasi Unit
1	Hit	Unit 2	Rompi	14:06:10	-7.943481, 112.613567
1	Hit	Unit 2	Hlem	14:06:45	-7.943483, 112.613569
1	Miss	Unit 2	-	14:07:19	-7.943482, 112.613568
1	Hit	Unit 2	Helm	14:07:54	-7.943480, 112.613566
1	Hit	Unit 2	Helm	14:08:29	-7.943484, 112.613570

BAB V

5.1 Kesimpulan

Berdasarkan hasil perancangan, implementasi, dan pengujian sistem Laser Tag berbasis ESP-NOW dan LoRa yang telah dilakukan, dapat ditarik kesimpulan sebagai berikut:

Pengujian regulator LM7805 menghasilkan tegangan keluaran sebesar 5,08 V dari tegangan masukan 7,98–8,14 V dengan error sebesar 0,16%, menunjukkan kestabilan catu daya yang sangat baik untuk seluruh subsistem. Rangkaian pemancar inframerah yang dilengkapi sensor suara, sensor getar, dan sensor api mampu mendeteksi kondisi aktivasi tembakan dengan akurat: sensor suara aktif pada intensitas di atas 59 dB, sensor getar merespons getaran ketukan dan jatuh dengan tegangan turun ke 120 mV–1,91 V (di bawah threshold 2,5 V), serta sensor api mendeteksi nyala api mulai jarak 246 cm. IC 74HC4078 sebagai gerbang NOR pengkondisi sinyal menghasilkan keluaran yang sepenuhnya sesuai dengan tabel kebenaran NOR tiga input, sehingga logika penggabungan sinyal sensor terverifikasi benar. Rangkaian penerima inframerah pada helm dan rompi mampu menerima sinyal IR pada jarak efektif hingga 200 meter pada malam hari, dengan cakupan sudut mencapai 45° tanpa lensa dan 180° saat menggunakan lensa Fresnel. Sistem anti-cheating berbasis empat sensor dengan gerbang NOR 4-input (IC 74HC4078) terbukti berfungsi: output bernilai HIGH (tidak curang) hanya ketika seluruh sensor dalam kondisi tidak terhalang, dan bernilai LOW (cheat terdeteksi) apabila salah satu atau lebih sensor tertutup, sesuai dengan 16 kombinasi tabel kebenaran yang diuji.

Pada pengujian protokol ESP-NOW antara ESP32-S3 Supermini (Helm) dan Heltec Tracker V1.2 (Rompi) pada jarak 4 meter dengan penghalang dinding dan kardus menunjukkan tingkat keberhasilan pengiriman data sebesar 100%, dengan seluruh 10 paket pesan diterima tanpa error. Nilai RSSI tercatat pada rentang –91 dBm hingga –95 dBm, menandakan sinyal dalam kondisi lemah namun masih andal untuk komunikasi jarak dekat antar unit pemain. Pengujian LoRaWAN menggunakan metode OTAA pada jaringan The Things Network (TTN) menunjukkan keberhasilan transmisi data uplink sebesar 100% dengan latensi rata-

rata 3–5 detik per event, membuktikan kemampuan LoRa sebagai jalur komunikasi jarak jauh yang efisien dan bertenaga rendah. Arsitektur komunikasi dua lapis—ESP-NOW untuk interaksi cepat antar pemain dalam jarak dekat dan LoRa untuk pengiriman data ke server pusat—terbukti saling melengkapi dan berjalan secara paralel tanpa konflik

Modul GPS pada Heltec Wireless Tracker V1.2 berhasil mengunci posisi dengan akurasi tinggi, mencatat koordinat dengan variasi $\pm 0,000002^\circ$ dan jumlah satelit antara 10 hingga 22 satelit. Data GPS, status IR, dan parameter LoRa dikirimkan melalui jalur GPRS (SIM900A, APN: indosatgprs) ke ThingSpeak channel 3268510 dengan latensi 13–24 detik, kemudian diteruskan secara otomatis ke endpoint Vercel (/api/track) melalui mekanisme ThingHTTP. Pada pengujian sistem secara keseluruhan dengan skenario dua pemain saling menembak pada jarak 60 meter, diperoleh hit rate sebesar 80% (8 dari 10 tembakan berhasil mengenai target) dengan seluruh event tercatat akurat di server beserta data koordinat, waktu, status IR, status anti-cheat, RSSI, dan SNR. Tidak ditemukan event cheat selama pengujian berlangsung. Hasil ini membuktikan bahwa hipotesis proyek akhir terpenuhi: kombinasi ESP-NOW dan LoRa mampu menghasilkan sistem Laser Tag yang akurat, responsif, dan dapat dipantau secara real-time melalui antarmuka web dalam skenario arena permainan nyata.

5.2 Saran

1. Pengembangan jarak deteksi sensor inframerah

Untuk medan latihan yang lebih luas, diperlukan penggunaan sensor atau lensa yang lebih sensitif agar jarak deteksi bisa diperpanjang lebih dari 200 meter

2. Uji Coba Lapangan dan Kondisi Ekstrem:

Disarankan pengujian sistem secara berkelanjutan di berbagai kondisi lingkungan, seperti saat hujan, panas, maupun kabut, guna memastikan keandalan pancaran dan penerimaan sinyal IR pada simulasi permainan sesungguhnya.

3. Optimalisasi Konsumsi Daya dan Efisiensi Driver:

Agar waktu operasional alat semakin optimal, riset tambahan terkait efisiensi rangkaian driver LED dan manajemen energi pada baterai perlu dilakukan,

misalnya melalui seleksi komponen dengan konsumsi daya rendah atau penggunaan algoritma sleep mode pada mikrokontroler.

4. Pengembangan Antarmuka Sistem:

Pengembangan fitur antarmuka (misal melalui aplikasi mobile atau dashboard berbasis web) sebaiknya terus dikembangkan agar lebih intuitif dan responsif

DAFTAR PUSTAKA

- Adelantado, F., Vilajosana, X., Tuset-Peiro, P., Martinez, B., Melia-Segui, J., & Watteyne, T. (2017). Understanding the Limits of LoRaWAN. *IEEE Communications Magazine*, 55(9), 34–40. <https://doi.org/10.1109/MCOM.2017.1600613>
- Adolfo, V., & Setia Budi, A. (2023). Perancangan Sistem Deteksi Node Baru Otomatis dalam Basis ESP-NOW dengan Tree Topology WSN. *Jurnal Pengembangan Teknologi Informasi Dan Ilmu Komputer*, 7(5), 2475–2480. <http://j-ptiik.ub.ac.id>
- Alliance, L. (2023). *About LoRaWAN® - LoRa Alliance®*. <https://Lora-Alliance.Org/about-Lorawan/>. <https://lora-alliance.org/about-lorawan/>
- Ariffin, A. A. Bin, Aziz, N. H. A., & Othman, K. A. (2011). Implementation of GPS for Location Tracking. In *IEEE Control and System Graduate Research Colloquium*. IEEE.
- Augustin, A., Yi, J., Clausen, T., & Townsley, W. M. (2016). A study of Lora: Long range & low power networks for the internet of things. *Sensors (Switzerland)*, 16(9). <https://doi.org/10.3390/s16091466>
- Automation, H. (2021). *Heltec Products Operation Documention*. https://Docs.Heltec.Org/?Spm=a2ty_o01.29997173.0.0.16715171k9VBi2. https://docs.heltec.org/?spm=a2ty_o01.29997173.0.0.16715171k9VBi2
- Azarov, O., Obertyukh, M., Prokofiev, M., Kalizhanova, A., & Kosaruk, O. (2025). PUSH-PULL VOLTAGE BUFFER WITH IMPROVED LOAD CAPACITY. *Informatyka, Automatyka, Pomiar w Gospodarce i Ochronie Srodowiska*, 15(2), 100–103. <https://doi.org/10.35784/iapgos.7257>
- Chengdu, H. A. T. Co. , Ltd. (2023). *Wireless Tracker V1.1*. Heltec Automation. <https://heltec.org>
- Deepika, K., & Renuka Prasad, B. (2023). *IoT-Based Dashboards for Monitoring Connected Farms Using Free Software and Open Protocols* (pp. 529–543). https://doi.org/10.1007/978-981-19-6004-8_43
- Electronics, R. (2020, April 29). *Vibration Sensor*. www.Rajguruelectronics.Com. www.rajguruelectronics.com

- Fajar Arofah, M., Mandayatma, E., & Nurcahyo, S. (2023). Penerapan Protokol Komunikasi ESP-Now pada Portable Traffic Light. *Jurnal Elektronika Dan Otomasi Industri*, 10(1), 52–59. <https://doi.org/10.33795/elkolind.v10i1.2749>
- Gromes, J. (2025). *Universal wireless communication library for embedded devices*. <https://github.com/Jgromes/RadioLib>
- Gromes, J., Daniel, V., Svoboda, P., Tremouillhe, O., Kubik, M., Jon, J., & Stejskal, M. (2025). Scalable small satellite platform for earth observation and scientific missions. *Journal of Physics: Conference Series*, 3078(1). <https://doi.org/10.1088/1742-6596/3078/1/012067>
- Guerbaoui, M., El Faiz, S., Ed-Dahhak, A., Lachhab, A., Benhala, B., Bakziz, Z., Ichou, I., & Selmani, A. (2025). From Data to Decisions: A Smart IoT and Cloud Approach to Environmental Monitoring. *E3S Web of Conferences*, 601. <https://doi.org/10.1051/e3sconf/202560100008>
- Hailan, M. A., Ghazaly, N. M., & Albaker, B. M. (2024). ESPNow Protocol-Based IIoT System for Remotely Monitoring and Controlling Industrial Systems. *Journal of Robotics and Control (JRC)*, 5(6), 1924–1942. <https://doi.org/10.18196/jrc.v5i6.21925>
- Itera. (2021). *BAB II LANDASAN TEORI II.1 Global Navigation Satellite System (GNSS)*.
- Kahn, J. M., & Barry, J. R. (1997). Wireless Infrared Communications. *IEEE*, 85, 265–298.
- Khan, H. (2023, June 23). *Fire/Flame Sensor Module*. <https://www.datasheethub.com/ir-flame-sensor-module/>.
- Konate, S., Li, C., Pranolo, A., & Traore, M. (2025). Modeling LoRa Backscatter Communication Range. *Global Journal of Computer Science and Technology: G Interdisciplinary*, 25(1).
- Lesmana, B., Heryana, G., & Jatira. (2023). Perancangan Sistem Kendali Mesin CNC (Computer Numerical Control) laser Cutting CO2 2 Axis Berbasis Arduino Uno. *Journal of Applied Mechanical Technology*, 2(2), 28–33. <https://doi.org/10.31884/jamet.v2i2.43>

- Maurya, K., Singh, M., & Jain, N. (2012). Real Time Vehicle Tracking System using GSM and GPS Technology-An Anti-theft Tracking System. *IJECS*, 1, 1103–1107. www.ijecse.org
- Munandar, A., David Maria Veronika, N., Abdulllah, D., & Sahputra, E. (2023). Miniature Design of Liquid Filling Machine Automatically Using ESP32 Based IOT (Internet of Things) Perancangan Miniatur Mesin Pengisi Cairan Otomatis Menggunakan ESP32 Berbasis IOT (Internet of Things). *JURNAL KOMITEK*, 3(1), 69–78. <https://doi.org/10.53697/jkomitek.v3i1>
- Nicola, C. I., Nicola, M., Sacerdotianu, D., & Pătru, I. (2024). *Real-time Monitoring of Cable Sag and Overhead Line Parameters Based on a Distributed Sensor Network and Implementation in a Web server and IoT*. <https://doi.org/10.20944/preprints202405.1414.v1>
- Nihayatus Sa'adah, Aries Pratiarso, Faridatun Nadziroh, Nailul Muna, Karimatun Nisa', I Gede Puja Astawa, Tri Budi Santoso, & Sultan Syahputra Yulianto. (2024). Analisa Performansi Komunikasi Lora (Long Range) pada Sistem Monitoring Buoy di Laut. *The Indonesian Journal of Computer Science*, 13(6). <https://doi.org/10.33022/ijcs.v13i6.4462>
- Nugraha, A. I., & Rosita, Y. D. (2024). Web-Based Integration of IoT and Artificial Intelligence for Monitoring Aquariums. *MATICS: Jurnal Ilmu Komputer Dan Teknologi Informasi (Journal of Computer Science and Information Technology)*, 16(1), 7–12. <https://doi.org/10.18860/mat.v16i1.23471>
- Parveen, Z., & Aldhlan, K. A. (2016). Missing Pilgrims Tracking System Using GPS, GSM and Arduino Microcontroller. In *International Conference on Recent Advances in Computer Systems*.
- Pratama, E. W., & Kiswantono, A. (2023). Electrical Analysis Using ESP-32 Module In Realtime. *JEECS (Journal of Electrical Engineering and Computer Sciences)*, 7(2), 1273–1284. <https://doi.org/10.54732/jeeecs.v7i2.21>
- Prettz, J. B., Da Costa, J. P. C. L., Alvim, J. R., Miranda, R. K., & Zanatta, M. R. (2017). Efficient and low cost MIMO communication architecture for smartbands applied to postoperative patient care. *RPC 2017 - Proceedings of the 2nd Russian-Pacific Conference on Computer Technology and Applications*, 2017-December, 1–5. <https://doi.org/10.1109/RPC.2017.8168055>
- Satheesh, M. B., Senthilkumar, B., Veeramanikandasamy, T., & Saravanakumar, O. M. (2016). Microcontroller and SD Card Based Standalone Data Logging

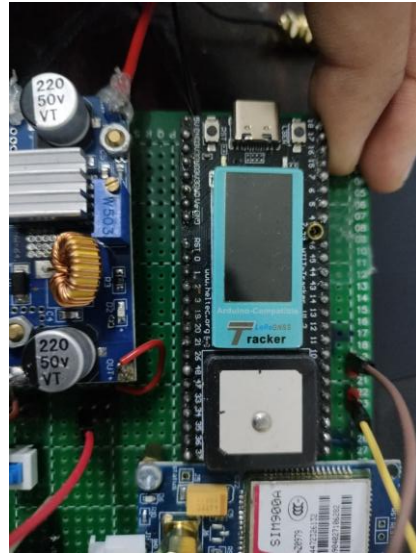
- System using SPI and I2C Protocols for Industrial Application. *International Journal of Advanced Research in Electrical, Electronics and Instrumentation Engineering* (An ISO, 5(4), 2208–2214. <https://doi.org/10.15662/IJAREEIE.2016.0504002>
- Shi, Z., Chow, C. W. K., Fabris, R., Liu, J., & Jin, B. (2022). Applications of Online UV-Vis Spectrophotometer for Drinking Water Quality Monitoring and Process Control: A Review. In *Sensors* (Vol. 22, Number 8). MDPI. <https://doi.org/10.3390/s22082987>
- Siradjuddin, I., Nurwicaksana, W. A., Riskitasari, S., Al Azhar, G., Rahman Hidayat, A., & Wicaksono, R. P. (2024). An Infrared Emitter Driver Circuit of SAT for MILES Application. In *Journal of Evrimata: Engineering and Physics* (Vol. 02, Number 02).
- Sunil Fulzele, V., & Athawale, S. V. (2024). IJIRID International Journal of Ingenious Research, Invention and Development GPS Technology: A Review, Architecture, and Working. *International Journal of Ingenious Research, Invention and Development*, 3, 543–549. <https://doi.org/10.5281/zenodo.14230440>
- Texas, I. (1976). $\mu A7800$ SERIES POSITIVE-VOLTAGE REGULATORS. www.ti.com/sc/package.
- Toyib, R., Bustami, I., Abdullah, D., & Onsardi, O. (2019). Penggunaan Sensor Passive Infrared Receiver (PIR) Untuk Mendeteksi Gerak Berbasis Short Message Service Gateway. *Pseudocode*, 6(2). <https://doi.org/10.33369/pseudocode.6.2.114-124>
- Urazayev, D., Eduard, A., Ahsan, M., & Zorbas, D. (2023). *Indoor Performance Evaluation of ESP-NOW*. <https://github.com/espressif/esp-now>
- Vishay Semiconductors. (2025). IR Receiver Modules for Remote Control Systems. In www.vishay.com. www.vishay.com
- Winternitz, L. M. B., Bamford, W. A., & Heckler, G. W. (2009). A GPS receiver for high-altitude satellite navigation. *IEEE Journal on Selected Topics in Signal Processing*, 3(4), 541–556. <https://doi.org/10.1109/JSTSP.2009.2023352>
- Yahya, M. S., Soeung, S., Chinda, F. E., Musa, U., & Yunusa, Z. (2024). Dual-band GPS/LoRa antenna for internet of thing applications. *Bulletin of Electrical Engineering and Informatics*, 13(2), 986–995. <https://doi.org/10.11591/eei.v13i2.6428>

- Yanziah, A., Sopian, S., & Rose, M. M. (2020). ANALISIS JARAK JANGKAUAN LORA DENGAN PARAMETER RSSI DAN PACKET LOSS PADA AREA URBAN. *JURNAL TEKNOLOGI TECHNOSCIENTIA*, 13, 59–67.
- Yunardi, R. T. (2017). Analisa Kinerja Sensor Inframerah dan Ultrasonik untuk Sistem Pengukuran Jarak pada Mobile Robot Inspection. *Setrum : Sistem Kendali-Tenaga-Elektronika-Telekomunikasi-Komputer*, 6(1), 33. <https://doi.org/10.36055/setrum.v6i1.1583>
- Zade, Prof. M. N., Nikose, Prof. M. D., & Aerkewar, Prof. P. N. (2012). Wireless Infrared Communications :A Survey. *International Journal of Engineering Research & Technology (IJERT)*, 1(3), 1–8. www.ijert.org
- Zhengyihong. (2025, August 29). *KY-038 Sound Sensor Module*. <https://Easyelecmodule.Com/Ky-038-Sound-Sensor-Module/>. <https://easyelecmodule.com/ky-038-sound-sensor-module/>

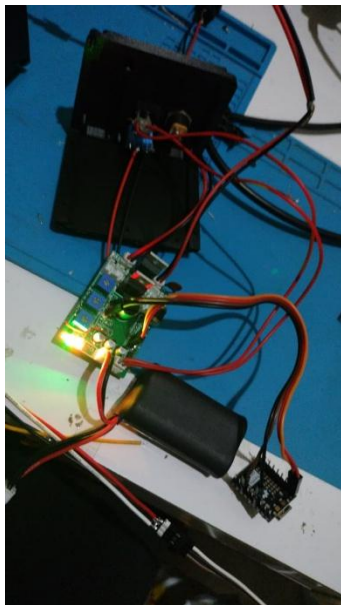
LAMPIRAN



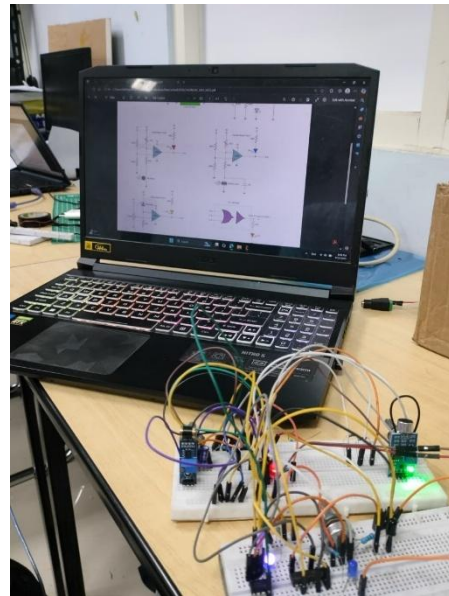
Gambar Lampiran 1 Proses dan pengambilan data GPS, GPRS dan SDCard



Gambar Lampiran 2 proses perancangan Electrical Rompi



Gambar Lampiran 3 proses pengambilan data sensor dan pengukuran tegangan



Gambar Lampiran 4 Proses perancangan electrical SAT

1. Program Transmitter Infrared

```
#include <Arduino.h>
#include <IRremote.hpp>
```

```
// =====
```

```

// Configuration
// =====
#define IR_SEND_PIN      2
#define BUTTON_PIN       5
#define STATUS_LED       LED_BUILTIN
#define SERIAL_BAUD      115200

#define TASK_PRIORITY_BUTTON  1
#define TASK_PRIORITY_SERIAL  1
#define TASK_PRIORITY_IR      2

#define TASK_STACK_SIZE_BUTTON 2048
#define TASK_STACK_SIZE_SERIAL 3072
#define TASK_STACK_SIZE_IR     2048

// =====
// Data Structure
// =====
struct IRCommand {
    uint32_t address;
    uint8_t  command;
    uint8_t  repeat;
};

// =====
// Global Shared Resources
// =====
IRCommand lastIRCommand = {0x04, 0x04, 1}; // Default + shared
SemaphoreHandle_t xCommandMutex = nullptr; // Protects
lastIRCommand
QueueHandle_t irSendQueue = nullptr;      // Queue to trigger
send

// =====
// Task Declarations
// =====
void taskIRSender(void *pvParameters);
void taskSerialCommand(void *pvParameters);
void taskButtonPress(void *pvParameters);

// =====
// Setup
// =====
void setup() {
    Serial.begin(SERIAL_BAUD);
    pinMode(BUTTON_PIN, INPUT_PULLUP);
    pinMode(STATUS_LED, OUTPUT);

```

```

pinMode(IR_SEND_PIN, OUTPUT);

IrSender.begin(IR_SEND_PIN);

// Create mutex and queue
xCommandMutex = xSemaphoreCreateMutex();
irSendQueue = xQueueCreate(10, sizeof(bool)); // Just a trigger

if (xCommandMutex == nullptr || irSendQueue == nullptr) {
    Serial.println("❌ Failed to create RTOS resources!");
    while (1) delay(10);
}

// Start tasks
xTaskCreate(taskButtonPress, "Button",
TASK_STACK_SIZE_BUTTON, NULL, TASK_PRIORITY_BUTTON, NULL);
xTaskCreate(taskSerialCommand, "Serial",
TASK_STACK_SIZE_SERIAL, NULL, TASK_PRIORITY_SERIAL, NULL);
xTaskCreate(taskIRSender, "IRSend",
TASK_STACK_SIZE_IR, NULL, TASK_PRIORITY_IR, NULL);

Serial.println("✅ SIMPERA TEST IR TX & RX");
Serial.println("👤 Indrazno Siradjuddin");
Serial.println("📅 2025");
Serial.println("System Ready.");
Serial.printf("💡 Default IR: Addr=0x%04X, Cmd=0x%02X\n",
lastIRCommand.address, lastIRCommand.command);
Serial.println("👉 Send 'F' or 'F,<addr>,<cmd>' via serial");
Serial.println("👋 Press button to send LAST received command");
}

void loop() {
    delay(1000);
}

// =====
// Task: IR Sender
// =====
void taskIRSender(void *pvParameters) {
    bool trigger;
    (void) pvParameters;

    for (;;) {
        // Wait for send trigger
        if (xQueueReceive(irSendQueue, &trigger, portMAX_DELAY) ==
pdTRUE) {
            // Acquire mutex to read last command

```

```

        if (xSemaphoreTake(xCommandMutex, pdMS_TO_TICKS(100)) ==
pdTRUE) {
            IRCommand cmd = lastIRCommand; // Copy
            xSemaphoreGive(xCommandMutex);

            // Feedback
            digitalWrite(STATUS_LED, HIGH);
            IrSender.sendNEC(cmd.address, cmd.command, cmd.repeat);
            vTaskDelay(pdMS_TO_TICKS(150));
            digitalWrite(STATUS_LED, LOW);
            digitalWrite(IR_SEND_PIN, LOW);

            Serial.printf("☑ IR Sent: NEC Addr=0x%04X, Cmd=0x%02X,
Repeat=%d\n",
                        cmd.address, cmd.command, cmd.repeat);
        } else {
            Serial.println("✗ Failed to acquire command lock!");
        }
    }
}

// =====
// Task: Serial Command Parser
// =====
void taskSerialCommand(void *pvParameters) {
    (void) pvParameters;

    for (;;) {
        if (Serial.available()) {
            String input = Serial.readStringUntil('\n');
            input.trim();

            if (input.startsWith("F") || input.startsWith("f")) {
                IRCommand cmd = lastIRCommand; // Start with current

                int comma1 = input.indexOf(',');
                int comma2 = input.indexOf(',', comma1 + 1);

                if (comma1 != -1 && comma2 != -1) {
                    String addrStr = input.substring(comma1 + 1, comma2);
                    String cmdStr = input.substring(comma2 + 1);

                    addrStr.trim(); cmdStr.trim();

                    cmd.address = strtoul(addrStr.c_str(), NULL, 0);
                    cmd.command = strtoul(cmdStr.c_str(), NULL, 0);
                }
            }
        }
    }
}

```

```

        cmd.repeat = 1;

        // Update shared last command
        if (xSemaphoreTake(xCommandMutex, pdMS_TO_TICKS(100)) ==
pdTRUE) {
            lastIRCommand = cmd;
            xSemaphoreGive(xCommandMutex);

            Serial.printf("📧 Updated last command: F, 0x%04X,
0x%02X\n",
                        cmd.address, cmd.command);
        } else {
            Serial.println("❌ Failed to update command (lock
timeout)");
        }

        } else {
            Serial.println("📧 Triggering last stored command via
'F'");
        }

        // Trigger IR send immediately
        bool sendTrigger = true;
        if (xQueueSendToBack(irSendQueue, &sendTrigger,
pdMS_TO_TICKS(100)) != pdTRUE) {
            Serial.println("❌ Send queue full!");
        }
    }
}
vTaskDelay(pdMS_TO_TICKS(10));
}
}

// =====
// Task: Button Press
// =====
void taskButtonPress(void *pvParameters) {
    bool buttonPressed = false;
    bool sendTrigger = true;
    (void) pvParameters;

    for (;;) {
        if (digitalRead(BUTTON_PIN) == LOW && !buttonPressed) {
            buttonPressed = true;

            Serial.println("👉 Button Pressed: Sending LAST user
command");

```



```

        if (xQueueSendToBack(irSendQueue, &sendTrigger,
pdMS_TO_TICKS(100)) != pdTRUE) {
            Serial.println("✗ Send queue full! Button ignored.");
        }

        // Cooldown
        vTaskDelay(pdMS_TO_TICKS(500));
    }
    if (digitalRead(BUTTON_PIN) == HIGH) {
        buttonPressed = false;
    }

    vTaskDelay(pdMS_TO_TICKS(20));
}
}

```

2. Program Helmet

```

/*
 * Laser Tag Helm - IR + Anti-Cheating + ESP-NOW
 * Semua dalam satu file agar kompatibel dengan Arduino IDE
 */

#include <Arduino.h>
#include <IRremote.hpp>
#include <WiFi.h>
#include <esp_now.h>
#include <esp_wifi.h>

// === Pin Configuration ===
#define IR_RECEIVE_PIN 4
#define IR_LED_PIN 6
#define PHOTODIODE_PIN 5

// === Ganti dengan MAC Heltec Tracker Anda ===
uint8_t HELTEC_MAC[] = {0x10, 0x20, 0xBA, 0x65, 0x4D, 0x14};

// === ESP-NOW Data ===
typedef struct {
    uint16_t address;
    uint8_t command;
} ir_data_t;

ir_data_t lastPacket;
bool dataPending = false;

```

```

volatile bool sendOK = false;

// === Anti-Cheating ===
bool antiCheatActive = true;
unsigned long lastCheckTime = 0;
const unsigned long CHECK_INTERVAL = 100; // ms

// === ESP-NOW Callback ===
void onDataSent(const wifi_tx_info_t *info, esp_now_send_status_t
status) {
    sendOK = (status == ESP_NOW_SEND_SUCCESS);
}

void setup() {
    Serial.begin(115200);
    delay(500);

    // Setup anti-cheat
    pinMode(IR_LED_PIN, OUTPUT);
    pinMode(PHOTODIODE_PIN, INPUT);
    digitalWrite(IR_LED_PIN, HIGH); // Nyalakan IR LED terus-menerus

    // Setup IR receiver
    IrReceiver.begin(IR_RECEIVE_PIN, false);

    // Setup ESP-NOW
    WiFi.mode(WIFI_STA);
    WiFi.begin();
    WiFi.setSleep(false);
    esp_wifi_set_channel(1, WIFI_SECOND_CHAN_NONE);
    esp_now_init();
    esp_now_register_send_cb(onDataSent);

    esp_now_peer_info_t peer = {};
    memcpy(peer.peer_addr, HELTEC_MAC, 6);
    peer.channel = 1;
    peer.encrypt = false;
    esp_now_add_peer(&peer);

    Serial.println("IR Receiver Ready");
    Serial.println("Anti-Cheating System Active");
    Serial.println("ESP-NOW initialized – data dikirim ke Heltec");
}

void loop() {
    // === Cek anti-cheating setiap 100ms ===
    if (millis() - lastCheckTime >= CHECK_INTERVAL) {

```

```

lastCheckTime = millis();
int status = digitalRead(PHOTODIODE_PIN); // LOW = cheating
if (status == LOW) {
    if (antiCheatActive) {
        antiCheatActive = false;
        Serial.println("  WARNING: CHEATING DETECTED!");
        Serial.println("  Sinyal IR tidak akan diproses");
    }
} else {
    if (!antiCheatActive) {
        antiCheatActive = true;
        Serial.println("✓ Anti-Cheat System Normal - Signals
Accepted");
    }
}
}

// === Terima sinyal IR ===
if (IrReceiver.decode()) {
    if (antiCheatActive) {
        Serial.println("IR signal received!");

        // Format: Protocol=NEC Address=0x4, Command=0x4, Raw-
Data=0xFB04FB04, ...
        Serial.printf("Protocol=NEC Address=0x%X, Command=0x%X, Raw-
Data=0x%08X, %d bits, LSB first, Gap=%luus, Duration=%luus\n",
            IrReceiver.decodedIRData.address,
            IrReceiver.decodedIRData.command,
            (uint32_t)IrReceiver.decodedIRData.decodedRawData,
            IrReceiver.decodedIRData.numberOfBits,
            IrReceiver.irparams.initialGapTicks * MICROS_PER_TICK,
            IrReceiver.decodedIRData.rawlen * MICROS_PER_TICK
        );

        Serial.printf("Received NEC Signal | Address: 0x%X | Command:
0x%X\n",
            IrReceiver.decodedIRData.address,
            IrReceiver.decodedIRData.command
        );

        Serial.printf("Send with: IrSender.sendNEC(0x%X, 0x%X,
<numberOfRepeats>);\n",
            IrReceiver.decodedIRData.address,
            IrReceiver.decodedIRData.command
        );

        // Kirim via ESP-NOW

```

```

        lastPacket.address = IrReceiver.decodedIRData.address;
        lastPacket.command = IrReceiver.decodedIRData.command;
        dataPending = true;
        sendOK = false;
        esp_err_t result = esp_now_send(HELTEC_MAC,
        (uint8_t*)&lastPacket, sizeof(lastPacket));
        if (result == ESP_OK) delay(10);

    } else {
        Serial.println("⚠ SIGNAL REJECTED - Anti-Cheat
        Triggered!");
    }

    IrReceiver.resume();
}

// === Retry jika gagal ===
if (dataPending && !sendOK) {
    esp_err_t result = esp_now_send(HELTEC_MAC,
    (uint8_t*)&lastPacket, sizeof(lastPacket));
    if (result == ESP_OK) {
        delay(10);
        if (sendOK) dataPending = false;
    }
}

delay(2);
}

```

3. Program Rompi

```

// =====
// HELTEC TRACKER V1.10
// Base : V1.8
// Fix : Tambah mGsmTcp mutex untuk serialisasi
//       semua akses TCP SIM900A
//       (sendThingSpeakGprs, sendImmediateGprsPayload,
//       taskGprsLoop DB send)
//       Root cause V1.8: sendImmediateGprsPayload dari
//       taskIr/taskAntiCheat bisa conflict dengan
//       ThingSpeak TCP connect → code:0 / TCP FAILED
// =====
#define ENABLE_WIFI    1
#define ENABLE_GPRS    1
#define ENABLE_LORAWAN 1
#define ENABLE_SD      1

```

```

#include <Arduino.h>
#include <SPI.h>
#include <SD.h>
#include <Adafruit_ST7735.h>
#include <Adafruit_GFX.h>
#include <RadioLib.h>
#include <TinyGPS++.h>
#include <IRremote.hpp>

#if ENABLE_WIFI
#include <WiFi.h>
#include <HTTPClient.h>
#endif

#if ENABLE_GPRS
#define TINY_GSM_MODEM_SIM900
#define TINY_GSM_RX_BUFFER 1024
#include <HardwareSerial.h>
#include <TinyGsmClient.h>
#endif

// =====
// KONFIGURASI
// =====
#define WIFI_SSID          "UserAndroid"
#define WIFI_PASSWORD      "55555550"
#define VERCEL_API_URL     "https://laser-tag-  
project.vercel.app/api/track"
#define GPRS_APN           "indosatgprs"
#define GPRS_USER          "indosat"
#define GPRS_PASS          "indosat"
#define GPRS_SIM_PIN       ""

#define GPRS_HOST           "simpera.teknoklop.com"
#define GPRS_PORT          10000
#define GPRS_API_KEY       "tPmAT5Ab3j7F9"
#define GPRS_PATH          "/lasertag/api/track.php"
#define GPRS_CHEAT_PATH    "/lasertag/api/cheat_event.php"

// =====
// THINGSPEAK VIA SIM900A
// =====
#define TS_HOST             "api.thingspeak.com"
#define TS_PORT             80
#define TS_WRITE_KEY        "N6ATMD4JVI90HTHH"
#define TS_CHANNEL_ID       "3268510"
#define TS_INTERVAL_MS     15000

```

```

// field1=lat, field2=lng, field3=alt, field4=sat
// field5=rssi, field6=irStatus(1=HIT/0=NONE)
// field7=cheat, field8=snr

#define DEVICE_ID          "Heltec-P1"
#define IR_RECEIVE_PIN     5

// =====
// GPS CSV CONFIG
// =====
#define GPS_CSV_FILE       "/gps_log.csv"
#define GPS_CSV_INTERVAL_MS 5000

// =====
// ANTI-CHEAT CONFIG
// =====
#define IR_AMBIENT_PIN     2
#define CHEAT_DETECTION_MS 5000
#define CHEAT_CHECK_INTERVAL 100
#define CHEAT_ALERT_DURATION 10000
#define IR_IMMEDIATE_SEND  2

// =====
// LORAWAN KEYS
// =====
const uint64_t joinEUI = 0x0000000000000003;
const uint64_t devEUI  = 0x70B3D57ED0075AC0;
const uint8_t appKey[] =
{0x48,0xF0,0x3F,0xDD,0x9C,0x5A,0x9E,0xBA,0x42,0x83,0x01,0x1D,0x7D,0x
BB,0xF3,0xF8};
const uint8_t nwkKey[] =
{0x9B,0xF6,0x12,0xF4,0xAA,0x33,0xDB,0x73,0xAA,0x1B,0x7A,0xC6,0x4D,0x
70,0x02,0x26};

// =====
// PIN
// =====
#define Vext_Ctrl      3
#define LED_K          21
#define LED_BUILTIN    18
#define RADIO_SCLK_PIN 9
#define RADIO_MISO_PIN 11
#define RADIO_MOSI_PIN 10
#define RADIO_CS_PIN   8
#define RADIO_RST_PIN  12
#define RADIO_DIO1_PIN 14

```

```

#define RADIO_BUSY_PIN 13
#define GPS_TX_PIN      34
#define GPS_RX_PIN      33
#define GPRS_TX_PIN     17
#define GPRS_RX_PIN     16
#define GPRS_RST_PIN    15
#define SD_CS           4
#define TFT_CS          38
#define TFT_DC          40
#define TFT_RST         39
#define TFT_SCLK        41
#define TFT_MOSI        42

// =====
// TIMING
// =====
const uint32_t UPLINK_INTERVAL_MS = 1000;
const uint32_t WIFI_UPLOAD_MS     = 1000;
const uint32_t GPRS_INTERVAL_MS   = 30000;
const uint8_t  MAX_JOIN_ATTEMPTS  = 5;
const uint32_t JOIN_RETRY_MS      = 2000;
const uint32_t SD_WRITE_MS        = 2000;
const size_t   LOG_QUEUE_SIZE     = 20;
const TickType_t MTX              = pdMS_TO_TICKS(100);
const LoRaWANBand_t Region        = AS923;
const uint8_t  subBand            = 0;

// =====
// TFT – 5 halaman
// =====
#define TFT_W          160
#define TFT_H          80
#define TFT_PAGES      5
#define TFT_ROWS       6
#define PAGE_MS        3000
#define LINE_H         10
#define LEFT_M         3
#define TOP_M          10

// =====
// STRUCTS
// =====
struct GpsData { bool valid, locValid; float lat, lng, alt;
uint8_t sat, h, m, s; };
struct LoraStatus { bool joined; String ev; int16_t rssi; float snr;
};

```

```

struct WifiSt      { bool connected; String ip; unsigned long lastRC;
int code; uint32_t ok, fail; };
struct GprsSt      { bool connected; String ip; String lastEvt;
String lastResp; };
struct IrSt        { bool got; uint16_t addr; uint8_t cmd; unsigned
long t; uint32_t hitCount; };
struct ThingSpeakSt {
    bool      lastOk;
    uint32_t lastSentMs;
    int       lastCode;
    uint32_t okCount;
    uint32_t failCount;
};
struct CsvSt { uint32_t rowCount; bool headerOk; };

// Sesi ID untuk CSV
char      g_csvSesiId[8]    = "000000";
bool      g_csvSesiIdSet    = false;

struct Page { String row[TFT_ROWS]; };
struct __attribute__((packed)) Payload {
    uint32_t addr_id, shooter_addr;
    uint8_t  sub_id, shooter_sub, status;
    float    lat, lng, alt;
    uint16_t bat;
    uint8_t  sat;
    int16_t  rssi;
    int8_t   snr;
    uint8_t  cheatDetected;
};

// =====
// OBJECTS
// =====
TinyGPSPlus GPS;
SX1262      radio = new Module(RADIO_CS_PIN, RADIO_DIO1_PIN,
RADIO_RST_PIN, RADIO_BUSY_PIN);
LoRaWANNode node(&radio, &Region, subBand);
Adafruit_ST7735 tft = Adafruit_ST7735(TFT_CS, TFT_DC, TFT_MOSI,
TFT_SCLK, TFT_RST);
GFXcanvas16 fb(TFT_W, TFT_H);

#if ENABLE_GPRS
HardwareSerial sim900(2);
TinyGsm        modem(sim900);
TinyGsmClient  gsmClient(modem);
TinyGsmClient  gsmClientTS(modem);

```



```

#endif

// =====
// STATE
// =====
GpsData      g_gps  = {};
LoraStatus   g_lora = {false, "IDLE", 0, 0};
WifiSt       g_wifi = {false, "N/A", 0, 0, 0, 0};
GprsSt       g_gprs = {false, "N/A", "IDLE", "-"};
IrSt         g_ir   = {};
ThingSpeakSt g_ts    = {false, 0, 0, 0, 0};
CsvSt        g_csv  = {0, false};

// FIX V1.6: Flag IR hit persistent untuk ThingSpeak
volatile bool g_irHitPendingTS = false;

volatile bool    g_cheatDetected    = false;
volatile uint32_t g_cheatAlertExpiry = 0;

// =====
// TFT PAGES
// =====
Page g_pg[TFT_PAGES] = {
    { { "LoRaWAN Status", "Event: IDLE", "RSSI: -", "SNR: -", "Joined: NO", "-" } },
    { { "GPS Status", "Lat: -", "Long: -", "Alt: -", "Sat: -", "--:--:--" } },
    { { "GPRS/DB Status", "Conn: NO", "IP: -", "Last: -", "Resp: -", "Evt: IDLE" } },
    { { "IR Hit Info", "Shooter: -", "Sub: -", "Loc: NONE", "Time: --:--", "Hits: 0" } },
    { { "ThingSpeak", "Status: -", "Code: -", "OK: 0", "Fail: 0", "Ch:" TS_CHANNEL_ID } }
};

bool    sdOK    = false;
bool    irSend  = false;
bool    irPend  = false;
uint16_t irAddr = 0;
uint8_t irCmd   = 0;

// =====
// RTOS
// =====
SemaphoreHandle_t mGps, mLora, mTft, mWifi, mGprs, mIr, mSpi, mTs, mCsv;
// FIX V1.9: mutex baru untuk serialisasi semua TCP SIM900A

```

```

// SIM900A hanya support 1 koneksi TCP aktif sekaligus
// Tanpa mutex ini, sendImmediateGprsPayload (dari
taskIr/taskAntiCheat)
// bisa conflict dengan sendThingSpeakGprs → TCP FAILED / code:0
SemaphoreHandle_t mGsmTcp;

QueueHandle_t    qLog;

// =====
// FORWARD DECLARATIONS
// =====
void taskTft(void*);
void taskGps(void*);
void taskLora(void*);
void taskSd(void*);
void taskWifi(void*);
void taskGprsLoop(void*);
void taskIr(void*);
void taskAntiCheat(void*);
void taskGpsCsv(void*);
void logSd(const char*);
void wifiPost(float lat, float lng);
void sendImmediateGprsPayload(bool isCheat);
void sendThingSpeakGprs();
void buildDataPayload(Payload &payload);
void sdSave(float, float, float, uint8_t, int16_t, float, const
char*);
void writeCsvHeader();
void writeGpsCsv();
bool sdUpload();
String decodeState(int16_t);
void initGprsHardware();

// =====
// SETUP
// =====
void setup()
{
    pinMode(Vext_Ctrl, OUTPUT); digitalWrite(Vext_Ctrl, HIGH);
    pinMode(LED_K, OUTPUT);      digitalWrite(LED_K, HIGH);
    pinMode(LED_BUILTIN, OUTPUT); digitalWrite(LED_BUILTIN, LOW);
    pinMode(IR_AMBIENT_PIN, INPUT);

    Serial.begin(115200);
    Serial.println("\n[BOOT] Heltec Tracker V1.9");

    Serial1.begin(115200, SERIAL_8N1, GPS_RX_PIN, GPS_TX_PIN);

```

```

IrReceiver.begin(IR_RECEIVE_PIN, DISABLE_LED_FEEDBACK);

tft.initR(INITR_MINI160x80);
tft.setRotation(1);
tft.fillScreen(ST7735_BLACK);
tft.setTextColor(ST7735_CYAN); tft.setTextSize(1);
tft.setCursor(10, 15); tft.print("Heltec Tracker V1.10");
tft.setCursor(10, 28); tft.print(DEVICE_ID);
tft.setCursor(10, 41); tft.setTextColor(ST7735_WHITE);
tft.print("Starting...");

mGps    = xSemaphoreCreateMutex();
mLora   = xSemaphoreCreateMutex();
mTft    = xSemaphoreCreateMutex();
mWifi   = xSemaphoreCreateMutex();
mGprs   = xSemaphoreCreateMutex();
mIr     = xSemaphoreCreateMutex();
mSpi    = xSemaphoreCreateMutex();
mTs     = xSemaphoreCreateMutex();
mCsv    = xSemaphoreCreateMutex();
// FIX V1.9: mutex TCP SIM900A
mGsmTcp = xSemaphoreCreateMutex();
qLog     = xQueueCreate(LOG_QUEUE_SIZE, sizeof(char*));

xTaskCreatePinnedToCore(taskTft, "TFT", 32768, NULL, 3, NULL, 0);

#if ENABLE_SD
xTaskCreatePinnedToCore(taskSd, "SD ", 8192, NULL, 1, NULL, 0);
#endif

xTaskCreatePinnedToCore(taskGps,      "GPS",      8192, NULL, 3,
NULL, 1);
xTaskCreatePinnedToCore(taskIr,       "IR",       4096, NULL, 3,
NULL, 1);
xTaskCreatePinnedToCore(taskAntiCheat, "Cheat", 2048, NULL, 2,
NULL, 1);

#if ENABLE_WIFI
xTaskCreatePinnedToCore(taskWifi, "WiFi", 32768, NULL, 1, NULL,
1);
#endif

#if ENABLE_LORAWAN
xTaskCreatePinnedToCore(taskLora, "LoRa", 16384, NULL, 1, NULL,
1);
#endif

```

```

    #if ENABLE_GPRS
    initGprsHardware();
    xTaskCreatePinnedToCore(taskGprsLoop, "GPRS", 32768, NULL, 1,
    NULL, 1);
    #endif

    xTaskCreatePinnedToCore(taskGpsCsv, "GpsCsv", 4096, NULL, 1, NULL,
    0);
}

void loop() { vTaskDelay(pdMS_TO_TICKS(1000)); }

// =====
// TFT TASK
// =====
void taskTft(void *pv)
{
    vTaskDelay(pdMS_TO_TICKS(2000));
    uint32_t pageSwitch = millis();
    uint8_t currentPage = 0;

    for (;;)
    {
        if (millis() - pageSwitch >= PAGE_MS) {
            currentPage = (currentPage + 1) % TFT_PAGES;
            pageSwitch = millis();
        }

        if (xSemaphoreTake(mTs, pdMS_TO_TICKS(10)) == pdTRUE) {
            g_pg[4].row[0] = "ThingSpeak(GPRS)";
            g_pg[4].row[1] = g_ts.lastOk ? "Status: OK" : "Status: FAIL";
            g_pg[4].row[2] = "Code: " + String(g_ts.lastCode);
            g_pg[4].row[3] = "OK: " + String(g_ts.okCount);
            g_pg[4].row[4] = "Fail: " + String(g_ts.failCount);
            g_pg[4].row[5] = "Ch:" + String(TS_CHANNEL_ID);
            xSemaphoreGive(mTs);
        }

        if (xSemaphoreTake(mTft, pdMS_TO_TICKS(100)) == pdTRUE) {
            fb.fillScreen(ST7735_BLACK);
            fb.setTextColor(ST7735_WHITE);
            fb.setTextSize(1);

            fb.setCursor(0, 1);

            fb.print("L=");
            fb.setTextColor(g_lora.joined ? ST7735_GREEN : ST7735_RED);

```

```

fb.print(g_lora.joined ? "Ok" : "No");
fb.setTextColor(ST7735_WHITE); fb.print(" ");

fb.print("G=");
fb.setTextColor(g_gprs.connected ? ST7735_GREEN : ST7735_RED);
fb.print(g_gprs.connected ? "Ok" : "No");
fb.setTextColor(ST7735_WHITE); fb.print(" ");

fb.print("W=");
fb.setTextColor(g_wifi.connected ? ST7735_GREEN : ST7735_RED);
fb.print(g_wifi.connected ? "Ok" : "No");
fb.setTextColor(ST7735_WHITE); fb.print(" ");

fb.print("S=");
fb.setTextColor(sdOK ? ST7735_GREEN : ST7735_RED);
fb.print(sdOK ? "Ok" : "No");
fb.setTextColor(ST7735_WHITE);
fb.printf(" [%d/%d]", currentPage + 1, TFT_PAGES);

fb.drawLine(0, 9, TFT_W, 9, ST7735_BLUE);

for (int i = 0; i < TFT_ROWS; i++) {
    int y = TOP_M + i * LINE_H;
    if (y >= TFT_H - 10) break;
    fb.setCursor(LEFT_M, y);
    fb.print(g_pg[currentPage].row[i]);
}

if (millis() < g_cheatAlertExpiry) {
    fb.fillRect(0, TFT_H - 10, TFT_W, 10, ST7735_RED);
    fb.setTextColor(ST7735_WHITE, ST7735_RED);
    fb.setCursor(15, TFT_H - 9);
    fb.print("CHEAT RECORDED");
}

xSemaphoreGive(mTft);

if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(40)) == pdTRUE) {
    tft.drawRGBBitmap(0, 0, fb.getBuffer(), TFT_W, TFT_H);
    xSemaphoreGive(mSpi);
}
}
vTaskDelay(pdMS_TO_TICKS(100));
}
}

// =====

```

```

// ANTI-CHEAT TASK
// =====
void taskAntiCheat(void *pv)
{
    uint32_t coverStart = 0;
    bool      wasCovered = false;

    for (;;) {
        bool isCovered = digitalRead(IR_AMBIENT_PIN) == LOW;

        if (isCovered) {
            if (!wasCovered) { coverStart = millis(); wasCovered = true; }
            else if (!g_cheatDetected && millis() - coverStart >=
CHEAT_DETECTION_MS) {
                g_cheatDetected      = true;
                g_cheatAlertExpiry = millis() + CHEAT_ALERT_DURATION;
                logSd("[CHEAT] IR covered >5s - CHEAT DETECTED!");
                Serial.println("[CHEAT] DETECTED!");
                if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[5] =
"Evt: CHEAT!"; xSemaphoreGive(mTft); }
                if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { g_gprs.lastEvt =
"CHEAT"; xSemaphoreGive(mGprs); }
                for (uint8_t i = 0; i < IR_IMMEDIATE_SEND; i++) {
                    sendImmediateGprsPayload(true);
                    vTaskDelay(pdMS_TO_TICKS(200));
                }
            }
        } else {
            wasCovered = false;
            if (g_cheatDetected) {
                g_cheatDetected = false;
                logSd("[CHEAT] cleared");
            }
        }
        vTaskDelay(pdMS_TO_TICKS(CHEAT_CHECK_INTERVAL));
    }
}

// =====
// GPS TASK
// =====
void taskGps(void *pv)
{
    for (;;) {
        int n = Serial1.available();
        for (int i = 0; i < n; i++) GPS.encode(Serial1.read());
    }
}

```

```

bool hasT = GPS.time.isValid(), hasL = GPS.location.isValid();

if (xSemaphoreTake(mGps, MTX) == pdTRUE) {
    g_gps.valid    = hasT || hasL;
    g_gps.locValid = hasL;
    g_gps.sat      = GPS.satellites.value();
    if (hasL) { g_gps.lat = GPS.location.lat(); g_gps.lng =
GPS.location.lng(); g_gps.alt = GPS.altitude.meters(); }
    if (hasT) { g_gps.h = GPS.time.hour(); g_gps.m =
GPS.time.minute(); g_gps.s = GPS.time.second(); }
    xSemaphoreGive(mGps);
}

// Catat sesi_id sekali saat GPS pertama valid
if (hasT && !g_csvSesiIdSet) {
    float lon = g_gps.locValid ? g_gps.lng : 107.0f;
    int utcOffset = (lon < 115.0f) ? 7 : (lon < 134.0f) ? 8 : 9;
    int8_t dh = (int8_t)((g_gps.h + utcOffset + 24) % 24);
    snprintf(g_csvSesiId, sizeof(g_csvSesiId), "%02d%02d%02d", dh,
g_gps.m, g_gps.s);
    g_csvSesiIdSet = true;
    Serial.printf("[GPS] Sesi ID set: %s\n", g_csvSesiId);
}

if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
    int8_t dh = 0;
    if (hasT) {
        float lon = g_gps.locValid ? g_gps.lng : 107.0f;
        int utcOffset = (lon < 115.0f) ? 7 : (lon < 134.0f) ? 8 : 9;
        dh = (int8_t)((g_gps.h + utcOffset + 24) % 24);
    }
    char tb[12]; snprintf(tb, 12, "%02d:%02d:%02d", hasT ? dh : 0,
hasT ? g_gps.m : 0, hasT ? g_gps.s : 0);
    g_pg[1].row[0] = "GPS Status";
    g_pg[1].row[1] = hasL ? ("Lat: " + String(g_gps.lat, 6)) :
"Lat: -";
    g_pg[1].row[2] = hasL ? ("Long: " + String(g_gps.lng, 6)) :
"Long: -";
    g_pg[1].row[3] = hasL ? ("Alt: " + String(g_gps.alt, 1) + "m")
: "Alt: -";
    g_pg[1].row[4] = "Sat: " + String(g_gps.sat);
    g_pg[1].row[5] = "Time: " + String(tb);
    xSemaphoreGive(mTft);
}
vTaskDelay(pdMS_TO_TICKS(1000));
}
}

```

```

// =====
// GPS CSV TASK (core 0)
// =====
void writeCsvHeader()
{
    if (!sdOK) return;
    if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(200)) == pdTRUE) {
        bool exists = SD.exists(GPS_CSV_FILE);
        File f = SD.open(GPS_CSV_FILE, FILE_APPEND);
        if (f) {
            if (!exists)

f.println("sesi_id,tanggal,millis,jam,menit,detik,latitude,longitude
,altitude_m,satellites,gps_valid,ir_status,cheat");
            f.close();
            if (xSemaphoreTake(mCsv, MTX) == pdTRUE) { g_csv.headerOk =
true; xSemaphoreGive(mCsv); }
        }
        xSemaphoreGive(mSpi);
    }
}

void writeGpsCsv()
{
    if (!sdOK) return;
    float lat = 0, lng = 0, alt = 0;
    uint8_t sat = 0, h = 0, m = 0, s = 0;
    uint8_t gday = 0, gmon = 0; uint16_t gyear = 0;
    bool locValid = false;

    if (xSemaphoreTake(mGps, MTX) == pdTRUE) {
        lat = g_gps.lat; lng = g_gps.lng; alt = g_gps.alt;
        sat = g_gps.sat; h = g_gps.h; m = g_gps.m; s = g_gps.s;
        locValid = g_gps.locValid;
        if (GPS.date.isValid()) {
            gday = GPS.date.day();
            gmon = GPS.date.month();
            gyear = GPS.date.year();
        }
        xSemaphoreGive(mGps);
    }

    // Baca IR status – pakai g_irHitPendingTS (persistent flag)
    const char* irStr = g_irHitPendingTS ? "HIT" : "NONE";

    // Baca cheat status

```



```

uint8_t cheatVal = g_cheatDetected ? 1 : 0;

int8_t dh = h;
float lonRef = locValid ? lng : 107.0f;
int utcOffset = (lonRef < 115.0f) ? 7 : (lonRef < 134.0f) ? 8 : 9;
dh = (int8_t)((h + utcOffset + 24) % 24);

char tglBuf[10];
if (gyear > 2000)
    snprintf(tglBuf, sizeof(tglBuf), "%02u%02u%04u", gday, gmon,
gyear);
else
    snprintf(tglBuf, sizeof(tglBuf), "00000000");

char row[160];
sprintf(row, sizeof(row),
"%s,%s,%lu,%02d,%02d,%02d,%.6f,%.6f,%.1f,%u,%s,%s,%u",
    g_csvSesiId, tglBuf,
    millis(), dh, m, s,
    lat, lng, alt, sat,
    locValid ? "1" : "0",
    irStr,
    cheatVal);

if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(200)) == pdTRUE) {
    File f = SD.open(GPS_CSV_FILE, FILE_APPEND);
    if (f) {
        f.println(row); f.close();
        if (xSemaphoreTake(mCsv, MTX) == pdTRUE) { g_csv.rowCount++;
xSemaphoreGive(mCsv); }
    }
    xSemaphoreGive(mSpi);
}

void taskGpsCsv(void *pv)
{
    while (!sdOK) vTaskDelay(pdMS_TO_TICKS(500));
    writeCsvHeader();
    uint32_t lastWrite = 0;
    for (;;) {
        if (millis() - lastWrite >= GPS_CSV_INTERVAL_MS) {
            writeGpsCsv();
            lastWrite = millis();
        }
        vTaskDelay(pdMS_TO_TICKS(1000));
    }
}

```

```

}

// =====
// LORAWAN TASK
// =====
#if ENABLE_LORAWAN
void taskLora(void *pv)
{
    vTaskDelay(pdMS_TO_TICKS(500));
    SPI.begin(RADIO_SCLK_PIN, RADIO_MISO_PIN, RADIO_MOSI_PIN,
RADIO_CS_PIN);
    delay(100);

    auto setLoraPage = [](const char *ev, int16_t rssi, float snr,
bool joined) {
        if (xSemaphoreTake(mTft, pdMS_TO_TICKS(30)) == pdTRUE) {
            g_pg[0].row[0] = "LoRaWAN Status";
            g_pg[0].row[1] = "Event: " + String(ev);
            g_pg[0].row[2] = "RSSI: " + String(rssi) + " dBm";
            g_pg[0].row[3] = "SNR: " + String(snr, 1) + " dB";
            g_pg[0].row[4] = "NA";
            g_pg[0].row[5] = joined ? "Joined: YES" : "Joined: NO";
            xSemaphoreGive(mTft);
        }
    };

    if (radio.begin() != RADIOLIB_ERR_NONE) {
setLoraPage("RADIO_FAIL", 0, 0, false); vTaskDelete(NULL); }
    if (node.beginOTAA(joinEUI, devEUI, nwkKey, appKey) !=
RADIOLIB_ERR_NONE) { setLoraPage("INIT_FAIL", 0, 0, false);
vTaskDelete(NULL); }

    Serial.println("[LoRa] Starting OTAA join...");
    setLoraPage("JOINING...", 0, 0, false);

    for (uint8_t a = 1; a <= MAX_JOIN_ATTEMPTS; a++) {
        String attemptMsg = "OTAA " + String(a) + "/" +
String(MAX_JOIN_ATTEMPTS);
        Serial.println("[LoRa] " + attemptMsg);
        setLoraPage(attemptMsg.c_str(), 0, 0, false);

        int16_t s = node.activateOTAA();
        vTaskDelay(pdMS_TO_TICKS(10));

        if (s == RADIOLIB_LORAWAN_NEW_SESSION) {
            Serial.println("[LoRa] OTAA Join OK!");

```

```

        if (xSemaphoreTake(mLora, MTX) == pdTRUE) { g_lora.joined =
true; xSemaphoreGive(mLora); }
        setLoraPage("JOINED", 0, 0, true);
        break;
    }
    Serial.println("[LoRa] Join fail: " + decodeState(s));
    vTaskDelay(pdMS_TO_TICKS(JOIN_RETRY_MS));
}

if (!g_lora.joined) { setLoraPage("NO_JOIN", 0, 0, false);
vTaskDelete(NULL); }

uint32_t tUp = 0;
for (;;) {
    if (irSend || millis() - tUp >= UPLINK_INTERVAL_MS) {
        irSend = false;
        Payload pl = {};
        if (xSemaphoreTake(mGps, MTX) == pdTRUE) { pl.sat = g_gps.sat;
pl.lat = g_gps.lat; pl.lng = g_gps.lng; pl.alt = g_gps.alt;
xSemaphoreGive(mGps); }
        if (irPend) { pl.shooter_sub = 1; pl.shooter_addr = irAddr;
pl.sub_id = irCmd; pl.status = 1; irPend = false; }
        else if (xSemaphoreTake(mIr, MTX) == pdTRUE) {
            pl.shooter_addr = g_ir.got ? g_ir.addr : 0;
            pl.sub_id      = g_ir.got ? g_ir.cmd : 0;
            pl.shooter_sub = g_ir.got ? 1 : 0;
            pl.status      = g_ir.got ? 1 : 0;
            xSemaphoreGive(mIr);
        }
        if (xSemaphoreTake(mLora, MTX) == pdTRUE) { pl.rssi =
g_lora.rssi; pl.snr = (int8_t)(g_lora.snr + 0.5f);
xSemaphoreGive(mLora); }
        pl.cheatDetected = g_cheatDetected ? 1 : 0;

        uint8_t buf[sizeof(Payload)]; memcpy(buf, &pl, sizeof(pl));
        int16_t s = node.sendReceive(buf, sizeof(buf));

        String ev; float snr = 0; int16_t rssi = 0;
        if (s < RADIOLIB_ERR_NONE) { ev = "TX_FAIL"; }
        else {
            ev = s > 0 ? "TX+RX_OK" : "TX_OK";
            snr = radio.getSNR();
            rssi = radio.getRSSI();
            // LoRa task hanya reset g_ir.got - g_irHitPendingTS TIDAK
            di-reset di sini
            if (xSemaphoreTake(mIr, MTX) == pdTRUE) { g_ir.got = false;
xSemaphoreGive(mIr); }

```

```

    }
    if (xSemaphoreTake(mLora, MTX) == pdTRUE) { g_lora.ev = ev;
g_lora.snr = snr; g_lora.rssi = rssi; xSemaphoreGive(mLora); }
    setLoraPage(ev.c_str(), rssi, snr, true);
    tUp = millis();
  }
  vTaskDelay(pdMS_TO_TICKS(100));
}
}
#endif

// =====
// WIFI TASK
// =====
#if ENABLE_WIFI
void taskWifi(void *pv)
{
  WiFi.disconnect(true); vTaskDelay(pdMS_TO_TICKS(200));
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  unsigned long t0 = millis();

  while (WiFi.status() != WL_CONNECTED && millis() - t0 < 20000)
    vTaskDelay(pdMS_TO_TICKS(500));

  if (WiFi.status() == WL_CONNECTED) {
    String ip = WiFi.localIP().toString();
    Serial.printf("[WiFi] Connected IP: %s\n", ip.c_str());
    if (xSemaphoreTake(mWifi, MTX) == pdTRUE) { g_wifi.connected =
true; g_wifi.ip = ip; xSemaphoreGive(mWifi); }
    sdUpload();
  } else {
    Serial.println("[WiFi] Not connected (hotspot off - normal)");
  }

  uint32_t tUp = 0;
  for (;;) {
    if (WiFi.status() != WL_CONNECTED) {
      if (xSemaphoreTake(mWifi, MTX) == pdTRUE) { g_wifi.connected =
false; xSemaphoreGive(mWifi); }
      if (millis() - g_wifi.lastRC >= 30000) {
        g_wifi.lastRC = millis();
        WiFi.disconnect(true); vTaskDelay(pdMS_TO_TICKS(300));
        WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
        unsigned long tw = millis();
        while (WiFi.status() != WL_CONNECTED && millis() - tw <
15000) vTaskDelay(pdMS_TO_TICKS(500));

```

```

        if (WiFi.status() == WL_CONNECTED) {
            String ip = WiFi.localIP().toString();
            Serial.printf("[WiFi] Connected IP: %s\n", ip.c_str());
            if (xSemaphoreTake(mWifi, MTX) == pdTRUE) {
                g_wifi.connected = true; g_wifi.ip = ip; xSemaphoreGive(mWifi); }
        }
    }
}

if (g_wifi.connected && millis() - tUp >= WIFI_UPLOAD_MS) {
    float lat = 0, lng = 0;
    if (xSemaphoreTake(mGps, MTX) == pdTRUE) { lat = g_gps.lat;
lng = g_gps.lng; xSemaphoreGive(mGps); }
    wifiPost(lat, lng);
    tUp = millis();
}
vTaskDelay(pdMS_TO_TICKS(1000));
}
}

void wifiPost(float lat, float lng)
{
    char irBuf[32] = "-"; float alt = 0; uint8_t sat = 0; int16_t rssi
= 0; float snr = 0;
    uint16_t ia = 0; uint8_t ic = 0;
    const char* hitStatus = "NONE";

    if (xSemaphoreTake(mIr, MTX) == pdTRUE) {
        if (g_ir.got) { snprintf(irBuf, 32, "HIT:0x%X-0x%X", g_ir.addr,
g_ir.cmd); ia = g_ir.addr; ic = g_ir.cmd; hitStatus = "HIT"; }
        xSemaphoreGive(mIr);
    }
    if (xSemaphoreTake(mGps, MTX) == pdTRUE) { alt = g_gps.alt; sat =
g_gps.sat; xSemaphoreGive(mGps); }
    if (xSemaphoreTake(mLora, MTX) == pdTRUE) { rssi = g_lora.rssi;
snr = g_lora.snr; xSemaphoreGive(mLora); }

    char pl[450];
    snprintf(pl, sizeof(pl),
        "{ \"deviceId\": \"%s\", \"lat\": %.6f, \"lng\": %.6f, \"alt\": %.1f, \"
        \"sats\": %u, \"rssi\": %d, \"snr\": %.1f, \"

        \"irAddress\": %u, \"irCommand\": %u, \"hitStatus\": \"%s\", \"cheatDetec
ted\": %d }",
        DEVICE_ID, lat, lng, alt, sat, rssi, snr, ia, ic, hitStatus,
g_cheatDetected ? 1 : 0);

```

```

    bool okVercel = false; int codeVercel = 0;
    for (int i = 1; i <= 3 && !okVercel; i++) {
        HTTPClient http; http.begin(VERCEL_API_URL);
        http.addHeader("Content-Type", "application/json");
    http.setTimeout(10000);
        codeVercel = http.POST(pl); okVercel = (codeVercel == 200 ||
codeVercel == 201); http.end();
        if (!okVercel && i < 3) vTaskDelay(pdMS_TO_TICKS(2000));
    }
    Serial.printf("[WiFi] Vercel: %s (code:%d)\n", okVercel ? "OK" :
"FAIL", codeVercel);
    if (!okVercel) sdSave(lat, lng, alt, sat, rssi, snr, irBuf);
}
#endif

// =====
// GPRS – sendThingSpeakGprs
// FIX V1.9: Ambil mGsmTcp sebelum connect TCP
//          Stop client dulu sebelum connect baru
//          Release mGsmTcp setelah selesai
// =====
#if ENABLE_GPRS

void sendThingSpeakGprs()
{
    if (!modem.isGprsConnected()) { logSd("[TS] GPRS not connected");
return; }

    // FIX V1.9: Minta mutex TCP – tunggu max 5 detik
    // Ini mencegah conflict dengan sendImmediateGprsPayload
    // yang bisa dipanggil dari taskIr/taskAntiCheat
    if (xSemaphoreTake(mGsmTcp, pdMS_TO_TICKS(5000)) != pdTRUE) {
        logSd("[TS] mGsmTcp timeout - skip");
        Serial.println("[TS] mGsmTcp timeout - skip");
        return;
    }

    // Pastikan client bersih sebelum connect baru
    gsmClientTS.stop();
    vTaskDelay(pdMS_TO_TICKS(300));

    float lat = 0, lng = 0, alt = 0;
    uint8_t sat = 0; int16_t rssi = 0; float snr = 0;
    uint8_t cheat = g_cheatDetected ? 1 : 0;
    uint16_t irAddr = 0; uint8_t irCmd = 0;

```

```

    if (xSemaphoreTake(mGps, MTX) == pdTRUE) { lat = g_gps.lat; lng =
g_gps.lng; alt = g_gps.alt; sat = g_gps.sat; xSemaphoreGive(mGps); }
    if (xSemaphoreTake(mLora, MTX) == pdTRUE) { rssi = g_lora.rssi;
snr = g_lora.snr; xSemaphoreGive(mLora); }
    if (xSemaphoreTake(mIr, MTX) == pdTRUE) { irAddr = g_ir.addr;
irCmd = g_ir.cmd; xSemaphoreGive(mIr); }

```

```

// FIX V1.6: Baca g_irHitPendingTS langsung (volatile, no mutex
needed)

```

```

uint8_t irFlag = g_irHitPendingTS ? 1 : 0;

```

```

Serial.printf("[TS] irFlag=%u (g_irHitPendingTS=%d) addr=0x%04X
cmd=0x%02X\n", irFlag, (int)g_irHitPendingTS, irAddr, irCmd);

```

```

char jsonBody[350];
// field6 = 0/1 (numeric untuk ThingSpeak React trigger)
// field9 = IR address (decimal)
// field10 = IR command (decimal)
snprintf(jsonBody, sizeof(jsonBody),
    "{ \"api_key\": \"%s\", \"
    \"field1\": %.6f, \" // lat
    \"field2\": %.6f, \" // lng
    \"field3\": %.1f, \" // alt
    \"field4\": %u, \" // sat
    \"field5\": %d, \" // rssi
    \"field6\": %d, \" // irFlag (1=HIT, 0=NONE) - numeric untuk
React
    \"field7\": %u, \" // cheat
    \"field8\": %.1f, \" // snr
    \"field9\": %u, \" // irAddr (address decimal)
    \"field10\": %u} \", // irCmd (command decimal)
    TS_WRITE_KEY, lat, lng, alt, sat, rssi, irFlag, cheat, snr,
irAddr, irCmd
);

```

```

int bodyLen = strlen(jsonBody);
bool ok = false; int respCode = 0;

```

```

Serial.printf("[TS] Connecting to %s:%d\n", TS_HOST, TS_PORT);

```

```

if (gsmClientTS.connect(TS_HOST, TS_PORT)) {
    gsmClientTS.print("POST /update.json HTTP/1.1\r\n");
    gsmClientTS.print("Host: api.thingspeak.com\r\n");
    gsmClientTS.print("Content-Type: application/json\r\n");
    gsmClientTS.print("Content-Length: ");
gsmClientTS.print(bodyLen);
    gsmClientTS.print("\r\nConnection: close\r\n\r\n");
}

```

```

gsmClientTS.print(jsonBody);

unsigned long timeout = millis();
while (gsmClientTS.available() == 0) {
    if (millis() - timeout > 8000) { Serial.println("[TS]
Timeout"); break; }
    vTaskDelay(pdMS_TO_TICKS(10));
}
String response = "";
while (gsmClientTS.available()) response +=
(char)gsmClientTS.read();
gsmClientTS.stop();

if (response.startsWith("HTTP/")) {
    int spaceIdx = response.indexOf(' ');
    if (spaceIdx > 0) respCode = response.substring(spaceIdx + 1,
spaceIdx + 4).toInt();
}
ok = (respCode == 200 && response.indexOf("entry_id") >= 0);

Serial.printf("[TS] Response code: %d\n", respCode);
Serial.println("[TS] Body: " +
response.substring(response.lastIndexOf("\r\n\r\n") + 4,
response.length()));
} else {
    Serial.println("[TS] TCP connect to ThingSpeak FAILED");
    logSd("[TS] TCP connect failed");
}

// FIX V1.6: Reset g_irHitPendingTS HANYA jika berhasil kirim
if (ok && g_irHitPendingTS) {
    g_irHitPendingTS = false;
    Serial.println("[TS] g_irHitPendingTS cleared (sent OK)");
}

if (xSemaphoreTake(mTs, MTX) == pdTRUE) {
    g_ts.lastOk = ok; g_ts.lastCode = respCode; g_ts.lastSentMs =
millis();
    if (ok) g_ts.okCount++; else g_ts.failCount++;
    xSemaphoreGive(mTs);
}

char logMsg[80];
snprintf(logMsg, 80, "[TS] %s code:%d lat:%.4f lng:%.4f ir:%u", ok
? "OK" : "FAIL", respCode, lat, lng, irFlag);
logSd(logMsg); Serial.println(logMsg);

```



```

    if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
        g_pg[4].row[1] = ok ? "Status: OK" : "Status: FAIL";
        g_pg[4].row[2] = "Code: " + String(respCode);
        xSemaphoreGive(mTft);
    }

    // FIX V1.9: Release mutex TCP
    xSemaphoreGive(mGsmTcp);
}

void buildDataPayload(Payload &payload)
{
    payload.addr_id = 0x12345678; payload.sub_id = random(0, 256);
    payload.status = 1;
    if (xSemaphoreTake(mGps, MTX) == pdTRUE) { payload.lat =
g_gps.lat; payload.lng = g_gps.lng; payload.alt = g_gps.alt;
xSemaphoreGive(mGps); }
    else { payload.lat = payload.lng = payload.alt = 0.0f; }
    if (xSemaphoreTake(mIr, MTX) == pdTRUE) { payload.shooter_addr =
g_ir.got ? g_ir.addr : 0; payload.shooter_sub = g_ir.got ? g_ir.cmd
: 0; payload.sub_id = g_ir.got ? g_ir.cmd : 0; xSemaphoreGive(mIr);
}
    else { payload.shooter_addr = 0; payload.shooter_sub = 0; }
    payload.cheatDetected = g_cheatDetected ? 1 : 0;
}

void initGprsHardware()
{
    Serial.println("Initializing GPRS modem...");
    pinMode(GPRS_RST_PIN, OUTPUT);
    digitalWrite(GPRS_RST_PIN, HIGH); delay(200);
    digitalWrite(GPRS_RST_PIN, LOW); delay(200);
    digitalWrite(GPRS_RST_PIN, HIGH); delay(1000);
    sim900.begin(57600, SERIAL_8N1, GPRS_RX_PIN, GPRS_TX_PIN);
    delay(1000);
}

void taskGprsLoop(void *pv)
{
    if (!modem.begin()) {
        logSd("GPRS init failed");
        if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[1] =
"Conn: NO"; g_pg[2].row[5] = "Evt: NO_MODEM"; xSemaphoreGive(mTft);
        }
        vTaskDelete(NULL);
    }
}

```

```

    if (strlen(GPRS_SIM_PIN) > 0 && !modem.simUnlock(GPRS_SIM_PIN)) {
        logSd("SIM unlock failed");
        if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[5] =
"Evt: SIM_FAIL"; xSemaphoreGive(mTft); }
        vTaskDelete(NULL);
    }

    bool gprsConnected = false;
    for (int retry = 0; retry < 5; retry++) {
        String msg = "GPRS try " + String(retry + 1) + "/5";
        Serial.println("[GPRS] " + msg);
        logSd(("[GPRS] " + msg).c_str());
        if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
            g_pg[2].row[1] = "Conn: NO"; g_pg[2].row[2] = msg;
g_pg[2].row[5] = "Evt: CONNECTING";
            xSemaphoreGive(mTft);
        }
        if (modem.gprsConnect(GPRS_APN, GPRS_USER, GPRS_PASS)) {
            gprsConnected = true;
            String ip = modem.getLocalIP();
            Serial.println("[GPRS] Connected! IP=" + ip);
            logSd(("GPRS connected IP=" + ip).c_str());
            if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
                g_pg[2].row[0] = "GPRS Status"; g_pg[2].row[1] = "Conn:
YES";
                g_pg[2].row[2] = "IP: " + ip; g_pg[2].row[5] = "Evt:
CONNECTED";
                xSemaphoreGive(mTft);
            }
            if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { g_gprs.connected =
true; g_gprs.ip = ip; xSemaphoreGive(mGprs); }
            break;
        }
        vTaskDelay(pdMS_TO_TICKS(5000));
    }

    if (!gprsConnected) {
        logSd("GPRS: Failed to connect");
        if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
            g_pg[2].row[1] = "Conn: NO"; g_pg[2].row[2] = "IP: -";
g_pg[2].row[5] = "Evt: CONN_FAIL";
            xSemaphoreGive(mTft);
        }
    }

    uint32_t lastTsSend = 0;
    uint32_t lastDbSend = 0;

```

```

    for (;;) {
        if (!gprsConnected || !modem.isGprsConnected()) {
            Serial.println("[GPRS] Reconnecting...");
            if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { g_gprs.connected =
false; xSemaphoreGive(mGprs); }
            if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[1] =
"Conn: NO"; g_pg[2].row[5] = "Evt: RECONNECTING";
xSemaphoreGive(mTft); }
            if (modem.gprsConnect(GPRS_APN, GPRS_USER, GPRS_PASS)) {
                String ip = modem.getLocalIP();
                gprsConnected = true;
                Serial.println("[GPRS] Reconnected! IP=" + ip);
                if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
                    g_pg[2].row[1] = "Conn: YES"; g_pg[2].row[2] = "IP: " +
ip; g_pg[2].row[5] = "Evt: RECONNECTED";
                    xSemaphoreGive(mTft);
                }
                if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { g_gprs.connected
= true; g_gprs.ip = ip; xSemaphoreGive(mGprs); }
            } else {
                gprsConnected = false;
                if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[1] =
"Conn: NO"; g_pg[2].row[5] = "Evt: RCNN_FAIL"; xSemaphoreGive(mTft);
            }
                vTaskDelay(pdMS_TO_TICKS(2000)); continue;
            }
        }

        // ThingSpeak – sendThingSpeakGprs sudah handle mGsmTcp di
        // dalamnya
        if (millis() - lastTsSend >= TS_INTERVAL_MS) {
            sendThingSpeakGprs();
            lastTsSend = millis();
        }

        // DB Server – FIX V1.9: wrap dengan mGsmTcp
        if (millis() - lastDbSend >= GPRS_INTERVAL_MS) {
            if (xSemaphoreTake(mGsmTcp, pdMS_TO_TICKS(5000)) == pdTRUE) {
                gsmClient.stop();
                vTaskDelay(pdMS_TO_TICKS(200));

                Payload payload; buildDataPayload(payload);
                String event = "HTTP_FAIL", responseLine = "-";
                if (gsmClient.connect(GPRS_HOST, GPRS_PORT)) {
                    String url = String(GPRS_PATH) + "?api_key=" +
GPRS_API_KEY;

```

```

        gsmClient.print(String("POST ") + url + " HTTP/1.1\r\n");
        gsmClient.print(String("Host: ") + GPRS_HOST + "\r\n");
        gsmClient.print("Content-Type: application/octet-
stream\r\n");
        gsmClient.print("Content-Length: " +
String(sizeof(Payload)) + "\r\n");
        gsmClient.print("Connection: close\r\n\r\n");
        gsmClient.write((uint8_t *)&payload, sizeof(payload));
        unsigned long timeout = millis();
        while (gsmClient.available() == 0) { if (millis() -
timeout > 5000) break; }
        if (gsmClient.available()) {
            String response = gsmClient.readString();
            event = (response.indexOf("OK") >= 0) ? "DB_OK" :
"DB_FAIL";
            responseLine = response.substring(0, min(50,
(int)response.length()));
        }
        gsmClient.stop();
    }

    xSemaphoreGive(mGsmTcp);

    Serial.println("[GPRS] Server: " + event);
    logSd(("GPRS DB: " + event).c_str());
    String currentIp = "";
    if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { currentIp =
g_gprs.ip; xSemaphoreGive(mGprs); }
    if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
        g_pg[2].row[3] = "Last: " + String(millis() / 1000) + "s";
        g_pg[2].row[4] = "Resp: " + (responseLine.length() > 10 ?
responseLine.substring(0, 10) : responseLine);
        g_pg[2].row[5] = "Evt: " + event;
        xSemaphoreGive(mTft);
    }
    if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { g_gprs.lastEvt =
event; g_gprs.lastResp = responseLine; xSemaphoreGive(mGprs); }
    } else {
        logSd("[GPRS] DB: mGsmTcp timeout - skip");
    }
    lastDbSend = millis();
}

vTaskDelay(pdMS_TO_TICKS(100));
}
}

```

```

// =====
// sendImmediateGprsPayload
// FIX V1.9: Ambil mGsmTcp sebelum connect TCP
//          Jika mutex tidak dapat dalam 2 detik, skip
//          (jangan block taskIr/taskAntiCheat terlalu lama)
// =====
void sendImmediateGprsPayload(bool isCheat)
{
    if (!modem.isGprsConnected()) {
        logSd("IMMEDIATE GPRS: Not connected");
        if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[5] =
"Evt: NO_GPRS"; xSemaphoreGive(mTft); }
        return;
    }

    // FIX V1.9: Minta mutex TCP - timeout 2 detik agar task IR tidak
    lama nunggu
    if (xSemaphoreTake(mGsmTcp, pdMS_TO_TICKS(2000)) != pdTRUE) {
        logSd("IMM GPRS: mGsmTcp timeout - skip");
        Serial.println("[IMM] mGsmTcp timeout - skip");
        return;
    }

    gsmClient.stop();
    vTaskDelay(pdMS_TO_TICKS(200));

    Payload payload; buildDataPayload(payload);
    const char* path = isCheat ? GPRS_CHEAT_PATH : GPRS_PATH;
    if (gsmClient.connect(GPRS_HOST, GPRS_PORT)) {
        String url = String(path) + "?api_key=" + GPRS_API_KEY;
        gsmClient.print("POST " + url + " HTTP/1.1\r\n");
        gsmClient.print("Host: " + String(GPRS_HOST) + "\r\n");
        gsmClient.print("Content-Type: application/octet-stream\r\n");
        gsmClient.print("Content-Length: " + String(sizeof(Payload)) +
"\r\n");
        gsmClient.print("Connection: close\r\n\r\n");
        gsmClient.write((uint8_t*)&payload, sizeof(payload));
        String response = "";
        unsigned long start = millis();
        while (millis() - start < 5000 && gsmClient.available() == 0)
vTaskDelay(1);
        while (gsmClient.available()) response +=
(char)gsmClient.read();
        gsmClient.stop();

        String event = (response.indexOf("OK") >= 0) ? (isCheat ?
"CHEAT_OK" : "IR_DB_OK") : (isCheat ? "CHEAT_FAIL" : "IR_DB_FAIL");

```

```

    logSd(("IMM GPRS: " + event).c_str());
    String currentIp = "";
    if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { currentIp =
g_gprs.ip; xSemaphoreGive(mGprs); }
    if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[2] = "IP:
" + currentIp; g_pg[2].row[5] = "Evt: " + event;
xSemaphoreGive(mTft); }
    if (xSemaphoreTake(mGprs, MTX) == pdTRUE) { g_gprs.lastEvt =
event; xSemaphoreGive(mGprs); }
    } else {
        logSd("IMM GPRS: TCP failed");
        if (xSemaphoreTake(mTft, MTX) == pdTRUE) { g_pg[2].row[5] =
"Evt: CONN_FAIL"; xSemaphoreGive(mTft); }
    }

    // FIX V1.9: Release mutex TCP
    xSemaphoreGive(mGsmTcp);
}
#endif

// =====
// IR TASK
// FIX V1.6: Set g_irHitPendingTS = true saat HIT
// =====
void taskIr(void *pv)
{
    for (;;) {
        if (IrReceiver.decode()) {
            if (IrReceiver.decodedIRData.protocol == NEC) {
                uint16_t a = IrReceiver.decodedIRData.address;
                uint8_t c = IrReceiver.decodedIRData.command;
                if (xSemaphoreTake(mIr, MTX) == pdTRUE) {
                    g_ir.got = true; g_ir.addr = a; g_ir.cmd = c; g_ir.t =
millis();
                    g_ir.hitCount++; irPend = true; irAddr = a; irCmd = c;
irSend = true;
                    xSemaphoreGive(mIr);
                }
                // FIX V1.6: Set flag persistent untuk ThingSpeak
                g_irHitPendingTS = true;
                Serial.println("[IR] g_irHitPendingTS = true (HIT
detected)");

                char buf[48]; snprintf(buf, 48, "[IR] HIT Addr:0x%04X
Cmd:0x%02X", a, c);
                logSd(buf); Serial.println(buf);
                uint32_t hc = 0;

```

```

        if (xSemaphoreTake(mIr, MTX) == pdTRUE) { hc =
g_ir.hitCount; xSemaphoreGive(mIr); }
        char tBuf[10]; snprintf(tBuf, 10, "%02lu:%02lu", (millis() /
60000) % 60, (millis() / 1000) % 60);
        if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
            g_pg[3].row[0] = "IR Hit Info";
            g_pg[3].row[1] = "Shooter: 0x" + String(a, HEX);
            g_pg[3].row[2] = "Sub: 0x" + String(c, HEX);
            g_pg[3].row[3] = "Loc: VEST";
            g_pg[3].row[4] = "Time: " + String(tBuf);
            g_pg[3].row[5] = "Hits: " + String(hc);
            xSemaphoreGive(mTft);
        }
        #if ENABLE_GPRS
        for (uint8_t i = 0; i < IR_IMMEDIATE_SEND; i++) {
sendImmediateGprsPayload(false); vTaskDelay(pdMS_TO_TICKS(200)); }
        #endif
    }
    IrReceiver.resume();
}
    if (xSemaphoreTake(mTft, MTX) == pdTRUE) {
        if (xSemaphoreTake(mIr, MTX) == pdTRUE) {
            if (!g_ir.got) { g_pg[3].row[1] = "Shooter: -";
g_pg[3].row[2] = "Sub: -"; g_pg[3].row[3] = "Loc: NONE";
g_pg[3].row[4] = "Time: --:--"; }
            xSemaphoreGive(mIr);
        }
        xSemaphoreGive(mTft);
    }
    vTaskDelay(pdMS_TO_TICKS(100));
}
}

// =====
// SD TASK
// =====
#if ENABLE_SD
void taskSd(void *pv)
{
    vTaskDelay(pdMS_TO_TICKS(3000));
    bool ok = false;
    if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(500)) == pdTRUE) {
        ok = SD.begin(SD_CS, SPI, 8000000);
        xSemaphoreGive(mSpi);
    }
    if (!ok) { Serial.println("[SD] Init fail"); vTaskDelete(NULL); }
    sdOK = true;
}

```

```

    Serial.printf("[SD] SDcard = Connected %luMb\n", (unsigned
long)(SD.cardSize() / (1024 * 1024)));
    if (SD.exists("/tracker.log")) SD.remove("/tracker.log");
    // gps_log.csv tidak dihapus saat reboot

    uint32_t tW = 0;
    for (;;) {
        if (millis() - tW >= SD_WRITE_MS) {
            if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(100)) == pdTRUE) {
                File f = SD.open("/tracker.log", FILE_APPEND);
                if (f) {
                    char *msg;
                    while (xQueueReceive(qLog, &msg, 0) == pdTRUE) {
f.println(msg); delete[] msg; }
                    f.close();
                }
                xSemaphoreGive(mSpi);
            }
            tW = millis();
        }
        vTaskDelay(pdMS_TO_TICKS(100));
    }
}
#endif

// =====
// HELPERS
// =====
void logSd(const char* msg)
{
    #if ENABLE_SD
        char* c = new char[strlen(msg) + 1]; strcpy(c, msg);
        if (xQueueSendToBack(qLog, &c, pdMS_TO_TICKS(10)) != pdTRUE)
delete[] c;
    #endif
}

void sdSave(float lat, float lng, float alt, uint8_t sat, int16_t
rssi, float snr, const char* ir)
{
    #if ENABLE_SD
        if (!sdOK) return;
        if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(100)) == pdTRUE) {
            File f = SD.open("/offline_queue.csv", FILE_APPEND);
            if (f) {

```



```

        char ln[200]; snprintf(ln, 200,
"%lu,%.6f,%.6f,%.1f,%u,%d,%.1f,%s", millis(), lat, lng, alt, sat,
rssi, snr, ir);
        f.println(ln); f.close();
    }
    xSemaphoreGive(mSpi);
}
#endif
}

bool sdUpload()
{
    #if ENABLE_WIFI
    if (!sdOK || WiFi.status() != WL_CONNECTED) return false;
    bool ex = false;
    if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(200)) == pdTRUE) { ex =
SD.exists("/offline_queue.csv"); xSemaphoreGive(mSpi); }
    if (!ex) return false;
    int okCnt = 0; String fail = "";
    if (xSemaphoreTake(mSpi, pdMS_TO_TICKS(200)) == pdTRUE) {
        File f = SD.open("/offline_queue.csv", FILE_READ);
        if (!f) { xSemaphoreGive(mSpi); return false; }
        while (f.available()) {
            String ln = f.readStringUntil('\n'); ln.trim(); if
(ln.length() < 10) continue;
            int c[7], cc = 0; for (size_t i = 0; i < ln.length() && cc <
7; i++) if (ln.charAt(i) == ',') c[cc++] = i;
            if (cc < 6) continue;
            float lat = ln.substring(c[0]+1, c[1]).toFloat(), lng =
ln.substring(c[1]+1, c[2]).toFloat(), alt = ln.substring(c[2]+1,
c[3]).toFloat();
            uint8_t sat = ln.substring(c[3]+1, c[4]).toInt();
            int16_t rssi = ln.substring(c[4]+1, c[5]).toInt(); float snr =
ln.substring(c[5]+1, c[6]).toFloat();
            String ir = ln.substring(c[6]+1);
            xSemaphoreGive(mSpi);
            HTTPClient http; http.begin(VERCEL_API_URL);
            http.addHeader("Content-Type", "application/json");
            http.setTimeout(10000);
            char pl[400]; snprintf(pl, 400,

"{\"source\":\"wifi_sd\",\"id\":\"%s\",\"lat\":%.6f,\"lng\":%.6f,\"a
lt\":%.1f,\"irStatus\":\"%s\",\"satellites\":%u,\"rssi\":%d,\"snr\":
%.1f}",
            DEVICE_ID, lat, lng, alt, ir.c_str(), sat, rssi, snr);
            int code = http.POST(pl); http.end();

```

```

        if (code == 200 || code == 201) okCnt++; else fail += ln +
"\n";
        vTaskDelay(pdMS_TO_TICKS(500));
        xSemaphoreTake(mSpi, pdMS_TO_TICKS(200));
    }
    f.close();
    if (fail.length() == 0) SD.remove("/offline_queue.csv");
    else { File fw = SD.open("/offline_queue.csv", FILE_WRITE); if
(fw) { fw.print(fail); fw.close(); } }
    xSemaphoreGive(mSpi);
}
return okCnt > 0;
#else
return false;
#endif
}

String decodeState(int16_t r)
{
    switch (r) {
        case RADIOLIB_ERR_NONE:                return "OK";
        case RADIOLIB_ERR_CHIP_NOT_FOUND:      return "NO_CHIP";
        case RADIOLIB_ERR_NETWORK_NOT_JOINED:  return "NOT_JOIN";
        case RADIOLIB_ERR_NO_JOIN_ACCEPT:      return "NO_ACCEPT";
        case RADIOLIB_LORAWAN_NEW_SESSION:     return "NEW_SESS";
        case RADIOLIB_LORAWAN_SESSION_RESTORED: return "RESTORED";
        default: return "ERR(" + String(r) + ")";
    }
}
}

```